



universidad
de león



Escuela de Ingenierías Industrial, Informática y Aeroespacial

GRADO EN INGENIERÍA AEROESPACIAL

Trabajo de Fin de Grado

Control híbrido en cascada de un cuadricóptero: sustitución neuronal de controladores especializados del lazo externo

Cascaded hybrid control of a quadcopter: neural replacement of
specialized outer-loop controllers

Autor: Christian González Pérez

Tutores: José Ramón Rodríguez Ossorio

Guzmán González Mateos

Junio, 2026

This page intentionally left blank.

Agradecimientos

Quiero comenzar agradeciendo a mi familia por haber estado siempre ahí y por haberme permitido estudiar en la universidad. Su apoyo constante ha sido la base para afrontar este esfuerzo.

A mi compañera Alba, por el trabajo conjunto y por el apoyo durante el primer semestre.

A mi compañero Guillermo, por todo el apoyo durante estos cuatro años. Gracias a él he participado en actividades y montado proyectos que, sin su impulso, no hubieran salido adelante.

*A mis amigos Alejandro y Alberto, compañeros de **refinement** continuo. Gracias a ellos formamos una trífeca con la que navegamos las novedades tecnológicas mes tras mes, perfeccionando nuestro estilo, nuestros procedimientos y nuestra forma de mejorar. Porque no sumamos tres, multiplicamos ocho. Gracias por todas las largas horas peleándonos con Gemini CLI u Opencode para conseguir que colocase una imagen en LaTeX exactamente como queríamos; por todos los intentos con herramientas chinas para convertir imágenes en texto seleccionable; por los experimentos creando agentes personalizados; por el trabajo con conexiones MCP y con CLI para ampliar las capacidades de la inteligencia artificial; y por todos los intentos de lograr aquello que nos decían que era imposible.*

A mis tutores del TFG, Joserra y Guzmán, por permitirme desarrollar el trabajo que deseaba y por darme margen para hacerlo con herramientas de vanguardia, aprovechando todos los conocimientos que había ido adquiriendo. Su confianza hizo posible que este proyecto fuera muy ambicioso, coherente con mi manera de avanzar, experimentar y desarrollar.

A mi profesor Andrés Merino, de Meteorología Aeronáutica, por no solo permitir, sino motivar, que en la parte práctica de su asignatura se utilizase IA generativa en programación para alcanzar resultados mucho más avanzados. Todo ello se apoyaba en la misma teoría que habría sustentado un trabajo convencional, pero gracias a las herramientas pudimos construir simuladores más complejos, montar estaciones meteorológicas y crear aplicaciones desplegadas en la nube dignas de ser presentadas en concursos. Gracias por permitirme desarrollar mis capacidades en un entorno real de ingeniería y por darme espacio para perfeccionar mis técnicas, mis procedimientos y mis conocimientos sobre cómo gestionar estas herramientas.

Y, finalmente, a todas las personas que me escuchan, que me apoyan y que están ahí en los momentos más complicados. A todas las personas que me dijeron “pruébalo”, “inténtalo”, “tú puedes”. Gracias, porque demostramos que sí era posible.

This page intentionally left blank.

UNIVERSIDAD DE LEÓN
Escuela de Ingenierías
Industrial, Informática y Aeroespacial
GRADO EN INGENIERÍA AEROESPACIAL
Trabajo de Fin de Grado

ESTUDIANTE: Christian González Pérez

TUTORES: José Ramón Rodríguez Ossorio
Guzmán González Mateos

TÍTULO: Control híbrido en cascada de un cuadricóptero: sustitución neuronal de controladores especializados del lazo externo

TITLE: Cascaded hybrid control of a quadcopter: neural replacement of specialized outer-loop controllers

CONVOCATORIA: Junio, 2026

RESUMEN:

Este trabajo estudia si un controlador neuronal entrenado por imitación puede sustituir a un conjunto de controladores proporcional-derivativos (PD) especializados en el seguimiento de trayectorias de un cuadricóptero simulado. Para ello se desarrolla un entorno reproducible de simulación que modela la dinámica de cuerpo rígido en sistemas de referencia ENU/FRD, la actitud mediante cuaterniones, actuadores con saturación y retardo, perturbaciones y frecuencias diferenciadas de física, control y telemetría. Sobre este entorno se implementa una arquitectura en cascada con un lazo externo que genera la fuerza deseada y un lazo interno clásico que conserva la estabilización de actitud y la asignación a rotores. Primero se sintonizan controladores PD especializados para familias de trayectorias **hold**, círculo, Lissajous y **waypoint**; después se usan como expertos para generar datos de imitación. Se entrenan y comparan tres arquitecturas, MLP, GRU y LSTM, con las mismas entradas, objetivos y procedimiento. La evaluación distingue fidelidad supervisada, seguimiento en bucle cerrado dentro de distribución, transferencia clásica entre familias y escenarios fuera de distribución con variaciones, composiciones, y trayectorias nuevas. Los resultados muestran que la imitación neuronal es viable y puede sustituir a varios PD especializados en condiciones conocidas y en algunas variaciones no vistas. Sin embargo, no se observa una superioridad global de las redes. GRU y LSTM aproximan mejor la fuerza en evaluación supervisada, pero esa ventaja no se traduce de forma sistemática en menor RMSE de posición; MLP resulta competitivo en varias familias y LSTM se degrada en trayectorias geométrica o dinámicamente exigentes. El estudio concluye que la calidad del seguimiento depende de la arquitectura, de la distribución de entrenamiento, de los límites de actuación y del lazo clásico compartido, por lo que la generalización requiere un mayor nivel de evaluaciones.

ABSTRACT:

This work studies whether an imitation-trained neural controller can replace a set of specialized proportional-derivative (PD) controllers for trajectory tracking of a simulated quadrotor. For this purpose, a reproducible simulation environment is developed that models rigid-body dynamics in ENU/FRD reference frames, attitude using quaternions, actuators with saturation and lag, disturbances, and differentiated frequencies for physics, control, and telemetry. On this environment, a cascaded architecture is implemented with an outer loop that generates the desired force and a classical inner loop that preserves attitude stabilization and rotor allocation. First, specialized PD controllers are tuned for the **hold**, **circle**, **Lissajous**, and **waypoint** trajectory families; they are then used as experts to generate imitation data. Three architectures, MLP, GRU, and LSTM, are trained and compared with the same inputs, targets, and procedure. The evaluation distinguishes supervised fidelity, in-distribution closed-loop tracking, classical transfer between families, and out-of-distribution scenarios with variations, compositions, and new trajectories. The results show that neural imitation is feasible and can replace several specialized PD controllers in known conditions and in some unseen variations. However, no global superiority of the networks is observed. GRU and LSTM approximate the force better in supervised evaluation, but this advantage does not systematically translate into lower position RMSE; MLP is competitive in several families, and LSTM degrades in geometrically or dynamically demanding trajectories. The study concludes that tracking quality depends on the architecture, the training distribution, the actuation limits, and the shared classical loop, which is why generalization requires a higher level of evaluations.

Palabras clave: cuadricóptero, simulación 6DOF, control PD, aprendizaje por imitación, redes neuronales.

Firma del estudiante:**V.º B.º Tutor/es:**

Índice

Índice de figuras	VII
Índice de tablas	X
Abreviaturas y acrónimos	XI
Símbolos y unidades	XIII
Declaración de uso de IA generativa	XVII
1 Introducción	1
1.1 Motivación	1
1.2 Hipótesis y objetivos	3
1.3 Estructura del documento	4
2 Estado del arte	5
2.1 Simulación y modelado de cuadricópteros	5
2.2 Entornos de programación y aprendizaje automático	7
2.3 Control clásico en cascada y programación de ganancias	8
2.4 Aprendizaje por imitación y control híbrido	9
2.5 Redes neuronales para control	10
2.5.1 Redes feedforward	11
2.5.2 Redes recurrentes	12
2.6 Evaluación, generalización y métricas	15
2.7 Posicionamiento del trabajo	16
3 Metodología	17
3.1 Modelo físico y simulador 6DOF	17
3.2 Control clásico en cascada	28
3.3 Control neuronal por imitación	34
3.4 Conjuntos de datos y métricas de evaluación	39
3.5 Flujo experimental y reproducibilidad	40
4 Resultados experimentales	44
4.1 Cobertura y validez de la ejecución	44
4.2 Referencia clásica y transferencia entre familias	44

4.3	Aprendizaje neuronal en condiciones conocidas	45
4.4	Generalización fuera de distribución	46
4.5	Caso representativo de arquitectura	49
4.6	Actuación, protecciones y límites compartidos	50
4.7	Discusión de resultados y limitaciones	50
5	Conclusiones y trabajo futuro	54
5.1	Conclusiones	54
5.2	Alcance y limitaciones	55
5.3	Trabajo futuro	57
	Bibliografía	59
	Anejo 1: Comandos y trazabilidad experimental	62
	Anejo 2: Arquitectura del código y fragmentos relevantes	68
	Anejo 3: Evidencias visuales de simulación	72

Índice de figuras

1.1	Esquema conceptual del flujo experimental del trabajo. Consta de la generación de trayectorias, especialización de controladores clásicos, entrenamiento de redes neuronales y evaluación comparativa. Fuente: elaboración propia.	2
2.1	Mecanismos de actuación y movimiento en un cuadricóptero. Para cada maniobra se muestra la vista superior del vehículo en planta (izquierda) con la dirección de giro de cada rotor e indicaciones de desequilibrio de empuje (+, -, =), y el efecto físico resultante en vista lateral (derecha). La inclinación del vehículo (ϕ , θ) desvía el vector de empuje total (T) generando las componentes de fuerza lateral (F_y) o longitudinal (F_x) que posibilitan la traslación horizontal. Fuente: elaboración propia.	6
2.2	Comparativa conceptual de arquitecturas neuronales para control. En (A) se muestra el carácter puramente estático de la MLP. En (B) se detalla el flujo desenrollado de la RNN básica y su vulnerabilidad al desvanecimiento del gradiente. En (C) y (D) se contrasta la estructura selectiva con compuertas de la célula LSTM y la célula GRU. Fuente: elaboración propia.	13
3.1	Sistemas de referencia empleados por el simulador. El sistema mundo sigue la convención ENU y el sistema cuerpo la convención FRD; el empuje total se aplica en la dirección opuesta al eje vertical del cuerpo (z_B). Fuente: elaboración propia.	17
3.2	Configuración en X de los motores y sentidos de giro del rotor en el sistema de referencia cuerpo. La dirección física de los rotores se asocia con el signo del momento de reacción en guiñada. Fuente: elaboración propia.	18
3.3	Esquema lógico y toma de decisiones del mezclador de rotores (mixer) ante condiciones nominales y saturadas. Se prioriza el control de actitud mediante el recorte proporcional de momentos (k) frente a la desviación del empuje colectivo solicitado. Fuente: elaboración propia.	21
3.4	Familias de trayectorias analíticas del simulador. Cada geometría introduce estímulos diferentes (estacionario, velocidad variable, aceleración acoplada en 3 ejes o tramos discretos). Fuente: elaboración propia.	24

3.5	Perfiles cinemáticos generados en las transiciones entre waypoints. A la izquierda se ilustra el perfil trapezoidal (para tramos largos) y a la derecha el perfil triangular (para tramos cortos, donde no se alcanza la velocidad de consigna). Todas las magnitudes representadas son escalares sobre el eje de avance del tramo. Fuente: elaboración propia.	25
3.6	Secuencia de ejecución temporal multirate y dependencias de datos. Se diferencian las tres bandas de frecuencia asociadas a la integración física (100 Hz), el ciclo digital de control (50 Hz) y el registro periódico de variables (10 Hz). Fuente: elaboración propia.	26
3.7	Arquitectura del controlador clásico en cascada implementado. Se destacan el lazo externo (posición en mundo ENU), la conversión geométrica de fuerza a actitud y el lazo interno (estabilización de actitud en cuerpo FRD). La frontera de sustitución neuronal se sitúa en la señal de fuerza deseada $\mathbf{F}_{d,W}$. Fuente: elaboración propia.	28
3.8	Flujo lógico de optimización y ajuste progresivo de ganancias clásicas. Los candidatos aleatorios globales se filtran por condiciones de estabilidad y seguridad antes de que se inicie el refinamiento local tipo simplex/Nelder–Mead. Fuente: elaboración propia.	33
3.9	Estructura del controlador neuronal híbrido. La red neuronal sustituye únicamente la decisión traslacional del lazo externo para generar $\mathbf{F}_{d,W}$, delegando el filtrado de seguridad, la conversión fuerza-actitud y la estabilización en el lazo clásico interno. Fuente: elaboración propia. . .	34
3.10	Gestión de la ventana temporal deslizante en redes recurrentes e inicialización mediante repetición de muestra en el arranque ($k = 0$). La cola FIFO genera un lote de dimensiones $[1, 20, 9]$ en cada inferencia. Fuente: elaboración propia.	37
3.11	Flujo general del procedimiento experimental estructurado en 10 fases reproducibles. Se ilustran la generación progresiva de los datasets de entrenamiento, el flujo de optimización clásico, el entrenamiento neuronal y las etapas de validación y evaluación OOD. Fuente: elaboración propia.	41
3.12	Clasificación de los niveles de evaluación y generalización experimental. Se estructuran cuatro niveles con novedad y exigencia incremental. Interpolación ID nominal, transferencia clásica, recombinación de capacidades conocidas y geometrías OOD. Fuente: elaboración propia.	42
4.1	Transferencia cruzada de los controladores PD en el conjunto de test . Las filas indican la familia usada para la sintonización y las columnas la familia evaluada. La diagonal marca el uso del controlador especializado en su familia. Fuente: evaluación propia	45

4.2	RMSE medio en condiciones conocidas para el PD representativo y las tres arquitecturas neuronales. Se muestra únicamente el seguimiento, por ser la métrica principal en esta comparación.	46
4.3	RMSE medio fuera de distribución por familia de escenario.	47
4.4	Desglose OOD por escenario y controlador para las cuatro referencias principales. El número de cada celda es el RMSE en metros; el asterisco indica que la ejecución no se completó aunque el episodio se conserva en la agregación.	48
4.5	Tipos de terminación en la serie OOD para los controladores principales.	48
4.6	Escenario OOD <code>lemniscate_fast_center_yaw</code> : comparación visual entre MLP y LSTM. El panel superior muestra el plano horizontal y el inferior el error de posición a lo largo del episodio.	49
4.7	Activación media de protecciones y límites de actuación en OOD. La figura separa error de seguimiento, demanda de control y restricciones compartidas por la arquitectura de control.	50
5.1	Muestra visual de familias conocidas.	73
5.2	Muestra visual de trayectorias fuera de distribución.	74
5.3	Vista tridimensional de una hélice OOD representativa. Fuente: elaboración propia.	75
5.4	Seguimiento geométrico de dos PD especializados del dataset clásico: caso estacionario perturbado y trayectoria Lissajous multieje. Fuente: elaboración propia.	76
5.5	Seguimiento geométrico en una referencia circular perturbada y en una trayectoria por waypoints del split de prueba. Fuente: elaboración propia.	77
5.6	Relación visual entre error de seguimiento y actitud en dos casos del dataset clásico con perturbaciones. Fuente: elaboración propia.	77
5.7	Evolución temporal de la posición frente a la referencia en dos familias con lectura geométrica diferente. Fuente: elaboración propia.	78
5.8	Señales de actuación inspeccionadas tras la simulación cerrada con PD experto nativo. Fuente: elaboración propia.	79
5.9	Velocidades angulares del vehículo en cuerpo para dos escenarios de seguimiento del dataset clásico. Fuente: elaboración propia.	79
5.10	Inspección visual de una transferencia cruzada en el escenario circular.	80

Índice de tablas

4.1	Cobertura de la ejecución comparativa.	44
4.2	Fidelidad supervisada de fuerza y recorte de inclinación asociado. Los porcentajes de recorte corresponden a la activación del límite de inclinación antes de aplicar la fuerza en el lazo interno.	46
5.1	Casos del dataset clásico usados para las figuras de telemetría.	76

Abreviaturas y acrónimos

6DOF Seis grados de libertad.

ALVINN Vehículo terrestre autónomo en una red neuronal (*Autonomous Land Vehicle in a Neural Network*).

BPTT Retropropagación a través del tiempo (*Backpropagation Through Time*).

CCW Sentido de giro antihorario (*Counter-Clockwise*).

CM Centro de masas.

CPU Unidad central de procesamiento (*Central Processing Unit*).

CSV Valores separados por comas (*Comma-Separated Values*).

CW Sentido de giro horario (*Clockwise*).

Dagger Agregación de conjuntos de datos (*Dataset Aggregation*).

ENU Sistema de referencia local East–North–Up.

FIFO Primero en entrar, primero en salir (*First In, First Out*).

FRD Sistema de referencia de cuerpo Forward–Right–Down.

GNSS Sistema global de navegación por satélite (*Global Navigation Satellite System*).

GPU Unidad de procesamiento gráfico (*Graphics Processing Unit*).

GRU Unidad recurrente con compuertas (*Gated Recurrent Unit*).

ID Dentro de la distribución de entrenamiento (*in-distribution*).

IMU Unidad de medida inercial (*Inertial Measurement Unit*).

JSON Notación de objetos de JavaScript (*JavaScript Object Notation*).

LSTM Memoria a corto y largo plazo (*Long Short-Term Memory*).

MAE Error absoluto medio (*Mean Absolute Error*).

MLP Perceptrón multicapa (*Multilayer Perceptron*).

MSE Error cuadrático medio (*Mean Squared Error*).

NaN No es un número (*Not a Number*).

OOD Fuera de la distribución de entrenamiento (*out-of-distribution*).

PD Controlador proporcional–derivativo.

PID Controlador proporcional–integral–derivativo.

ReLU Unidad lineal rectificadora (*Rectified Linear Unit*).

RK4 Método de Runge–Kutta de cuarto orden.

RMS Raíz del valor cuadrático medio (*Root Mean Square*).

RMSE Raíz del error cuadrático medio (*Root Mean Square Error*).

RNN Red neuronal recurrente (*Recurrent Neural Network*).

TOML Formato de configuración minimalista (*Tom’s Obvious Minimal Language*).

UAV Vehículo aéreo no tripulado (*Unmanned Aerial Vehicle*).

YAML Formato de serialización de datos (*YAML Ain’t Markup Language*).

ZOH Bloqueador de orden cero (*Zero-Order Hold*).

Símbolos y unidades

Salvo indicación contraria, las magnitudes vectoriales de posición, velocidad, aceleración y fuerza se expresan en el sistema mundo ENU, y las magnitudes de actitud, giro, momento e inercia se expresan en el sistema cuerpo FRD.

Sistemas de referencia y estado

Símbolo	Significado	Unidad
$\mathcal{W}$	Sistema de referencia mundo, con convención ENU.	–
$\mathcal{B}$	Sistema de referencia cuerpo, con convención FRD.	–
x_W, y_W, z_W	Ejes del sistema mundo: este, norte y arriba.	–
x_B, y_B, z_B	Ejes del sistema cuerpo: delante, derecha y abajo.	–
$\mathbf{x}$	Estado del vehículo en la simulación.	–
$\mathbf{p}_W, \mathbf{p}$	Posición del centro de masas en el sistema mundo.	m
$\mathbf{v}_W, \mathbf{v}$	Velocidad lineal en el sistema mundo.	m/s
$\mathbf{p}_r, \mathbf{v}_r$	Posición y velocidad de referencia.	m; m/s
$\mathbf{a}, \mathbf{a}_r$	Aceleración y aceleración de referencia.	m/s ²
$\mathbf{a}_{cmd}$	Aceleración comandada por el lazo externo.	m/s ²
$\mathbf{v}_{w,W}$	Velocidad del viento constante en el sistema mundo.	m/s
$\mathbf{v}_{r,B}$	Velocidad relativa respecto al viento expresada en cuerpo.	m/s

Actitud, orientación y rotación

Símbolo	Significado	Unidad
$\mathbf{q}_{WB}$	Cuaternión de actitud que relaciona cuerpo y mundo.	–
$\mathbf{q}_{WB,d}$	Cuaternión de actitud deseada usado por el lazo interno.	–
q_0, q_1, q_2, q_3	Componentes del cuaternión de actitud.	–
$\mathbf{q}_e, \mathbf{q}_{e,v}$	Cuaternión de error y parte vectorial del error de actitud.	–
$\boldsymbol{\omega}_B$	Velocidad angular del vehículo en el sistema cuerpo.	rad/s
$\mathbf{R}_{WB}$	Matriz de rotación de cuerpo FRD a mundo ENU.	–
$\mathbf{R}_{WB,d}$	Matriz de rotación deseada de cuerpo FRD a mundo ENU.	–
$\mathbf{x}_C$	Vector auxiliar horizontal construido a partir de la guiñada de referencia.	–
$\mathbf{x}_{B,d}, \mathbf{y}_{B,d}, \mathbf{z}_{B,d}$	Ejes deseados del sistema cuerpo usados para construir la actitud de referencia.	–
ψ_r	Guiñada de referencia.	rad
$\otimes$	Producto de cuaterniones.	–

Vehículo y parámetros físicos

Símbolo	Significado	Unidad
m	Masa del vehículo.	kg
g	Aceleración de la gravedad.	m/s ²
I_B	Matriz de inercia respecto al centro de masas.	kg m ²
$\mathbf{r}_{i,B}$	Posición del rotor i respecto al centro de masas.	m
x_i, y_i	Coordenadas del rotor i en el sistema cuerpo.	m
s_i	Signo del par de reacción de guiñada del rotor i .	–
τ	Constante de tiempo del lag de primer orden del actuador.	s
M	Matriz de asignación entre empujes de rotores y mando colectivo/momentos.	–
D_B	Matriz de drag lineal expresada en cuerpo.	N/(m/s)

Fuerzas, momentos y actuación

Símbolo	Significado	Unidad
F_B	Fuerza aplicada expresada en cuerpo.	N
$F_{i,B}$	Fuerza generada por el rotor i expresada en cuerpo.	N
$F_{d,W}$	Fuerza con subíndice d en mundo: fuerza deseada en el lazo de control o fuerza de drag en el modelo de perturbaciones, según el contexto.	N
$\hat{F}_{d,W}$	Fuerza deseada predicha por el controlador neuronal.	N
T	Empuje colectivo total.	N
T_i	Empuje generado por el rotor i .	N
$T_{\min}, T_{\max}$	Límites inferior y superior del empuje individual de rotor.	N
$T_{\text{req},i}$	Empuje requerido para el rotor i tras el mixer.	N
T_{target}	Empuje medio objetivo usado en la ruta saturada del mixer.	N
$T_{\text{thrust_part}}$	Contribución de empuje colectivo antes de aplicar momentos.	N
$T_{\text{moment_part}}$	Contribución de momentos antes de aplicar límites de rotor.	N
ω_i	Velocidad angular del rotor i .	rad/s
$\omega_{\max}$	Velocidad angular máxima de rotor usada en los escenarios.	rad/s
rpm	Revoluciones por minuto, unidad auxiliar de lectura para velocidades de rotor en telemetría.	rev/min
k_f	Coefficiente cuadrático de empuje del rotor.	N s ² rad ⁻²
k_m	Coefficiente cuadrático de par de reacción del rotor.	N m s ² /rad ²
Q_i	Par de reacción del rotor i .	N m
τ_B	Momento aplicado en el sistema cuerpo.	N m
τ_x, τ_y, τ_z	Componentes de momento de alabeo, cabeceo y guiñada.	N m
$\delta_{\min}, \delta_{\max}, \delta$	Límites y desplazamiento de holgura del empuje colectivo en el mixer.	N
$\times$	Producto vectorial.	–

Control clásico y métricas

Símbolo	Significado	Unidad
e_p, e_v	Errores de posición y velocidad.	m; m/s
e_q	Error de actitud usado por el lazo interno.	–
$K_{p,p}, K_{d,p}$	Ganancias proporcional y derivativa del lazo de posición.	s^{-2} ; s^{-1}
$K_{p,a}, K_{d,a}$	Ganancias proporcional y derivativa del lazo de actitud.	N m; N m s
J	Puntuación usada para seleccionar controladores PD.	–
e_k	Error escalar de seguimiento en la muestra k .	m
e_{RMSE}, e_{rms}	Raíz del error cuadrático medio de posición.	m
e_{MAE}	Error absoluto medio de posición.	m
$e_{máx}$	Error máximo de posición.	m
e_{att}	RMS de los ángulos de actitud en la puntuación de ajuste.	rad
u	Esfuerzo de control normalizado usado en la puntuación.	–
s, d	Fracciones de muestras con saturación y degradación del mando.	–
$\bar{T}$	Empuje medio usado en el término de esfuerzo de control.	N
N	Número de muestras usadas en una métrica.	–

Tiempo, muestreo y trayectorias

Símbolo	Significado	Unidad
t	Tiempo continuo usado en trayectorias y referencias.	s
k	Índice temporal discreto de una muestra.	–
Δt	Periodo de integración, control o muestreo indicado en cada contexto.	s
$f_{phys}, f_{ctrl}, f_{tel}$	Frecuencias de física, control y telemetría.	Hz
T_p, T_c, T_t	Periodos asociados a física, control y telemetría.	s
L	Longitud de ventana temporal usada por modelos recurrentes; en perfiles de waypoint puede indicar longitud escalar de tramo cuando se especifica en una figura.	muestras; m
c_x, c_y, c_z	Coordenadas del centro de una trayectoria Lissajous.	m
A_x, A_y, A_z	Amplitudes espaciales de una trayectoria Lissajous.	m
$\omega_x, \omega_y, \omega_z$	Frecuencias angulares por eje en una trayectoria Lissajous.	rad/s
$s(t)$	Coordenada escalar de avance usada para generar perfiles entre waypoints.	m

Redes neuronales y optimización

Símbolo	Significado	Unidad
B	Tamaño del lote de entrenamiento o inferencia.	muestras
$\mathbf{x}_k$	Vector de entrada de la red en el instante k .	norm.
$\mathbf{y}_k$	Salida de una red recurrente en el instante k .	norm.
θ	Conjunto de parámetros entrenables de una red neuronal.	—
$\mathcal{L}(\theta)$	Función de pérdida dependiente de los parámetros entrenables.	—
n	Índice de iteración en descenso por gradiente.	—
η	Tasa de aprendizaje en descenso por gradiente.	—
∇_{θ}	Gradiente respecto a los parámetros de la red.	—
$\mathbf{W}, \mathbf{b}$	Pesos y sesgos de una capa neuronal, incluidas matrices y sesgos específicos de capas o compuertas recurrentes.	norm.
$\mathbf{z}^{(\ell)}, \mathbf{a}^{(\ell)}$	Pre-activación y activación de la capa ℓ .	norm.
ℓ	Índice de capa neuronal.	—
$\mathbf{h}_k$	Estado oculto de una red recurrente en el instante k .	—
$\mathbf{c}_k$	Estado de celda de una LSTM en el instante k .	—
$\mathbf{f}_k, \mathbf{i}_k, \mathbf{o}_k$	Compuertas de olvido, entrada y salida de una LSTM.	—
$\tilde{\mathbf{c}}_k$	Estado candidato de celda de una LSTM.	—
$\mathbf{z}_k, \mathbf{r}_k$	Compuertas de actualización y reinicio de una GRU.	—
$\tilde{\mathbf{h}}_k$	Estado oculto candidato de una GRU.	—
$\phi, \sigma, \tanh$	Activación no lineal genérica, sigmoide y tangente hiperbólica; ϕ también puede denotar el ángulo de alabeo cuando el contexto es actitud.	—
$\odot$	Producto componente a componente.	—
$\mathbb{R}^n$	Espacio vectorial real de dimensión n .	—
$\ \cdot\ $	Norma vectorial.	—

Declaración de uso de IA generativa

Declaro de forma explícita que durante este Trabajo de Fin de Grado he usado herramientas de inteligencia artificial generativa. La IA ha formado parte de mi manera de trabajar en este proyecto, siempre como herramienta auxiliar y bajo mi responsabilidad, pero de una forma bastante más profunda que la de copiar un texto en un chat y aceptar una respuesta, como hace la gran mayoría de compañeros que conozco y el uso común que se observa. Es por esto que me quiero extender en esta sección.

Mi relación con estas herramientas empezó mucho antes de este trabajo. En 2021 y 2022 ya seguía con interés los primeros avances en modelos generativos de texto e imagen. Recuerdo generar imágenes en Google Colab, ver resultados todavía imperfectos, pero suficientemente buenos como para intuir que esto era el comienzo de algo mucho más grande y que iba a tener un impacto rompedor. Me fascinaba ver unas dunas, algo parecido a un coche abstracto o una escena todavía rara, pero generada por un ordenador. También conocí modelos de texto como GPT-2, que visto desde ahora era limitado y fallaba mucho, pero ya dejaba ver que podía venir algo importante después.

Después llegaron herramientas más accesibles de texto e imagen, como ChatGPT a finales de 2022. Yo lo viví como uno más, porque llevaba ya tiempo metido en este campo, pero no fue uno más sino que marcó el primer punto de inflexión. A partir de ahí cambió la forma en que mucha gente iba a escribir, buscar, programar, aprender y organizar trabajo. En imágenes ocurrió algo parecido, cada pocos meses aparecían mejoras palpables, hasta el punto en que una imagen creada con IA, pedida con criterio, no puede distinguirse fácilmente de una imagen producida por otros medios. También por eso considero importante que existan mecanismos de identificación digital, como el SynthID de Google e implementado por otras empresas como OpenAI, para mantener la trazabilidad de lo generado y separar en internet el contenido real del generado con IA generativa.

En 2023, 2024 y 2025 seguí viendo cómo la IA generativa dejaba de ser una curiosidad y empezaba a convertirse en una herramienta informática factible. Usaba ChatGPT, Google Bard y después Gemini, Mixtral y otros modelos que iban apareciendo, también algunos modelos chinos que empezaron a demostrar que con optimización se podían lograr capacidades muy serias. Aun así, durante bastante tiempo lo seguí viendo como una herramienta más dentro de mi interés general por la tecnología, igual que podía interesarme por procesadores, tarjetas gráficas, ordenadores o cualquier avance técnico nuevo.

El cambio de visión llegó al final del curso 2024–2025, después de una charla en la Escuela de Ingenierías que me hizo dejar de verlo solo como algo que seguía por gusto. Ahí entendí que tenía que formarme de verdad, porque no era una moda pasajera, sino una competencia que iba a formar parte del trabajo de ingeniería. En junio de 2025 tomé la decisión de aprender a usar estas herramientas con criterio y también de aprender sobre ellas. Mientras empezaba a pensar en este Trabajo de Fin de Grado, hice cursos introductorios de inteligencia artificial y redes neuronales, varios de ellos de Google, básicos pero muy útiles para fijar fundamentos. Esa formación influyó en el tema del TFG y en la idea del control por imitación.

En ese momento las herramientas de programación con IA todavía estaban en una fase muy temprana. Claude Code había aparecido en febrero de 2025 junto con Claude 3.7 Sonnet, pero todavía era algo minoritario, usado por poca gente y más cerca de completar o ayudar con código que de llevar un flujo completo. Los modelos de la familia Claude Sonnet 3, y después Sonnet 3.5 y 3.7, empezaban a ser interesantes para programar, pero todavía no tenían la capacidad de poder convertirse en un compañero de trabajo completo. Codex también empezó a ganar presencia en 2025, primero como agente de código y después con mejoras para programación agéntica y tareas del mundo real. Mientras tanto, yo iba aprendiendo cómo hablar con estas herramientas, cómo pedirles cosas, cómo corregirlas y cómo convertir una idea mía en trabajo verificable.

También empecé a aprender todo el ecosistema que se estaba formando alrededor de los agentes. Aparecieron las *Agent skills*, que son como pequeños prompts o instrucciones reutilizables para reproducir formas de trabajar. Empecé a entender los MCP, *Model Context Protocol*, que permiten conectar una IA con herramientas externas, documentación, búsquedas, repositorios o sistemas locales. También aparecieron más integraciones con editores, sistemas de *hooks* y formas de poner límites al comportamiento de los agentes. Con todas estas herramientas puedes darle contexto, reglas y puertas de control a una IA que actúa de forma libre en una carpeta de tu ordenador.

Durante el curso 2025–2026 fui usando cada trabajo y cada práctica como una oportunidad para aprender a trabajar mejor con IA. Mejoré mi forma de escribir instrucciones, de explicar intención, de pedir revisiones y de distinguir cuándo una respuesta era útil y cuándo simplemente sonaba convincente. También vi mejorar mucho las capacidades de estas herramientas en diseño web, SVG, diagramas y figuras. Esto fue muy relevante para el TFG, ya que todas las figuras son TikZ de LaTeX o código Python, que puede ser modificado, compilado o ejecutado y revisado.

El segundo gran punto de inflexión para mí llegó entre octubre y diciembre de 2025. El uso avanzado ya no eran chatbots, sino agentes capaces de planificar, editar archivos, ejecutar comandos, probar, fallar, corregir y volver a intentar. Google Antigravity apareció en noviembre de 2025 como un entorno de desarrollo desde el inicio para

trabajar con agentes; OpenAI siguió empujando Codex como una herramienta de trabajo agéntico; Claude Code y los modelos Opus se convirtieron en una referencia clara en código; y alrededor fueron apareciendo herramientas para conectar de forma más profunda los agentes con los programas de un ordenador. En 2026 se sumaron también herramientas del ecosistema de xAI, como Grok Build, usadas también en el TFG.

Con todo ese recorrido llegué al desarrollo de este TFG convencido de que iba a usar estas herramientas. No como una forma de escaquearme del trabajo, sino porque era la forma de trabajar para lograr resultados de una calidad y alcance mucho mayor. Igual que no tendría sentido hacer un trabajo en la década de 2010 buscando todo en una enciclopedia en papel y evitando Internet, tampoco me parecía razonable ignorar herramientas que permiten programar, documentar, revisar y experimentar con una velocidad mucho mayor. La clave era construir el proyecto de manera que la IA tuviese contexto, documentación, reglas y una trazabilidad constante.

Por eso este trabajo se desarrolló en mi ordenador, con editores y herramientas preparadas para agentes. He usado Google Antigravity como editor de código con modelos de Gemini integrados, OpenAI Codex y Grok Build, NotebookLM como apoyo para organizar y consultar documentación, chatbots normales como herramienta de búsqueda con esteroides y las herramientas del ecosistema de agentes y código mencionadas previamente. También he usado MCPs para extender las capacidades de la IA, siendo el más relevante Context7 para contar con documentación actualizada de librerías de código.

Lo que quiero dejar claro es el flujo de trabajo, no es “pregunto algo a un chatbot” y lo pego sino “le doy a un agente un objetivo, contexto, reglas, pruebas y una forma de verificar si lo que ha hecho tiene sentido”. Cuando la IA solo recibe un fragmento de código copiado en un chat, no entiende bien el proyecto, no conoce las decisiones anteriores y no puede mantener una trazabilidad. En este TFG he trabajado de otra forma, con un repositorio montado para que la IA tuviese contexto desde el momento uno y actuar como mi compañera de trabajo.

Además, he usado herramientas de reconocimiento de voz para verbalizar ideas y convertirlas en texto, ya que la forma más natural de explicar una intención no es escribirla desde cero, sino hablarla primero y después ordenarla.

En este trabajo la IA se ha usado principalmente para traducir intención a código y documentación. Esto incluye código Python, archivos de configuración, documentación Markdown, pruebas, comandos, scripts auxiliares, LaTeX, TikZ, diagramas y figuras. El código no es solo Python, hay código en una figura hecha con TikZ, en una plantilla LaTeX, un archivo YAML o un registro JSON de telemetría. En todos esos casos la IA ha servido para acelerar la ejecución de decisiones que yo tomaba, revisaba y corregía.

La metodología seguida no ha sido pedir “Hola, hazme el TFG”. El repositorio se pensó desde el primer documento para trabajar con la IA como compañera con especificaciones, planes, revisiones, documentación actualizada, trazabilidad de requisitos, pruebas y registro de decisiones. La técnica que he seguido se conoce como *Spec Driven Development*, antes de realizar una implementación se define una especificación de lo que se quiere conseguir, se discute, se revisan los casos límite y se convierte la intención inicial en un conjunto de requisitos concretos.

La parte de las preguntas y respuestas con el modelo ha sido central. El proceso partía de una idea mía, extensa, pero incompleta. El agente me hacía preguntas, yo respondía, se corregían supuestos, matizaba mis intenciones, y así cuadrando la intención hasta una especificación y plan claros. Así quedaba registrado qué idea se iba a ejecutar, cuáles eran los límites y qué decisiones se tomaron. Tras la implementación he revisado el resultado, comparado salidas, pedido revisiones adicionales y corregido lo que no encajaba. Gracias a la IA se ha iterado más rápido, pero las decisiones del alcance del trabajo, la extensión y criterio físicos, la interpretación de resultados y la dirección general han sido mías.

En el repositorio público se puede observar toda la documentación de soporte que se fue generando para entender qué se estaba haciendo. Hay planes, especificaciones, revisiones, documentación del simulador, tablas de trazabilidad, requisitos, instrucciones de mantenimiento, principios de trabajo y archivos AGENTS.md que explican a los agentes cómo deben comportarse dentro del proyecto. Siempre se dice que hay que dar contexto a la IA, pero nunca leo sobre gente que obligue a la IA a una trazabilidad experimental completa.

La primera iteración del simulador fue el ejemplo más claro de la metodología. Yo no partía de “haz un simulador para el TFG”, sino de una intención concreta: cómo quería que estuvieran organizados los módulos, qué opciones debía tener el programa, qué tipo de escenarios quería, qué datos debían generarse y cómo quería estructurar la ejecución. La IA ayudó a convertir esa intención, expresada en lenguaje natural, en código Python y en archivos de configuración como JSON, YAML o TOML.

Después de esa primera versión no se aceptó lo generado y ya está. Hubo muchos *refinement* para ajustar casos, cambiar parámetros, detectar fallos, mejorar escenarios, revisar salidas, ejecutar pruebas, rehacer partes y volver a comprobar. Se dejaba al agente trabajar en el ordenador: editar archivos, lanzar comandos, ejecutar tests, leer errores y proponer la siguiente corrección. La IA no contestaba en un chat, estaba controlando el entorno de desarrollo para cumplir con un objetivo verificable.

Se creó documentación actualizada del simulador sobre arquitectura, uso, escenarios, telemetría, validación, requisitos, pruebas y estado de cada parte. Esa documentación servía para mí, para cualquier persona que pudiera consultar el proyecto en GitHub

y también para la propia IA, porque le daba contexto sobre el proyecto. Si había una duda sobre un módulo, un escenario, una métrica o una decisión de diseño, la respuesta dependía de la documentación escrita previamente en el repositorio. Así se avanzó de manera ordenada, mediante múltiples iteraciones entre varios agentes, diferenciando entre implementación y revisión, algo que habría sido imposible sin ese contexto.

El código resultante está estructurado de forma modular y con buenas prácticas de software, pero sin ofuscar la parte importante del trabajo, física, aeroespacial y control. Las funciones relacionadas con la dinámica, los sistemas de referencia, la integración, el control o el entrenamiento de redes se han documentado con nombres claros, comentarios y explicaciones.

La IA también se utilizó para crear herramientas auxiliares que no son el núcleo del simulador, pero que facilitan trabajar con él: scripts, comandos, utilidades de análisis, generación de resultados, revisión de documentación y apoyo a la reproducibilidad.

También he usado IA en la memoria para el apoyo de tareas esencialmente de programación. La redacción, es decir, la intención de lo que se quiere decir y defender, es enteramente mía. Pero LaTeX es código, y se escribe en el mismo editor que Python. Por eso la IA ha trabajado con los preámbulos, parámetros de tablas y figuras, referencias, maquetación y trabajo general con las figuras.

Estas últimas son un caso especialmente importante. Ninguna es una imagen generada con IA generativa de imagen, sino diagramas hechos con TikZ (LaTeX) y Python, código que se modifica y ejecuta como cualquier otro. He usado Codex y Gemini para generación, ajuste y refinamiento de esas figuras, **siempre** partiendo de una intención mía. Qué debía mostrar el diagrama, qué elementos había de contener, una propuesta de colocación y paleta de colores. Todas las figuras tienen una ficha en Markdown que recoge la intención, convenciones y forma de trazabilidad.

Sin estas herramientas no habría podido hacer figuras con ese nivel de calidad y flexibilidad. La IA entiendo bien TikZ y LaTeX porque, en su base, son código. Así, he podido disponer de figuras hechas a medida en el trabajo en vez de imágenes genéricas. Esto, a mi juicio, muestra el potencial real. Permitir que cualquiera, incluidos todos los investigadores y científicos, puedan usarlas para artefactos que, de otra manera, no harían. El experto define el qué y el por qué; la IA rellena el cómo.

Otro aspecto relevante del trabajo ha sido la verbalización. A menudo he usado el habla como interfaz principal para explicar ideas, ya que es la forma más natural de comunicar intención. Esto no es exactamente IA generativa, sino reconocimiento de voz y flujo de trabajo, pero ha sido importante para este TFG. He usado herramientas locales propias, como Voice Stall, para pasar voz a texto y expresar ideas largas antes de convertirlas en planes o instrucciones para agentes.

Como cierre, sí, he usado IA generativa para programación, documentación, consulta de información, coherencia lingüística, generación y ajuste de figuras, revisión de código, análisis del repositorio, escritura de scripts, labores con LaTeX, *refinement* de especificaciones con planes y ejecución de flujos de trabajo ágenticos. Asumo plenamente la responsabilidad sobre la originalidad, veracidad, rigor y autoría de este documento. A mi juicio, este uso ha sido parte de mi aprendizaje como próximo Ingeniero Técnico Aeroespacial y una competencia profesional que será muy demandada. Gracias a usarla así durante el curso puedo explicar qué herramientas empleo, procedimientos, límites, revisión y cómo se puede usar la IA sin renunciar a la originalidad humana. Puedo enseñar este repositorio y no solo afirmar que usé IA, sino cómo; y lograr confianza de terceras personas para compartir el conocimiento.

Igual que no tendría sentido hacer en 2015 un trabajo ignorando Internet y buscando enciclopedias en papel, tampoco me parece razonable ignorar herramientas que ya forman parte real del trabajo de ingeniería en múltiples empresas, de forma más acusada en Estados Unidos. La diferencia está en el uso, sustituto o multiplicador de capacidades. En este TFG la he usado para lo segundo. Para mí este es el camino. No prohibir o demonizar a la IA generativa, sino usarla de forma responsable para aumentar la complejidad de lo que se puede alcanzar, programar mejor con menos experiencia y producir material de calidad.

1. Introducción

1.1. MOTIVACIÓN

El seguimiento autónomo de trayectorias de un cuadricóptero exige disponer de una estimación suficiente de su estado respecto al entorno y a la referencia, y transformar el error de seguimiento en una actuación que estabilice el vehículo mientras lo conduce por el recorrido indicado. La dificultad no reside en minimizar un error instantáneo respecto a una posición fija, sino que la referencia evoluciona con el tiempo y el controlador actúa sobre un sólido rígido con dinámica acoplada y expuesto a perturbaciones internas y externas. Por tanto, debe coordinar la traslación y actitud sin perder la estabilidad ni fallar en el desplazamiento.

Esta naturaleza acoplada motiva el uso de una arquitectura de control en dos niveles [1–3]. En este caso, un PD en cascada transforma en primer lugar los errores de posición y velocidad en una fuerza deseada en el sistema de referencia local East–North–Up (ENU). Posteriormente, el lazo interno convierte esa fuerza, junto con la referencia de guiñada, en una actitud deseada, empuje y momentos en el sistema de referencia cuerpo Forward–Right–Down (FRD). Finalmente, el mezclador (mixer) distribuye estos mandos entre los cuatro rotores. Ajustar un único controlador para maniobras de diversa naturaleza puede imponer un compromiso entre, por ejemplo, mantener una posición y recorrer una geometría con movimiento cambiante continuo. La especialización evita ese compromiso, pero escala el número de controladores a ajustar y validar así como el cambio constante de ganancias en operación. Es por ello que en este trabajo se decide construir un conjunto de controladores con un PD para cada familia de trayectorias y se estudia si una implementación de tipo neuronal puede condensar el funcionamiento en conjunto de los expertos. Esta idea de disponer de parámetros de control distintos en función del régimen de operación está estrechamente relacionada con la programación de ganancias (gain scheduling) [4].

La comparación entre los dos enfoques requiere un entorno donde el modelo, referencias, perturbaciones, semillas de aleatoriedad, límites de actuación y métricas permanezcan bajo control. El simulador 6DOF desarrollado proporciona esta base de ensayo donde todos los controladores comparten vehículo, observación de estado y cadena de actuación. Asimismo, cada ejecución conserva trazabilidad sobre la configuración para poder reproducir el experimento así como la telemetría necesaria para la revisión posterior. Esta plataforma no pretende sustituir una futura validación en vuelo, sino aislar las diferencias en un entorno controlado y reproducible.

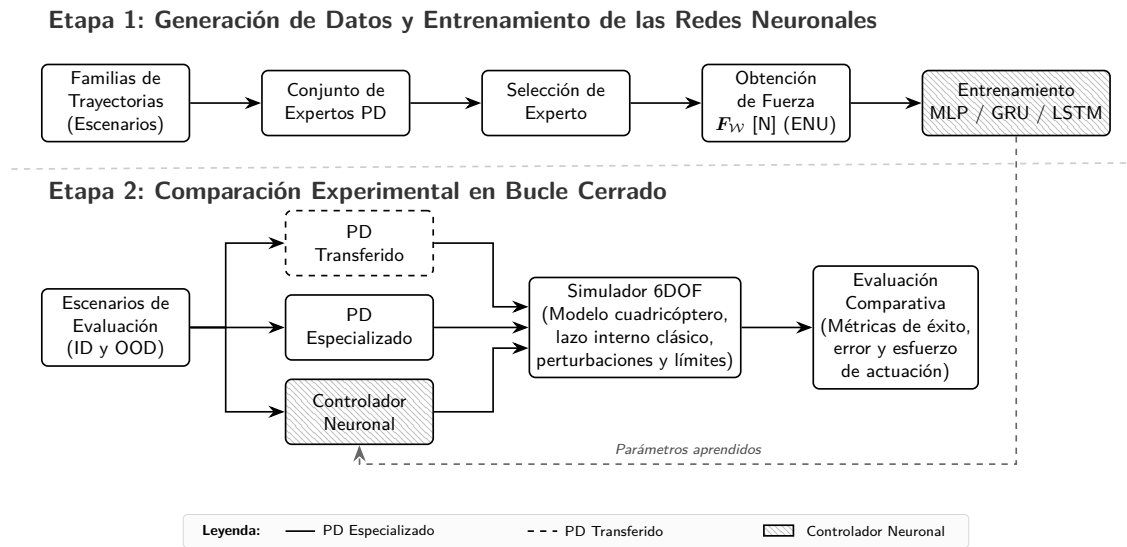


Figura 1.1. Esquema conceptual del flujo experimental del trabajo. Consta de la generación de trayectorias, especialización de controladores clásicos, entrenamiento de redes neuronales y evaluación comparativa. Fuente: elaboración propia.

En la [Figura 1.1](#) se detalla visualmente la lógica experimental estructurada en dos etapas. La primera etapa ilustra la obtención de los controladores expertos y generación de datos para entrenar las redes neuronales. La segunda etapa muestra la evaluación comparativa de las tres referencias de control en el simulador bajo las mismas condiciones: condiciones dentro de la distribución de entrenamiento o *in-distribution* (ID), y condiciones fuera de dicha distribución u *out-of-distribution* (OOD).

Problema abordado

El problema principal del trabajo consiste en determinar si un único controlador mediante redes neuronales, entrenado a partir de datos generados por controladores PD optimizados para distintas familias de trayectorias, puede sustituir ese conjunto de controladores cuando se evalúa bajo las mismas referencias y perturbaciones. La comparativa debe indicar si la implementación neuronal conserva un comportamiento competitivo bajo las condiciones conocidas y si posee una mejor transferencia a trayectorias compuestas, nuevas y con variaciones.

Se comparan tres referencias. El *PD especializado* es el controlador cuyos parámetros se han fijado para la familia a la que pertenece la trayectoria. Un *PD transferido* es uno de esos mismos controladores aplicado, sin reajuste, a otra familia. El sistema neuronal sustituye el lazo externo y realiza una *predicción de fuerza deseada* a partir de la observación y la referencia, produce un vector tridimensional en el sistema de referencia ENU y conserva el lazo interno clásico. De este modo, la red no transmite

comandos directamente a los rotores ni genera momentos en el sistema de referencia FRD.

Por un lado, el *rendimiento competitivo* se define como la capacidad de preservar, en bucle cerrado, la finalización de la trayectoria obteniendo al mismo tiempo errores de seguimiento, esfuerzo de control y un uso de las protecciones comparables a los de la referencia pertinente. Las métricas y criterios de evaluación asociados a estos conceptos se definen en el [epígrafe de métricas, criterios de éxito y análisis](#). Por otro lado, la *transferencia* se evalúa aplicando el controlador fuera de las condiciones de operación para las que fue ajustado originalmente. Para estructurar este análisis, la evaluación distingue entre escenarios dentro de la distribución de entrenamiento (ID, *in-distribution*) y fuera de ella (OOD, *out-of-distribution*), cuyos atributos se definen posteriormente en el [apartado 3.5](#). En el caso ID, se evalúan escenarios del mismo tipo que los usados para ajustar los PD y entrenar las redes, aunque con parámetros reservados para pruebas. En el caso OOD, se imponen variaciones dinámicas no vistas, perturbaciones externas o nuevas geometrías. Dentro de estas últimas referencias geométricas, se diferencian composiciones de trayectorias ya conocidas y familias geométricas completamente inéditas.

Es importante precisar que este estudio no pretende demostrar la superioridad completa de una arquitectura en particular, ni garantizar la robustez del sistema ante cualquier perturbación física o su aptitud para vuelo real. Asimismo, no se atribuye al controlador neuronal la acción de estabilización del lazo interno ni la gestión de protecciones impuestas mediante saturaciones y límites de fuerza. El alcance de este trabajo se limita a una comparación experimental trazable, realizada sobre un mismo modelo físico y entorno, entre la especialización de ganancias, la transferencia de controladores clásicos y la imitación neuronal.

1.2. HIPÓTESIS Y OBJETIVOS

La hipótesis central sostiene que un sistema neuronal único, entrenado por imitación para predecir la fuerza deseada del lazo externo, puede alcanzar una capacidad de seguimiento competitiva frente al PD especializado de cada familia en condiciones conocidas y transferir mejor que los PD aplicados fuera de su familia, mientras se conserva el lazo interno clásico en ambos.

El objetivo general es desarrollar un entorno reproducible de simulación que permita determinar si el controlador neuronal por imitación puede sustituir a un conjunto de controladores PD especializados y ofrecer un mejor rendimiento en trayectorias no empleadas durante el ajuste de ganancias y entrenamiento.

Este objetivo se descompone en los siguientes objetivos específicos:

1. Formular e implementar un modelo 6DOF de cuerpo rígido para un cuadricóptero, con sistemas de referencia apropiados, actuadores, perturbaciones simplificadas y frecuencias de actualización diferenciadas;
2. Definir familias de trayectorias y parámetros que permitan repetir las condiciones experimentales;
3. Implementar un PD en cascada y obtener, mediante un ajuste determinista sobre los escenarios de entrenamiento, un controlador especializado para cada familia;
4. Valorar la especialización de los PD y su capacidad de transferencia al aplicar cada controlador a familias distintas de la nativa;
5. Entrenar MLP, GRU y LSTM para predecir la fuerza deseada y comparar la ausencia o presencia de memoria temporal bajo las mismas condiciones;
6. Evaluar la calidad de la predicción en ejecución supervisada y, de forma separada, el seguimiento en bucle cerrado en condiciones ID y OOD; y
7. Conservar resultados suficientes para reconstruir los experimentos.

1.3. ESTRUCTURA DEL DOCUMENTO

La aportación del trabajo es la integración de cuatro elementos: un simulador 6DOF, un conjunto de trayectorias y parámetros mediante escenarios, un proceso de selección de PD en cascada especializado por familia y una comparación de MLP, GRU y LSTM en bucle cerrado. El interés reside en construir un entorno que permita distinguir qué propiedades proceden del PD especializado, qué aprende la red, qué aporta el lazo interno clásico y cuáles son los límites de esta aproximación al problema.

Tras esta introducción, el documento se estructura en cuatro bloques. El estado del arte sobre modelado, control clásico y control por imitación; la metodología, que detalla el simulador y las arquitecturas de control evaluadas; el análisis de los resultados experimentales; y, por último, las conclusiones y líneas futuras de desarrollo.

2. Estado del arte

2.1. SIMULACIÓN Y MODELADO DE CUADRICÓPTEROS

El cuadricóptero es un vehículo aéreo de seis grados de libertad cuya posición y actitud evolucionan de forma acoplada, y cuya actuación proviene de un número limitado de rotores. Esta combinación convierte el seguimiento de trayectorias en un problema especialmente notable dentro de los sistemas no tripulados, dado que el vehículo debe modificar su orientación para lograr la traslación requerida, a la vez que mantiene la estabilidad y respeta los límites de actuación. Por este motivo, la literatura de pequeños UAV tiende a describir el problema mediante modelos de cuerpo rígido, sistemas de referencia bien definidos y arquitecturas de control en cascada [1–3].

En un modelo de cuerpo rígido 6DOF, el estado del vehículo combina posición, velocidad, actitud y velocidad angular. La traslación se logra al sumar gravedad, fuerzas aerodinámicas y la fuerza generada por los rotores, transformada del sistema de referencia cuerpo al sistema local. La rotación se describe mediante las ecuaciones de Euler, donde los momentos aplicados por los rotores modifican la velocidad angular en función de la matriz de inercia. Este tipo de formulación permite estudiar la interacción entre seguimiento de posición, inclinación, limitación de actuadores y estabilidad, si bien no reproduce todos los fenómenos aerodinámicos de un vuelo.

El término *6DOF* indica que el vehículo dispone de tres grados de libertad en traslación y tres en rotación. Sin embargo, no puede imponer de forma independiente aceleraciones en cada eje y actitud arbitraria al mismo tiempo. En un cuadricóptero tradicional, los rotores generan empuje a lo largo del eje vertical del vehículo. La fuerza lateral aparece al inclinar el cuadricóptero y, con ello, orientar una parte del empuje hacia la dirección horizontal deseada. Esta es la razón física por la que se dice que el seguimiento de posición y el control de actitud están acoplados.

Los sistemas de referencia son una parte esencial de esta formulación. En aeronaves y vehículos aéreos se distinguen un sistema de referencia asociado al entorno y otro asociado al vehículo. El primero permite expresar la posición, velocidad y referencia de trayectorias de forma absoluta; mientras que el segundo facilita la determinación de los momentos y fuerzas generados por los rotores. La transformación entre ambos se realiza mediante una matriz de rotación asociada a la actitud. En este contexto, los cuaterniones son una alternativa habitual a los ángulos de Euler ya que representan la orientación sin introducir singularidades cinemáticas en actitudes concretas [5]. Los ángulos de Euler siguen siendo útiles para visualización y para fijar límites de operación, pero no el método más robusto para el cálculo y propagación de la actitud durante la integración numérica.

La generación de fuerzas en un cuadricóptero se modela mediante una relación cuadrática entre la velocidad angular del rotor y el empuje. Cada rotor contribuye además al momento total mediante su posición respecto del centro de masas y el par de reacción asociado a su sentido de giro [1, 6]. El mixer de los actuadores permite resolver el problema inverso, dado un empuje total y tres momentos, calcula los empujes individuales que deberían producir los rotores. Cuando la solución exige empujes negativos o que superan el máximo disponible, aparecen saturaciones en los mandos. En este caso, el controlador podría haber calculado una orden coherente, si bien no realizable. Por ello, la comparación de controladores no se limita al error de posición sino que también se registra cuando una actuación no se ha podido llevar a cabo.

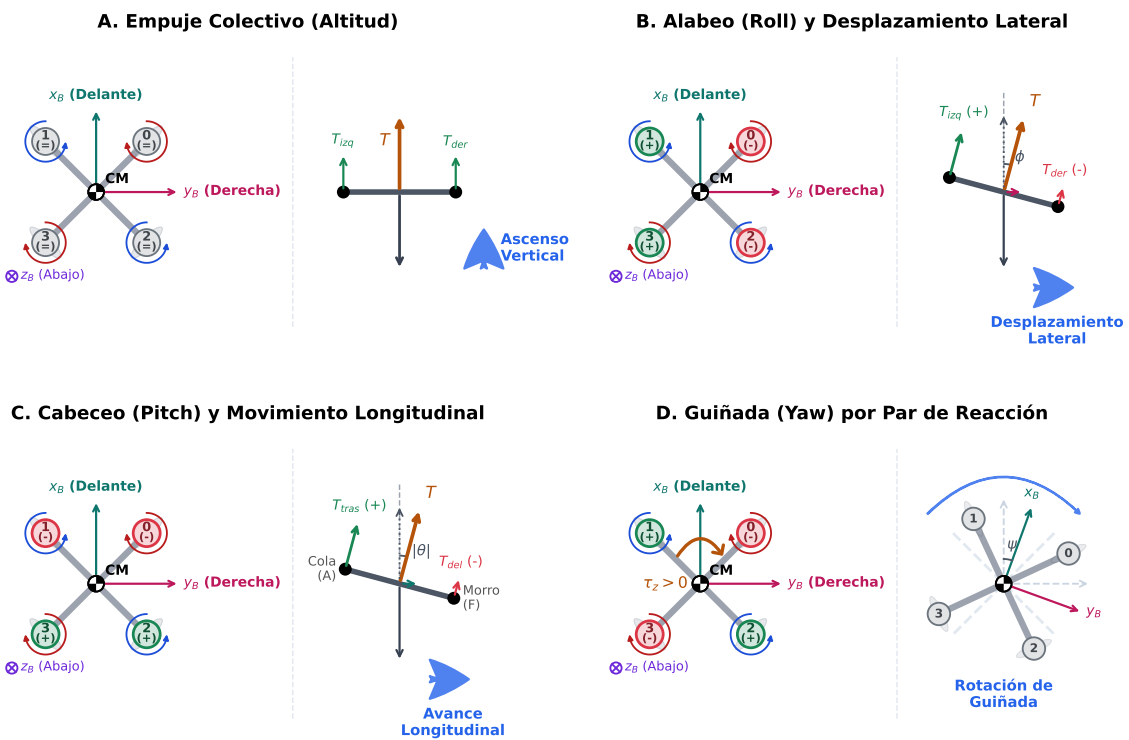


Figura 2.1. Mecanismos de actuación y movimiento en un cuadricóptero. Para cada maniobra se muestra la vista superior del vehículo en planta (izquierda) con la dirección de giro de cada rotor e indicaciones de desequilibrio de empuje (+, -, =), y el efecto físico resultante en vista lateral (derecha). La inclinación del vehículo (ϕ , θ) desvía el vector de empuje total (T) generando las componentes de fuerza lateral (F_y) o longitudinal (F_x) que posibilitan la traslación horizontal. Fuente: elaboración propia.

La gestión de la actuación puede entenderse a partir de cuatro combinaciones básicas (véase la figura 2.1). Si los cuatro rotores aumentan o reducen su empuje de forma parecida, cambia el empuje total y, por tanto, la aceleración vertical en una actitud nivelada. Si aumenta el empuje en un lado frente al opuesto, aparece un momento de alabeo y si la diferencia se produce entre la parte delantera y la trasera, aparece un momento de cabeceo. La guiñada se obtiene aprovechando los pares de reacción de

rotores que giran en sentidos opuestos. De forma resumida, cada rotor aporta una fuerza T_i y el conjunto produce una salida del tipo

$$\mathbf{u} = [T \ \tau_x \ \tau_y \ \tau_z]^\top,$$

donde T es el empuje colectivo y τ_x , τ_y y τ_z son los momentos en el sistema FRD.

La integración temporal del modelo exige discretizar las ecuaciones diferenciales. Los métodos de Runge–Kutta, y en particular RK4, son una opción clásica cuando se busca una relación favorable entre precisión y coste computacional [7]. En entornos de control digital es habitual separar la frecuencia de integración física, la frecuencia de actualización del controlador y la frecuencia de registro de telemetría. Entre dos ciclos de control, el último mando calculado se mantiene constante mediante un bloqueador de orden 0, aproximando así el comportamiento de un sistema digital que actúa sobre una dinámica continua [8].

Existen plataformas probadas para simular vehículos aéreos. MATLAB, Simulink y UAV Toolbox proporcionan herramientas para diseñar, simular, probar y desplegar aplicaciones UAV, incluyendo escenarios, sensores, autopilotos y flujos de prueba tanto en un ordenador como en hardware-in-the-loop [9]. En el ámbito del software libre, RotorPy ofrece un simulador Python de multirrotores orientado a educación e investigación, con dinámica 6DOF, aerodinámica, actuadores, sensores, viento y validación frente a datos reales [10].

El presente trabajo no necesita reproducir toda la amplitud de esas plataformas, sino que la modularidad completa y trazabilidad son los aspectos más importantes. Se implementan las partes tratadas previamente; dinámica 6DOF, sistemas de referencia concretos, actitud mediante cuaterniones, ley de control cuadrática, mixer con saturación, perturbaciones de carácter simplificado, integración y telemetría suficiente para reproducir y revisar cada ejecución.

2.2. ENTORNOS DE PROGRAMACIÓN Y APRENDIZAJE AUTOMÁTICO

La elección de un entorno de implementación condiciona las herramientas disponibles, la reproducibilidad y la facilidad de inspección del trabajo. Python se ha consolidado como un entorno para programación científica y simulaciones, con licencia abierta y un ecosistema extenso de librerías. Su interés en este contexto no reside en ser la opción más eficiente para un futuro vuelo embarcado, sino en permitir expresar con claridad el modelo, generar datos, ejecutar comparativas y conectar el simulador con herramientas de aprendizaje automático.

PyTorch proporciona tensores, funciones de pérdida y optimizadores que permiten entrenar modelos supervisados y recurrentes en CPU o GPU [11]. Además, incluye implementaciones estándar de capas lineales, GRU, LSTM, pérdida MSE y Adam, lo que permite trabajar a más alto nivel y evita el riesgo de introducir errores en componentes ampliamente establecidos.

La reproducibilidad también depende de la gestión del entorno y de los archivos relacionados con los experimentos. Herramientas como `uv` permiten declarar, instalar y ejecutar entornos Python de forma repetible y sencilla [12]. El uso de archivos de configuración legibles, como YAML (*YAML Ain't Markup Language*, un formato de texto legible para describir datos estructurados mediante claves, listas y valores), facilita que escenarios, semillas, trayectorias, controladores y condiciones de terminación puedan revisarse sin modificar código de una forma visual y sencilla. Frente a JSON, que está muy extendido pero es más rígido para la lectura y escritura por parte de un investigador, y frente a TOML, que resulta claro para configuraciones relativamente planas, YAML ofrece una sintaxis cómoda para describir estructuras jerárquicas con listas, diccionarios y bloques anidados [13–15]. Finalmente, Git y GitHub proporcionan un soporte común para historial de cambios, distribución del código y trazabilidad de los experimentos [16]. Gracias a este conjunto de herramientas, se facilita la auditoría y reproducción de los experimentos por cualquier persona.

2.3. CONTROL CLÁSICO EN CASCADA Y PROGRAMACIÓN DE GANANCIAS

El control clásico de cuadricópteros suele organizarse en cascada. El lazo externo trabaja sobre posición y velocidad, y calcula una fuerza o aceleración deseada en el sistema de referencia ENU. El lazo interno transforma esa intención de traslación en una actitud deseada y estabiliza el vehículo mediante momentos en el sistema de referencia FRD. Esta separación también se observa en la velocidad, la dinámica traslacional es más lenta y requiere menos correcciones que la dinámica angular [1–3].

En esta arquitectura, el lazo externo no decide directamente las velocidades de los rotores, sino que su salida es una fuerza deseada en el sistema local. Si el vehículo se encuentra por debajo de la referencia, la fuerza tendrá una componente vertical mayor; y si necesita avanzar horizontalmente, tendrá una componente en esta dirección. El lazo interno toma la fuerza, calcula qué orientación del vehículo permitiría generarla con el empuje disponible y modifica la actitud hacia esa orientación. La guiñada de referencia completa la orientación, ya que la dirección de empuje fija únicamente la dirección del eje vertical.

Los controladores PD y PID continúan siendo la referencia por su simplicidad, interpretabilidad y bajo coste computacional. Un término proporcional corrige el error de posición o actitud, un término derivativo amortigua la respuesta a partir de la velocidad o la velocidad angular, y un término integral puede reducir errores estacionarios. Sin embargo, la acción integral puede acumular mando durante saturaciones, por lo que requeriría mecanismos de protección contra saturación (anti-windup). En este caso, una estructura de PD junto con el dato de aceleración de trayectoria y compensación de fuerza de gravedad resulta en una implementación más factible.

La sintonización de ganancias puede abordarse de varias maneras. El ajuste manual permite contar con una intuición, pero dificulta la trazabilidad si no se conservan todas las iteraciones. Las reglas clásicas de Ziegler–Nichols ofrecen procedimientos sistemáticos para ciertas clases de sistemas, aunque no representan directamente un cuadricóptero 6DOF con saturaciones y trayectorias espaciales [17]. También es posible utilizar controladores sobre modelos linealizados en torno a puntos de operación, o buscar ganancias mediante optimización numérica evaluada en simulación cerrada. Esta última aproximación resulta de especial interés cuando el objetivo es comparar diversos candidatos bajo las mismas trayectorias, límites y métricas.

Cuando un único conjunto de ganancias no ofrece un resultado satisfactorio en todo el rango de operación, una respuesta clásica es la programación de ganancias. Esta técnica selecciona o interpola parámetros de control según ciertas variables que describen el punto de operación, de manera que distintas regiones del vuelo pueden usar ajustes distintos [4]. Conceptualmente, un conjunto de controladores especializados por familia de trayectoria se relaciona con esta idea, ya que se cuenta con varios expertos y se aplica el más adecuado para cada condición. No obstante, en este trabajo la implementación neuronal no realiza una interpolación de ganancias explícita según unos criterios fijos, sino que aprende a predecir la fuerza del lazo externo a partir de los mismos datos que usa el PD.

2.4. APRENDIZAJE POR IMITACIÓN Y CONTROL HÍBRIDO

El aprendizaje por imitación busca realizar un entrenamiento a partir de datos generados por un sistema experto. En su forma más directa, esta clonación de comportamiento plantea un problema supervisado, se reciben las observaciones del sistema y se aprende a reproducir sus acciones. Este concepto tiene antecedentes en conducción autónoma, como ALVINN, donde una red aprendía a producir comandos de dirección a partir de ejemplos humanos [18]. En robótica y control, la imitación evita el diseño de una

función de recompensa completa y permite la transferencia de comportamiento de un controlador del que ya se dispone.

Sin embargo, la clonación de comportamiento tiene una limitación importante, los datos proceden de estados visitados por el experto, pero durante la ejecución del procedimiento aprendido se pueden cometer pequeños errores y visitar estados no representados en los datos de entrenamiento. Esta diferencia entre las distribuciones de entrenamiento y la inducida por el sistema aprendido se conoce como desplazamiento de distribución. Existen métodos como DAgger [19] que reducen este problema mediante la agregación de datos de los estados visitados por el sistema neuronal y consultando de nuevo al sistema original. Si bien este trabajo no implementa DAgger, es importante recordar que una pérdida supervisada baja en el entrenamiento no garantiza un buen comportamiento en el bucle cerrado.

En aplicaciones de control físico, un procedimiento de aprendizaje automático puede reemplazar todo el controlador o solo una parte de la arquitectura. Los enfoques híbridos conservan elementos clásicos cuando estos aportan estabilidad, límites o interpretación. En este caso, una red aprende el comportamiento del lazo externo, mientras que el lazo interno, el mixer y las protecciones siguen siendo clásicos. No obstante, esto introduce el riesgo de atribuir efectos erróneamente a la red, por lo que se han de identificar y separar las contribuciones de cada elemento.

2.5. REDES NEURONALES PARA CONTROL

Las redes neuronales usadas en este trabajo se formulan como modelos supervisados de regresión. La entrada es un vector o secuencia de vectores contruidos a partir de la observación y de la referencia de la trayectoria mientras que la salida es una fuerza tridimensional en el sistema de referencia ENU. Durante el entrenamiento, los parámetros se ajustan para reducir la diferencia entre la fuerza predicha y la fuerza emitida por el PD especializado. Esta formulación requiere representar los datos como tensores, agrupar muestras en lotes y optimizar una función de pérdida diferenciable mediante retropropagación [11, 20].

El entrenamiento de una red no queda determinado solo por la arquitectura y los datos. También depende de hiperparámetros, decisiones fijadas antes del ajuste de pesos y sesgos que regulan cómo se realiza la optimización. Entre ellos se encuentran la tasa de aprendizaje, el tamaño del lote, el número máximo de épocas y los criterios de parada. A diferencia de los parámetros entrenables de la red, estos valores no se aprenden sino que se seleccionan como parte del diseño.

Un lote o *batch* es el conjunto de muestras que se procesan conjuntamente antes de calcular una pérdida y actualizar los parámetros del modelo. Su tamaño se denota

habitualmente por B , valores mayores proporcionan una estimación más estable del gradiente, mientras que valores menores introducen más variación entre actualizaciones y requieren más iteraciones. Una época corresponde a una pasada completa por las muestras de entrenamiento. La parada temprana detiene el entrenamiento cuando la pérdida de validación deja de mejorar, y la paciencia indica cuántas épocas consecutivas sin mejora se toleran antes de interrumpir el proceso.

Una red neuronal no almacena reglas explícitas del tipo *if-then*, sino que aprende pesos y sesgos que transforman las entradas en salidas. En regresión supervisada, cada muestra proporciona un par entrada-objetivo y el entrenamiento modifica los parámetros para minimizar una pérdida. Si θ agrupa todos los parámetros de la red y $\mathcal{L}(\theta)$ mide el error medio, el descenso por gradiente actualiza los parámetros en la dirección opuesta al gradiente:

$$\theta_{n+1} = \theta_n - \eta \nabla_{\theta} \mathcal{L}(\theta_n),$$

donde η es la tasa de aprendizaje. Un valor demasiado grande puede hacer que el entrenamiento oscile o diverja; uno demasiado pequeño puede volverlo excesivamente lento. Optimizadores como Adam adaptan el tamaño efectivo de actualización usando información acumulada de los gradientes, pero siguen dependiendo de una tasa de aprendizaje base [21].

2.5.1. Redes feedforward

Una red feedforward calcula la salida mediante una sucesión de capas aplicadas en una sola dirección, desde la entrada hasta la predicción de la salida. En una capa lineal, la salida se obtiene multiplicando la entrada por una matriz de pesos y sumando un vector de sesgos. Si solo se encadenasen transformaciones lineales, el modelo seguiría siendo equivalente a una transformación lineal global. Es por esto que se introducen funciones de activación no lineales entre las capas. La unidad lineal rectificada (ReLU) es una activación habitual por su simplicidad y por facilitar el entrenamiento frente a saturaciones propias de otras funciones [22].

Para una capa general, la operación puede escribirse como

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)}),$$

donde $\mathbf{W}^{(\ell)}$ y $\mathbf{b}^{(\ell)}$ son pesos y sesgos, y ϕ es una activación no lineal. La profundidad viene dada por el número de capas, mientras que la anchura se asocia al número de unidades por capa.

El perceptrón multicapa, o MLP, combina varias capas lineales con activaciones no lineales. Teóricamente, las redes feedforward con suficientes unidades pueden aproximar un amplio rango de funciones continuas [23]. De esta manera, se emplea la MLP como

la referencia neuronal sin memoria, de manera que dos entradas iguales producen la misma fuerza, sin importar lo ocurrido en el paso anterior. Esta característica la hace más simple y de un menor coste computacional que las arquitecturas recurrentes, pero también limita su capacidad para usar información de carácter temporal cuando existen retardos o estados transitorios.

2.5.2. Redes recurrentes

Las redes neuronales recurrentes (RNN) amplían la formulación feedforward mediante un estado oculto que se actualiza a lo largo del tiempo. En cada instante el modelo procesa la entrada actual y una representación compacta de la historia previa. Así, la salida no dependerá solo del error en ese instante, sino también de cómo se ha alcanzado ese estado.

Una RNN básica puede resumirse mediante

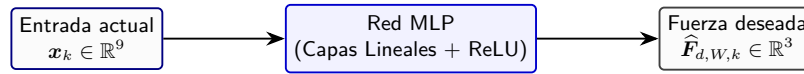
$$\mathbf{h}_k = \phi(\mathbf{W}_x \mathbf{x}_k + \mathbf{W}_h \mathbf{h}_{k-1} + \mathbf{b}_h), \quad \mathbf{y}_k = \mathbf{W}_y \mathbf{h}_k + \mathbf{b}_y.$$

El vector $\mathbf{x}_k$ es la entrada en el instante determinado k , $\mathbf{h}_k$ es el estado oculto que resume la historia procesada hasta ese instante e $\mathbf{y}_k$ es la salida. Durante el entrenamiento, la red se desenrolla en el tiempo y el gradiente se propaga hacia atrás a través de la secuencia temporal. Este procedimiento, conocido como retropropagación a través del tiempo (BPTT), permite ajustar los mismos parámetros a partir de los errores observados en la secuencia temporal.

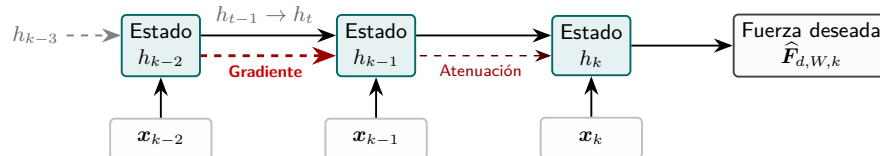
Sin embargo, la RNN básica presenta dificultades para aprender dependencias temporales largas. Durante la retropropagación, los gradientes pueden atenuarse o crecer de forma excesiva, dificultando la conservación de información relevante a lo largo de un gran número de pasos. Es por ello que las arquitecturas recurrentes modernas cuentan con compuertas internas para regular qué información se conserva, se actualiza o se descarta [24]. En la [Figura 2.2](#) se ilustra la diferencia fundamental entre los enfoques estáticos y los temporales de los dos tipos de arquitecturas.

Long short-term memory (LSTM). La arquitectura LSTM introduce un estado de celda y varias compuertas que controlan la entrada, el olvido y la salida de información. Su objetivo es facilitar que la red conserve información relevante durante más pasos temporales que una RNN simple, mitigando el problema de gradientes desvanecidos [25]. En el contexto del seguimiento de trayectorias, esta memoria es positiva si la historia reciente contiene información sobre retardos o evolución del error. Sin embargo, también provoca un aumento de parámetros en la red y puede introducir sensibilidad ante patrones temporales débilmente representados en el conjunto de datos de entrenamiento.

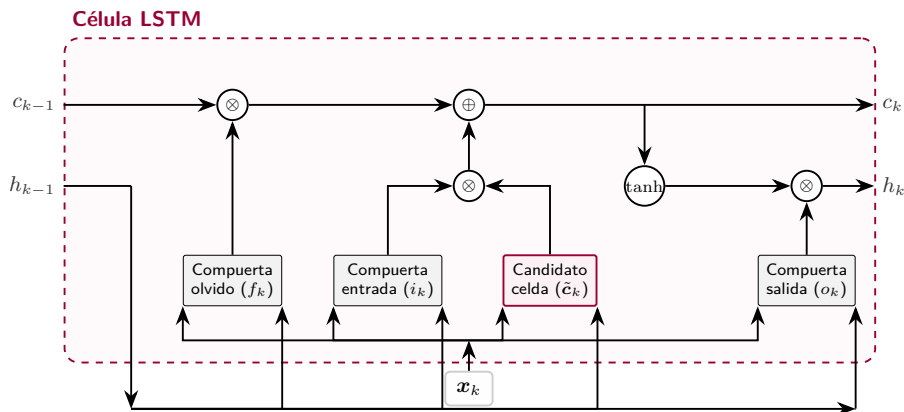
A. Perceptrón Multicapa (MLP) – Sin memoria temporal



B. Red Recurrente Básica (RNN) – Memoria lineal simple sin compuertas



C. Célula Long Short-Term Memory (LSTM) – Regulación mediante estado de celda



D. Célula Gated Recurrent Unit (GRU) – Compuertas simplificadas

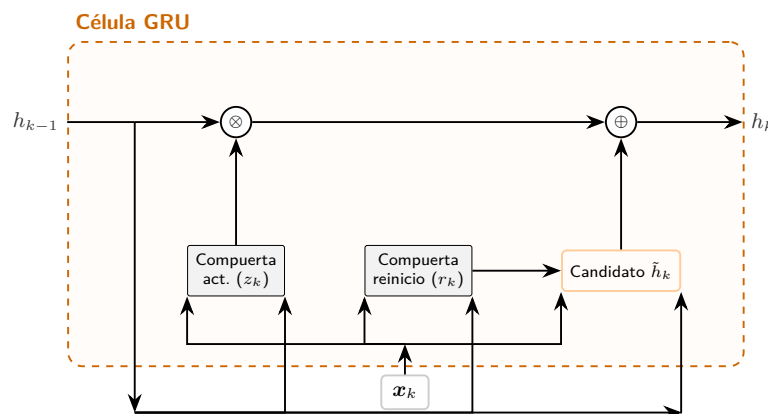


Figura 2.2. Comparativa conceptual de arquitecturas neuronales para control. En (A) se muestra el carácter puramente estático de la MLP. En (B) se detalla el flujo desenrollado de la RNN básica y su vulnerabilidad al desvanecimiento del gradiente. En (C) y (D) se contrasta la estructura selectiva con compuertas de la célula LSTM y la célula GRU. Fuente: elaboración propia.

La LSTM separa dos magnitudes internas. El estado oculto $\mathbf{h}_k$ es la representación que se entrega al exterior de la celda y se usa para calcular la salida o que sirve como entrada para el siguiente paso. El estado de celda $\mathbf{c}_k$ actúa como una memoria interna más directa, diseñada para transportar información a lo largo de la secuencia. Las tres compuertas principales regulan esa memoria, la compuerta de olvido decide qué parte de $\mathbf{c}_{k-1}$ se conserva, la compuerta de entrada qué información nueva se incorpora, y la compuerta de salida qué parte de la memoria actualizada se expone como $\mathbf{h}_k$. Una formulación habitual de la LSTM es

$$\mathbf{f}_k = \sigma(\mathbf{W}_f[\mathbf{h}_{k-1}, \mathbf{x}_k] + \mathbf{b}_f), \quad (2.1)$$

$$\mathbf{i}_k = \sigma(\mathbf{W}_i[\mathbf{h}_{k-1}, \mathbf{x}_k] + \mathbf{b}_i), \quad (2.2)$$

$$\tilde{\mathbf{c}}_k = \tanh(\mathbf{W}_c[\mathbf{h}_{k-1}, \mathbf{x}_k] + \mathbf{b}_c), \quad (2.3)$$

$$\mathbf{c}_k = \mathbf{f}_k \odot \mathbf{c}_{k-1} + \mathbf{i}_k \odot \tilde{\mathbf{c}}_k, \quad (2.4)$$

$$\mathbf{o}_k = \sigma(\mathbf{W}_o[\mathbf{h}_{k-1}, \mathbf{x}_k] + \mathbf{b}_o), \quad (2.5)$$

$$\mathbf{h}_k = \mathbf{o}_k \odot \tanh(\mathbf{c}_k). \quad (2.6)$$

Aquí σ es la activación sigmoide, $\tanh$ es la activación tangente hiperbólica, $[\mathbf{h}_{k-1}, \mathbf{x}_k]$ denota concatenación y $\odot$ producto elemento a elemento.

Gated recurrent unit (GRU). La GRU también utiliza compuertas, pero con una estructura más compacta que la LSTM. En vez de separar de manera explícita un estado de celda y otro oculto, combina mecanismos de actualización y reinicio para decidir cuánta información previa conserva y cómo incorpora la entrada actual [26]. Esto la sitúa como una alternativa de red recurrente con menor complejidad que LSTM, manteniendo la capacidad de procesar secuencias. Así, comparar GRU y LSTM bajo unas mismas condiciones permitirá valorar si una memoria más compleja aporta una mejora para el control en lazo externo.

En una GRU, la compuerta de actualización $\mathbf{z}_k$ regula cuánto del estado anterior se conserva frente a cuánto se reemplaza por una nueva propuesta. La compuerta de reinicio $\mathbf{r}_k$ controla cuánto influye el estado anterior al calcular el candidato $\tilde{\mathbf{h}}_k$. Una formulación habitual es

$$\mathbf{z}_k = \sigma(\mathbf{W}_z[\mathbf{h}_{k-1}, \mathbf{x}_k] + \mathbf{b}_z), \quad (2.7)$$

$$\mathbf{r}_k = \sigma(\mathbf{W}_r[\mathbf{h}_{k-1}, \mathbf{x}_k] + \mathbf{b}_r), \quad (2.8)$$

$$\tilde{\mathbf{h}}_k = \tanh(\mathbf{W}_h[\mathbf{r}_k \odot \mathbf{h}_{k-1}, \mathbf{x}_k] + \mathbf{b}_h), \quad (2.9)$$

$$\mathbf{h}_k = (1 - \mathbf{z}_k) \odot \mathbf{h}_{k-1} + \mathbf{z}_k \odot \tilde{\mathbf{h}}_k. \quad (2.10)$$

Al no mantener un estado de celda separado, la GRU suele requerir menos parámetros que una LSTM equivalente. No obstante, puede haber casos en los que esta menor complejidad sea suficiente para capturar la dependencia temporal e, incluso, reducir la sensibilidad frente a secuencias con escasa representación.

En inferencia existen dos formas prácticas de usar una red recurrente. En una ejecución *stateful*, el estado oculto calculado en un paso se conserva y se entrega como estado inicial del paso siguiente. En una ejecución basada en ventana, en cambio, para cada predicción se reconstruye una secuencia de longitud fija, se procesa completamente y se provee la salida. La primera opción evita recalcular de forma repetida información pasada, pero provoca una dependencia del estado interno de la propagación de la red durante el episodio. La segunda es más costosa computacionalmente, pero mantiene un impacto local dentro de la ventana. Esta distinción será relevante posteriormente, ya que la implementación efectuada usa ventanas temporales de longitud fija en vez de una inferencia recurrente plenamente *stateful*.

El entrenamiento de estas arquitecturas conserva elementos del aprendizaje supervisado. La normalización calculada solo con el conjunto (*split*) de entrenamiento evita que variables de distinta escala dominen la función de pérdida y reduce fugas de información desde validación o prueba. El error cuadrático medio (MSE) es una función útil en este caso, al penalizar con mayor intensidad las desviaciones grandes entre la predicción y la realidad. También se emplea validación para seleccionar un checkpoint sin ajustar los pesos sobre el conjunto de prueba y parada temprana para detener el entrenamiento si no hay mejoría tras un número determinado de épocas.

2.6. EVALUACIÓN, GENERALIZACIÓN Y MÉTRICAS

En sistemas dinámicos de este tipo, la evaluación se debe realizar tanto de forma supervisada como sobre el comportamiento del sistema en lazo cerrado. Puede ocurrir que una red aproxime muy bien la fuerza del controlador especializado en las muestras del conjunto de prueba y, aun así, genere una trayectoria peor o incluso divergente cuando sus predicciones se realimentan en la dinámica. Esta diferencia se debe al hecho de que el control en el bucle cerrado modifica los propios estados futuros que ve la red, transformando desviaciones en las fuerzas en datos de entrada diferentes. Por este motivo, una evaluación de aprendizaje por imitación ha de distinguir entre fidelidad supervisada, error de seguimiento y uso de las limitaciones del sistema.

La separación entre entrenamiento, validación y prueba es necesaria para poder estimar el comportamiento en datos no usados para el ajuste de parámetros o la selección de checkpoints. Además, cuando se busca estudiar transferencia, también se ha de distinguir entre condiciones dentro de la distribución de entrenamiento (*in-*

distribution, ID) y fuera de ella (*out-of-distribution*, OOD). Así, las primeras comprueban interpolación dentro de familias y perturbaciones ya representadas en el entrenamiento, mientras que las segundas evalúan si se conserva un comportamiento razonable ante recombinaciones o geometrías nuevas.

Las métricas de seguimiento han de capturar aspectos complementarios al error. De esta manera, el RMSE penaliza con más intensidad errores grandes, MAE describe el error medio y el error máximo detecta extremos puntuales. Estas magnitudes se deben interpretar junto a saturaciones, esfuerzos de control y causas de finalización de las simulaciones, ya que se puede lograr un error medio bajo con una demanda continua de acciones no realizables.

2.7. POSICIONAMIENTO DEL TRABAJO

El trabajo se sitúa en un punto intermedio entre el control clásico y el control puramente aprendido. Por un lado, el control en cascada permite la separación entre traslación y actitud, observabilidad sobre su funcionamiento, fuerza, momento y saturaciones. Pero cuenta con una limitación debido al proceso de sintonización dependiente de la familia de trayectorias, movimientos transitorios y perturbaciones consideradas. Por otro lado, las redes neuronales son una manera flexible de aproximar relaciones a partir de los datos, pero obliga a la distinción entre fidelidad supervisada y de comportamiento en lazo cerrado.

Este trabajo, por tanto, emplea un esquema híbrido, donde se mantiene el lazo interno clásico, el mixer y las protecciones de actuación, y se sustituye únicamente el lazo externo por una red neuronal entrenada por imitación. La red produce una fuerza deseada en el sistema ENU, conservando así la separación entre el componente estocástico y los elementos deterministas. En este contexto, se construye un simulador con trazabilidad, se ajustan controladores PD especializados y se entrenan arquitecturas MLP, GRU y LSTM para condensar el comportamiento de los controladores clásicos. La comparativa completa la pérdida supervisada con métricas de seguimiento, terminaciones y activación de las protecciones en lazo cerrado.

3. Metodología

3.1. MODELO FÍSICO Y SIMULADOR 6DOF

Requisitos, convenciones e hipótesis

El simulador se ha planteado para que el controlador clásico y el controlador neuronal se enfrenten al mismo problema físico y al mismo flujo de actuación, es decir, comparten la misma interfaz, de manera que el simulador no diferencia la naturaleza del controlador. De esta manera, las diferencias observadas se deberán exclusivamente al controlador y no a factores externos.

El estado se escribe como $\mathbf{x} = (\mathbf{p}_W, \mathbf{v}_W, \mathbf{q}_{WB}, \boldsymbol{\omega}_B)$, posición y velocidad en el sistema de referencia mundo $\mathcal{W}$, orientación del cuerpo respecto del mundo y velocidad angular en el sistema de referencia cuerpo $\mathcal{B}$. La entrada que se aplica al simulador está formada por la fuerza total de los rotores y el momento total, ambos expresados en el sistema de referencia cuerpo. La salida registrada incluye el estado, la observación entregada al controlador, la referencia de trayectoria, el comando solicitado y la actuación aplicada. De esta manera se puede comprobar cuándo y por qué un mando no ha podido aplicarse.

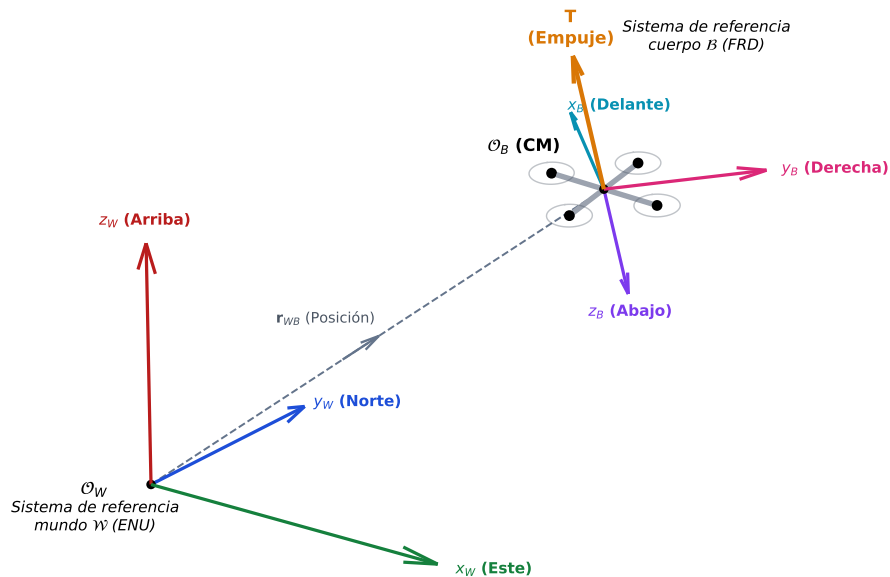


Figura 3.1. Sistemas de referencia empleados por el simulador. El sistema mundo sigue la convención ENU y el sistema cuerpo la convención FRD; el empuje total se aplica en la dirección opuesta al eje vertical del cuerpo (z_B). Fuente: elaboración propia.

El sistema de referencia $\mathcal{W}$ sigue la convención ENU, donde x apunta al Este, y al Norte y z hacia arriba. El sistema de referencia $\mathcal{B}$ es FRD, donde x_B apunta hacia delante, y_B a la derecha y z_B hacia abajo. Los rotores deben generar empuje en dirección $-z_B$ (véase [Figura 3.1](#)). La matriz de rotación $\mathbf{R}_{WB}(\mathbf{q}_{WB})$ transforma

vectores expresados en FRD a vectores expresados en ENU. La actitud se almacena mediante el cuaternión $\mathbf{q}_{WB} = [q_0, q_1, q_2, q_3]^T$. Las magnitudes se expresan en el Sistema Internacional y los ángulos solo se convierten a grados para visualización.

Se supone un cuerpo rígido de masa e inercia constantes, centro de masas fijo y gravedad uniforme. Los rotores se modelan con un empuje proporcional al cuadrado de su velocidad angular, con retardo y lag de primer orden. No se modelan aerodinámica avanzada, flapping de rotores, flexibilidad estructural ni sensores físicos detallados.

Vehículo de referencia y parámetros físicos

Se ha definido un cuadricóptero hipotético de 1 kg, gravedad 9.81 m s^{-2} e inercia en ejes principales $\mathbf{I}_B = \text{diag}(0.05, 0.05, 0.10) \text{ kg m}^2$. Los cuatro rotores se sitúan, en el sistema de referencia FRD, en

$$\mathbf{r}_{i,B} \in \{(0.17, 0.17, 0), (0.17, -0.17, 0), (-0.17, 0.17, 0), (-0.17, -0.17, 0)\} \text{ m},$$

lo que corresponde a una configuración en X respecto a los ejes de cuerpo. Los sentidos de giro se codifican mediante $s_i = \{-1, 1, 1, -1\}$, donde el signo determina el par de reacción de guiñada. En [Figura 3.2](#) se muestra un esquema de la configuración del vehículo.

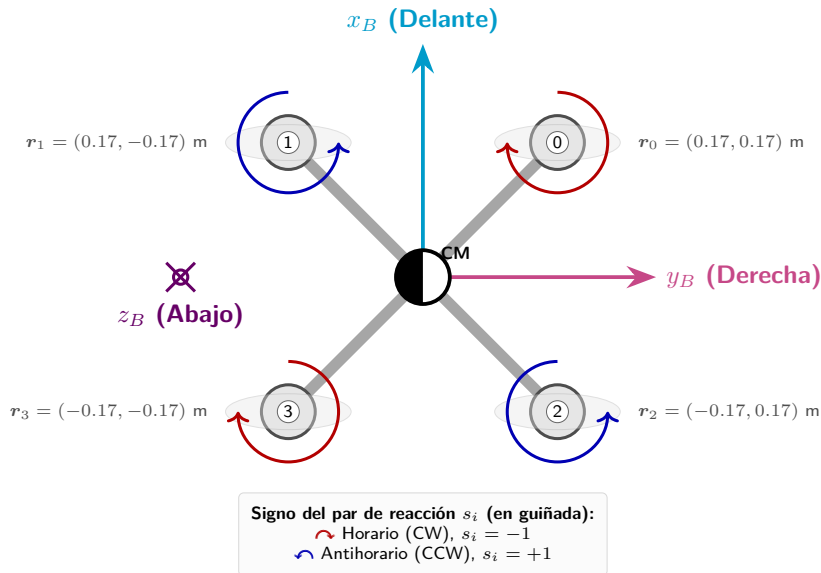


Figura 3.2. Configuración en X de los motores y sentidos de giro del rotor en el sistema de referencia cuerpo. La dirección física de los rotores se asocia con el signo del momento de reacción en guiñada. Fuente: elaboración propia.

Cada rotor utiliza $k_f = 1 \times 10^{-4} \text{ N s}^2 \text{ rad}^{-2}$ para el empuje y $k_m = 1 \times 10^{-6} \text{ N m s}^2 \text{ rad}^{-2}$ para el par de reacción. En los escenarios se toma $\omega_{\text{máx}} = 500 \text{ rad s}^{-1}$, de modo que el empuje máximo por rotor es

$$T_{\text{máx}} = k_f \omega_{\text{máx}}^2 = 25 \text{ N}. \quad (3.1)$$

Este límite es arbitrario y muy superior al empuje por rotor en estacionario ($mg/4 \simeq 2.45 \text{ N}$). El controlador impone además un límite de fuerza total de $2.5mg$ antes de la conversión a actitud, de manera que una saturación de rotor se interpretará como límite en la asignación.

La metodología seguida mantiene el mismo vehículo en todos los escenarios de comparación, si bien el simulador permitiría cambiar masa, inercia o parámetros de rotor en la configuración individual de cada escenario.

Cinemática y dinámica del cuerpo rígido

Sean $\mathbf{F}_B$ y $\boldsymbol{\tau}_B$ la fuerza y el momento aplicados en el sistema de referencia FRD. La dinámica traslacional es

$$\dot{\mathbf{p}}_{\mathcal{W}} = \mathbf{v}_{\mathcal{W}}, \quad (3.2)$$

$$m\dot{\mathbf{v}}_{\mathcal{W}} = \mathbf{R}_{WB}\mathbf{F}_B + \mathbf{F}_{d,W} + [0 \ 0 \ -mg]^T, \quad (3.3)$$

donde $\mathbf{F}_{d,W}$ es el drag lineal definido en el [epígrafe de perturbaciones, observación y seguridad](#). La dinámica rotacional se expresa como

$$\mathbf{q}_{\dot{W}B} = \frac{1}{2} \mathbf{q}_{WB} \otimes [0 \ \boldsymbol{\omega}_B^T]^T, \quad (3.4)$$

$$\dot{\boldsymbol{\omega}}_B = \mathbf{I}_B^{-1} (\boldsymbol{\tau}_B - \boldsymbol{\omega}_B \times \mathbf{I}_B \boldsymbol{\omega}_B). \quad (3.5)$$

Durante las evaluaciones intermedias del método de Runge–Kutta y al final de cada paso se normaliza el cuaternión, evitando así que los errores numéricos acumulados separen su norma de la unidad.

El estado avanza mediante Runge–Kutta de cuarto orden (RK4) y paso fijo de 0.01 s. Esta elección se sustenta en un menor coste computacional y una discretización suficiente para trayectorias sin maniobras bruscas. El comando aplicado se mantiene constante durante cada paso de integración [7].

Actuadores, mixer y límites de actuación

Para el rotor i , la velocidad aplicada determina empuje y par de reacción mediante

$$T_i = k_f \omega_i^2, \quad Q_i = s_i k_m \omega_i^2, \quad (3.6)$$

donde $s_i \in \{-1, 1\}$ representa el sentido de giro. La fuerza del rotor en el sistema FRD es $\mathbf{F}_{i,B} = [0 \ 0 \ -T_i]^\top$ y su contribución al momento incluye el término de brazo $\mathbf{r}_{i,B} \times \mathbf{F}_{i,B}$ y el par de reacción $[0 \ 0 \ Q_i]^\top$. El modelo cuadrático de empuje y par de reacción es una aproximación habitual en modelos de cuadricóptero de bajo orden [1, 2].

El mixer transforma una petición de empuje total y momentos en comandos individuales de rotor. Para ello construye una matriz de asignación $\mathbf{M}$ cuyas columnas recogen la contribución de cada empuje T_i al empuje total y momentos de alabeo, cabeceo y guiñada:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ -y_1 & -y_2 & -y_3 & -y_4 \\ x_1 & x_2 & x_3 & x_4 \\ s_1 k_m / k_f & s_2 k_m / k_f & s_3 k_m / k_f & s_4 k_m / k_f \end{bmatrix}}_{\mathbf{M}} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix}. \quad (3.7)$$

Por tanto, un rotor situado a la derecha del vehículo ($y_i > 0$) aporta momento de alabeo negativo cuando aumenta su empuje, mientras que un rotor situado delante ($x_i > 0$) aporta momento de cabeceo positivo, como se muestra en la [Figura 2.1](#).

La solución del sistema se obtiene invirtiendo la matriz de asignación ($\mathbf{M}$) para la geometría definida. Tal como se ilustra en el esquema de la [Figura 3.3](#), el mixer separa la contribución del empuje colectivo solicitado (T) de la parte debida a los momentos de actitud ($\boldsymbol{\tau}_B$):

$$T_{\text{thrust_part}} = \mathbf{M}^{-1}[T, 0, 0, 0]^\top, \quad (3.8)$$

$$T_{\text{moment_part}} = \mathbf{M}^{-1}[0, \tau_x, \tau_y, \tau_z]^\top. \quad (3.9)$$

Para evitar que los empujes finales superen los límites físicos individuales de los actuadores, establecidos en el rango $[T_{\min}, T_{\max}]$, se evalúa la existencia de una holgura en el empuje colectivo. Los límites inferior ($\delta_{\min}$) y superior ($\delta_{\max}$) para el desplazamiento del colectivo se calculan determinando el valor más restrictivo de entre los cuatro rotores

$$\delta_{\min} = \max_i (T_{\min} - T_{\text{thrust_part},i} - T_{\text{moment_part},i}), \quad (3.10)$$

$$\delta_{\max} = \min_i (T_{\max} - T_{\text{thrust_part},i} - T_{\text{moment_part},i}). \quad (3.11)$$

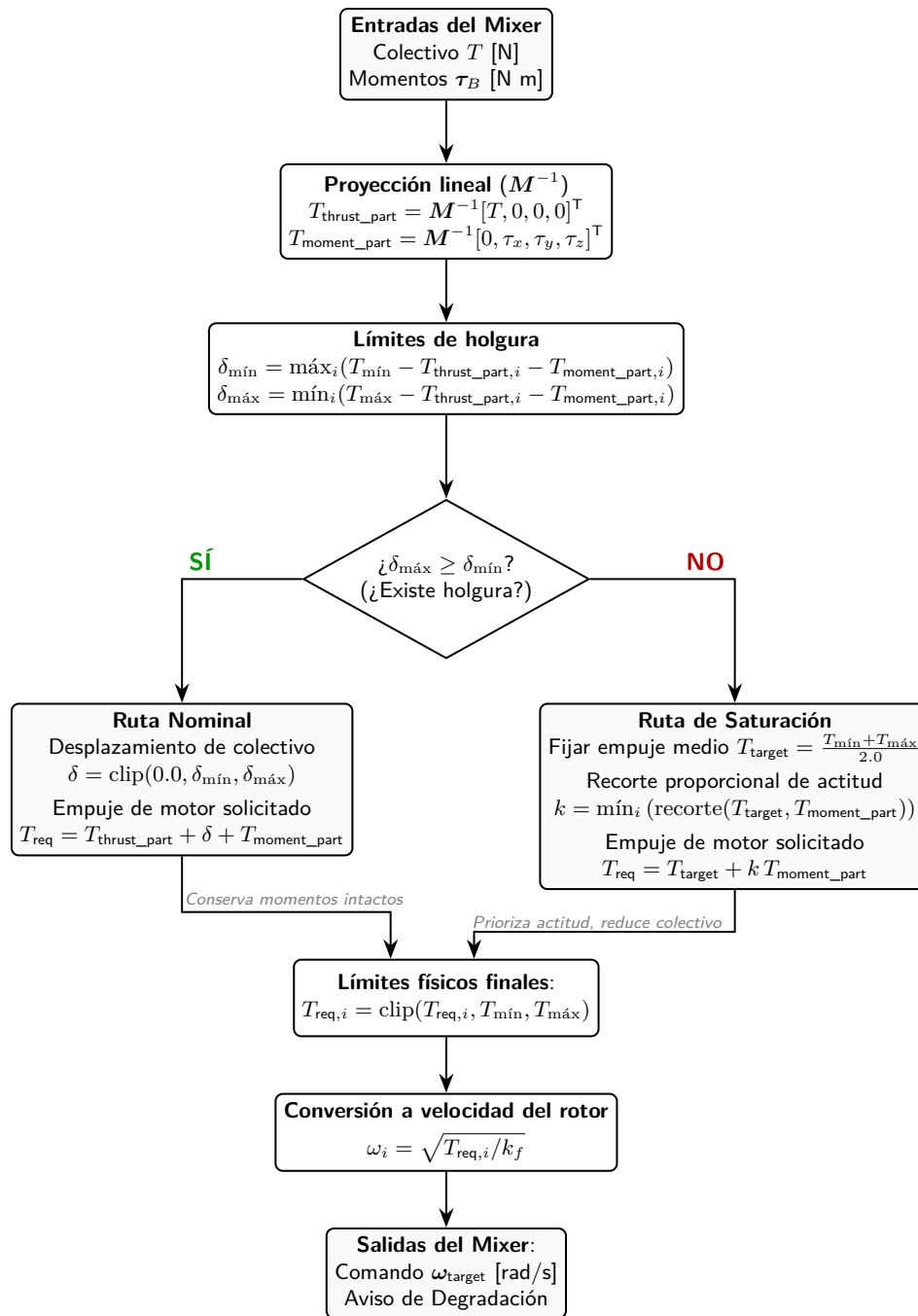


Figura 3.3. Esquema lógico y toma de decisiones del mezclador de rotores (mixer) ante condiciones nominales y saturadas. Se prioriza el control de actitud mediante el recorte proporcional de momentos (k) frente a la desviación del empuje colectivo solicitado. Fuente: elaboración propia.

A partir de estos márgenes de holgura, el algoritmo del mixer bifurca su ejecución en dos posibles rutas:

- **Ruta Nominal** ($\delta_{máx} \geq \delta_{mín}$): Existe un rango para ajustar el colectivo sin perturbar los momentos. El desplazamiento se halla mediante $\delta = \text{clip}(0.0, \delta_{mín}, \delta_{máx})$,

dando lugar al empuje solicitado por rotor:

$$T_{\text{req},i} = T_{\text{thrust_part},i} + \delta + T_{\text{moment_part},i}. \quad (3.12)$$

- **Ruta de Saturación** ($\delta_{\text{máx}} < \delta_{\text{mín}}$): No existe ninguna solución factible y los límites de los motores impiden satisfacer simultáneamente el colectivo y la actitud. En esta condición, el mixer prioriza la estabilidad frente a la consigna de empuje colectivo. Se fija un empuje medio de referencia $T_{\text{target}} = (T_{\text{mín}} + T_{\text{máx}})/2.0$ y se aplica un recorte proporcional $k \in [0, 1]$ a la parte de momentos para asegurar que ningún rotor sature:

$$k = \min_i (\text{recorte}(T_{\text{target}}, T_{\text{moment_part},i})), \quad (3.13)$$

donde la función de recorte evalúa la holgura disponible hasta los límites de empuje en función del sentido de giro y actuación del rotor:

$$\text{recorte}(T_{\text{target}}, T_{\text{moment_part},i}) = \begin{cases} \frac{T_{\text{máx}} - T_{\text{target}}}{T_{\text{moment_part},i}} & \text{si } T_{\text{moment_part},i} > 0, \\ \frac{T_{\text{mín}} - T_{\text{target}}}{T_{\text{moment_part},i}} & \text{si } T_{\text{moment_part},i} < 0. \end{cases} \quad (3.14)$$

El empuje resultante para cada motor es:

$$T_{\text{req},i} = T_{\text{target}} + k T_{\text{moment_part},i}. \quad (3.15)$$

En cualquiera de los dos casos en que la consigna original deba modificarse para cumplir los límites físicos de los actuadores, el simulador registra una degradación del mando. Finalmente, los empujes se limitan de forma estricta mediante $T_{\text{req},i} = \text{clip}(T_{\text{req},i}, T_{\text{mín}}, T_{\text{máx}})$ y se transforman en velocidades angulares de consigna del rotor $\omega_i = \sqrt{T_{\text{req},i}/k_f}$.

La dinámica de rotor combina un retardo puro discretizado y un lag de primer orden. El retardo puro retrasa la consigna un número entero de pasos del lazo de física, mientras que el lag aproxima que la velocidad del rotor no alcanza instantáneamente el comando, sino que se aproxima a él con una constante de tiempo τ . En las ejecuciones se emplean, según el perfil de la trayectoria, valores de $\tau = 0.03\text{s}$ con retardo de 0.01s y $\tau = 0.05\text{s}$ con retardo de 0.02s .

Perturbaciones, observación y seguridad

El drag lineal se calcula en el sistema de referencia FRD a partir de la velocidad relativa respecto al viento:

$$\mathbf{v}_{r,B} = \mathbf{R}_{WB}^T (\mathbf{v}_W - \mathbf{v}_{w,W}), \quad (3.16)$$

y se vuelve a expresar en ENU como

$$\mathbf{F}_{d,W} = \mathbf{R}_{WB} (-\mathbf{D}_B \mathbf{v}_{r,B}). \quad (3.17)$$

En el simulador se han implementado dos niveles de trayectorias con distinta disipación. También puede aplicarse un viento definido por una magnitud, dirección y sentido fijos durante toda la ejecución. La observación que recibe el controlador puede incluir una pequeña componente gaussiana que representa de forma simplificada que la señal observada no es perfecta; mientras que la dinámica base, la integración y la evaluación continúan usando el estado real. La semilla del escenario fija la inicialización pseudoaleatoria, de modo que un mismo YAML puede reproducirse.

La simulación termina si el vehículo impacta con el suelo, si la inclinación de roll/pitch supera el límite configurado, si aparece un valor no finito o si la saturación de actuadores persiste más que el tiempo permitido. En las ejecuciones se usa un umbral de 2 s para saturación persistente y un límite de inclinación de 1.256 rad. La ejecución del simulador mantiene además límites internos de posición y velocidad para detener trayectorias numéricamente divergentes.

Trayectorias y escenarios reproducibles

Las familias de trayectorias se eligieron para introducir una variabilidad suficiente sin convertir este módulo en un problema excesivamente complejo. En la [Figura 3.4](#) se muestra un caso representativo de cada familia. **hold** exige mantener una posición y una guiñada constantes. **circle** impone una trayectoria horizontal con aceleración centrípeta y avance continuo, que introduce un efecto no periódico con el viento. **lissajous** introduce un movimiento periódico acoplado entre ejes:

$$x(t) = c_x + A_x \sin(\omega_x t), \quad (3.18)$$

$$y(t) = c_y + A_y \sin(\omega_y t), \quad (3.19)$$

$$z(t) = c_z + A_z \sin(\omega_z t). \quad (3.20)$$

Al usar frecuencias distintas por eje, la referencia cubre una región espacial más rica que un círculo y obliga al controlador a seguir cambios simultáneos de posición, velocidad y aceleración.

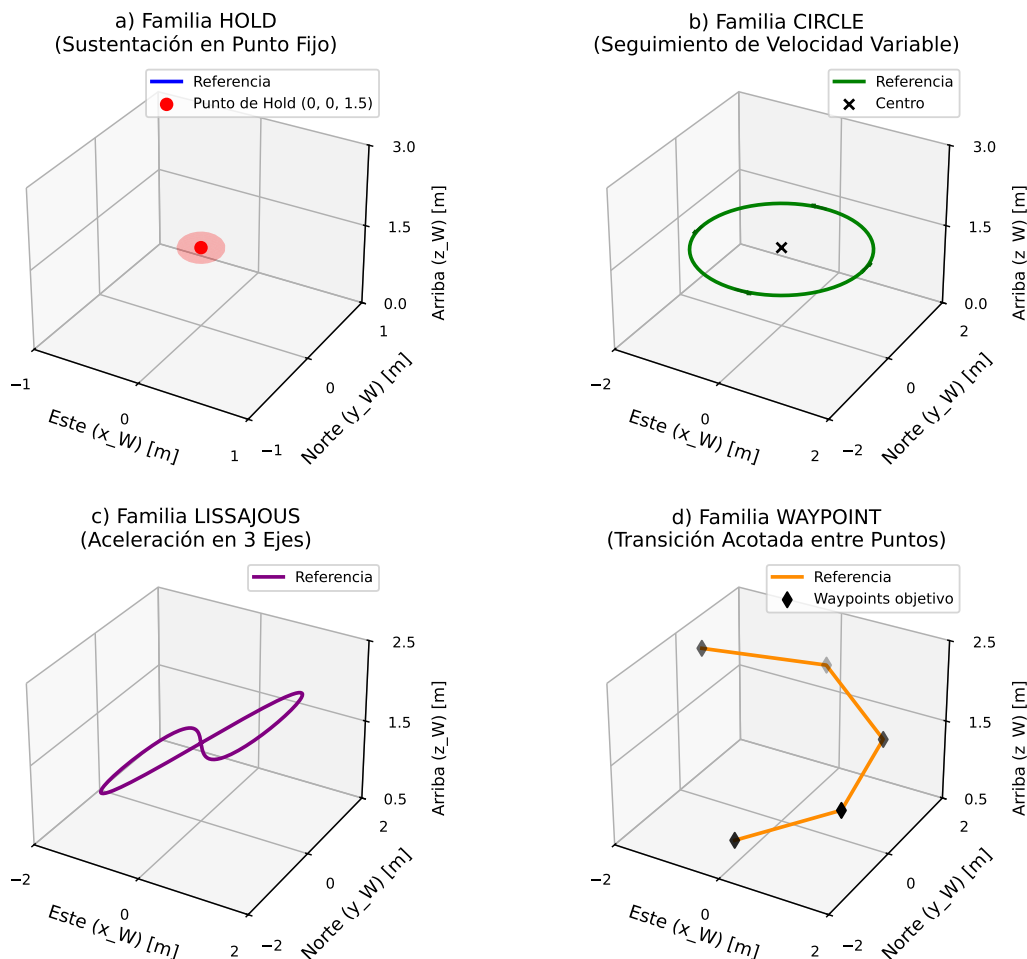


Figura 3.4. Familias de trayectorias analíticas del simulador. Cada geometría introduce estímulos diferentes (estacionario, velocidad variable, aceleración acoplada en 3 ejes o tramos discretos). Fuente: elaboración propia.

La familia **waypoint** define una trayectoria punto a punto. El vehículo debe alcanzar una serie de posiciones, frenar, asentarse dentro de una tolerancia de posición y velocidad y permanecer allí durante un tiempo de asentamiento configurable (con un valor por defecto de 0.40 s) antes de avanzar al siguiente punto. Para cada tramo se genera una coordenada escalar $s(t)$ sobre la recta que une dos waypoints. Si la distancia permite alcanzar la velocidad máxima, el perfil es trapezoidal, con aceleración constante, tramo de velocidad constante y frenado. Si el tramo es corto, el perfil es triangular y no se alcanza la velocidad máxima. En ambos casos la posición y la velocidad son continuas, evitando escalones en la referencia. En la [Figura 3.5](#) se muestran los perfiles cinemáticos de ambas casuísticas.

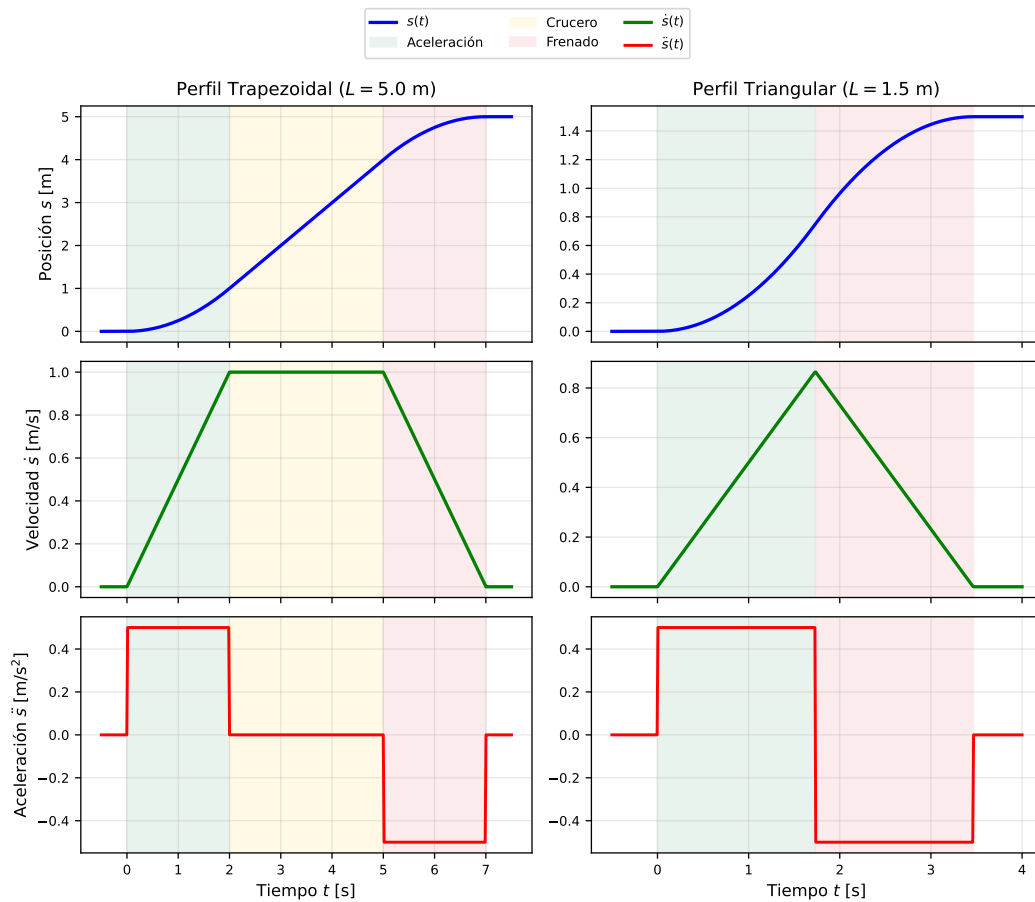


Figura 3.5. Perfiles cinemáticos generados en las transiciones entre waypoints. A la izquierda se ilustra el perfil trapezoidal (para tramos largos) y a la derecha el perfil triangular (para tramos cortos, donde no se alcanza la velocidad de consigna). Todas las magnitudes representadas son escalares sobre el eje de avance del tramo. Fuente: elaboración propia.

```

1 name: "Waypoint Clean"
2 seed: 42
3 vehicle: {mass_kg: 1.0, rotors: [...]}
4 initial_state: {position_W_m: [0, 0, 0], velocity_W_m_s: [0, 0, 0]}
5 trajectory:
6   type: "waypoint"
7   waypoints: [[0, 0, 0], [0, 0, 2], [2, 0, 2]]
8   max_speed_m_s: 0.6
9   max_acceleration_m_s2: 0.5
10 controller: {type: "classic"}
11 perturbations: {constant_wind_W_m_s: [0, 0, 0], pos_std_m: 0.0}
12 timing: {physics_dt_s: 0.01, control_dt_s: 0.02, telemetry_dt_s: 0.1}
13 termination: {max_duration_s: 40.0, z_min_m: -0.1}

```

Listing 1: Estructura simplificada de un escenario YAML.

En cada fichero YAML, como se muestra en el [Listing 1](#), se fija el vehículo, el estado inicial, la trayectoria, el controlador, las perturbaciones, la semilla, los periodos de ejecución y las condiciones de terminación. Por convención, el estado inicial se iguala a la referencia de la trayectoria en $t = 0$, con actitud nivelada y empuje inicial de situación estacionaria. Así se evalúa el seguimiento de la trayectoria y no la recuperación desde una caída inicial provocada por arrancar con los rotores apagados.

La especificación de la cantidad y características de las trayectorias de cada familia usadas para entrenamiento, validación y prueba (train, validation, test) se desarrolla en el [epígrafe de diseño del conjunto de datos clásico](#). Adicionalmente, el simulador permite composiciones secuenciales de trayectorias de las familias descritas, así como lemniscatas. Estas variantes se desarrollan posteriormente dentro del contexto de evaluación OOD, distinguiendo entre [variaciones y composiciones de familias conocidas](#) y [trayectorias completamente nuevas](#).

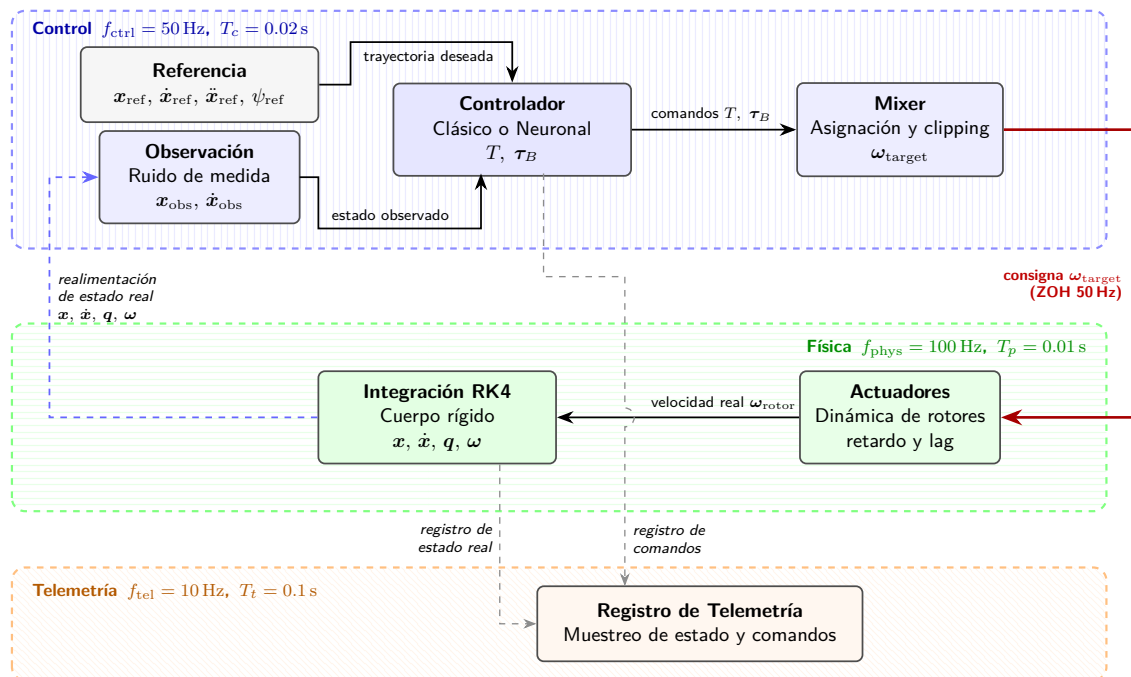


Figura 3.6. Secuencia de ejecución temporal multirate y dependencias de datos. Se diferencian las tres bandas de frecuencia asociadas a la integración física (100 Hz), el ciclo digital de control (50 Hz) y el registro periódico de variables (10 Hz). Fuente: elaboración propia.

Flujo multirate de simulación

El simulador trabaja con un flujo multirate, es decir, con varios periodos de actualización dentro de una misma ejecución, como se puede observar en la [Figura 3.6](#). La física se integra cada 0.01 s, el controlador calcula un nuevo comando cada 0.02 s y la telemetría se guarda cada 0.1 s. Cabe destacar que, al igual que el resto de parámetros,

estas frecuencias son configurables en el YAML del episodio. En cada instante se obtiene la referencia, se forma la observación y, cuando corresponde, se calcula un comando nuevo. El mixer y los actuadores producen la fuerza y el momento aplicados a partir del último comando disponible, RK4 avanza el estado y la telemetría registra las variables.

Entre dos ciclos de control se aplica un bloqueador de orden 0, de manera que el último comando solicitado por el controlador se mantiene constante hasta la siguiente actualización. Esto trata de aproximar un sistema real, donde la física es continua y los sensores, controladores y actuadores operan a frecuencias finitas.

Telemetría

Cada toma de telemetría contiene el tiempo de simulación, el estado, la observación que recibe el controlador, la referencia, el comando de control, el comando de cada rotor y la actuación aplicada. En los rotores se cuenta con el empuje y la velocidad tanto objetivo como aplicado, par de reacción y revoluciones por minuto. También se incluye el motivo de finalización de la simulación en la última toma.

Esta estructura permite el estudio posterior del error de seguimiento, la diferencia entre estado y observación, estado del mixer, saturación de actuadores y el efecto del retardo y lag del rotor. Adicionalmente se cuenta con un resumen de valores estadísticos así como los datos de configuración de la simulación.

Elección del entorno de implementación

La implementación se ha realizado en Python ya que ofrece un equilibrio adecuado entre capacidades, facilidad de uso y disponibilidad. Es un lenguaje de programación abierto, sin coste de licencia, con una amplia cantidad de programadores y numerosas librerías científicas. Se cuenta con PyTorch, una librería de altas capacidades y flexibilidad para las labores de entrenamiento y trabajo con redes neuronales. Asimismo, Python permite trabajar de forma natural con archivos externos usados en el desarrollo, como CSV, YAML, JSON, Markdown y LaTeX.

La gestión del entorno se realiza con `uv`, de manera que las dependencias de paquetes y comandos de reproducción quedan definidos de forma clara y sencilla para que otros usuarios puedan usarlo. El trabajo en su conjunto ha sido desarrollado de forma abierta en un repositorio de GitHub, lo que facilita la inspección de código, trazabilidad de cambios y reutilización del material. ¹ [11, 12, 16].

No obstante, no se afirma que Python sea la mejor opción para todas las fases del trabajo. Una implementación en C o C++ contaría con unos requisitos de rendimiento menores y sería ideal para una plataforma embarcada o para ejecución en tiempo real, si bien aumentaría la complejidad del desarrollo, especialmente con las redes neuronales.

¹<https://github.com/ChristianGonper/modelado-control-dron-v2>

Dado que el objetivo central es comparar estrategias de control en un entorno de simulación asíncrono, Python resulta suficiente y adecuado para el desarrollo.

3.2. CONTROL CLÁSICO EN CASCADA

Arquitectura PD en cascada

El controlador clásico cumple tres funciones. En primer lugar, proporciona la referencia contra la que se compararán las implementaciones neuronales. En segundo lugar, genera los datos para el propio aprendizaje por imitación. Y, en tercer lugar, aporta el lazo interno de actitud que acompaña al lazo externo de tipo neuronal.

La estructura se separa en dos lazos. Un lazo externo para traslación, que recibe la observación de posición y velocidad y la referencia de la trayectoria en el sistema ENU, y calcula una fuerza deseada $F_{d,W}$. El lazo interno recibe esa fuerza y la guiñada de referencia, construye una actitud compatible con la dirección del empuje y calcula los momentos en el sistema FRD. Por último, el mixer previamente descrito en el [epígrafe de actuadores, mixer y límites de actuación](#) transforma el empuje colectivo y los momentos en comandos de rotor. Este flujo se puede ver en la [Figura 3.7](#).

Mediante esta separación se observa de forma clara qué parte será sustituida por aprendizaje y por qué se dice que emplea la misma interfaz. Tanto el lazo externo clásico como la red neuronal reciben los errores de posición y velocidad y entregan la fuerza deseada. Cabe mencionar que la red recibirá como entrada también la aceleración requerida por la referencia, mientras que la compensación de gravedad se añadirá de la misma manera que con el lazo externo clásico. A partir de la salida de fuerza, el resto del flujo permanecerá idéntico.

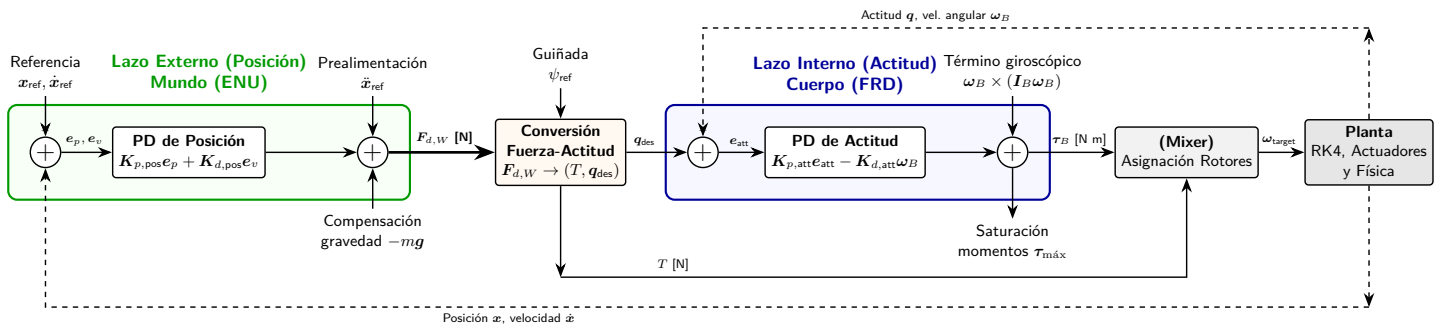


Figura 3.7. Arquitectura del controlador clásico en cascada implementado. Se destacan el lazo externo (posición en mundo ENU), la conversión geométrica de fuerza a actitud y el lazo interno (estabilización de actitud en cuerpo FRD). La frontera de sustitución neuronal se sitúa en la señal de fuerza deseada $F_{d,W}$. Fuente: elaboración propia.

Ecuaciones, ganancias y límites

La referencia de la trayectoria proporciona en cada instante posición, velocidad, aceleración y guiñada. En las familias continuas, la aceleración procede de la derivada analítica de la referencia mientras que en la familia **waypoint**, del perfil descrito e ilustrado en la [Figura 3.5](#).

Con $\mathbf{e}_p = \mathbf{p}_r - \mathbf{p}$ y $\mathbf{e}_v = \mathbf{v}_r - \mathbf{v}$ representando los errores de posición y velocidad, respectivamente, donde $\mathbf{p}, \mathbf{v} \in \mathbb{R}^3$ corresponden a la posición y velocidad observadas del vehículo en el sistema ENU, y $\mathbf{p}_r, \mathbf{v}_r \in \mathbb{R}^3$ son la posición y velocidad de referencia dictadas por la trayectoria, el lazo externo calcula el comando de aceleración $\mathbf{a}_{cmd}$ y la fuerza deseada en ENU $\mathbf{F}_{d,W}$ mediante:

$$\mathbf{a}_{cmd} = \mathbf{K}_{p,p} \odot \mathbf{e}_p + \mathbf{K}_{d,p} \odot \mathbf{e}_v + \mathbf{a}_r, \quad (3.21)$$

$$\mathbf{F}_{d,W} = m \left(\mathbf{a}_{cmd} - [0 \ 0 \ -g]^\top \right). \quad (3.22)$$

Aquí, $\mathbf{a}_r \in \mathbb{R}^3$ representa la aceleración de referencia, m es la masa del cuadricóptero, g la aceleración de la gravedad y el símbolo $\odot$ indica producto componente a componente. La aceleración de referencia actúa como prealimentación de la trayectoria y el término gravitatorio convierte la aceleración cinemática deseada en fuerza física. En un estacionario, por ejemplo, $\mathbf{a}_r = \mathbf{0}$ y los errores nulos conducen a $\mathbf{F}_{d,W} = [0 \ 0 \ mg]^\top$, es decir, a la fuerza ascendente necesaria para compensar el peso.

La norma de $\mathbf{F}_{d,W}$ define el empuje colectivo solicitado, limitado en el controlador a $[0, 2.5mg]$. Como se usa un sistema FRD, el empuje actúa en dirección $-z_B$, de manera que para que el vehículo genere una fuerza $\mathbf{F}_{d,W}$, el eje z_B deseado se toma opuesto a dicha fuerza:

$$\mathbf{z}_{B,d} = -\frac{\mathbf{F}_{d,W}}{\|\mathbf{F}_{d,W}\|}.$$

En caso de que la norma sea prácticamente nula, entonces la actitud se establece como nivelada.

La guiñada de referencia completa la orientación deseada. Primero se forma un eje horizontal auxiliar $\mathbf{x}_C = [-\sin \psi_r \ \cos \psi_r \ 0]^\top$ y obtiene $\mathbf{y}_{B,d}$ como producto vectorial entre $\mathbf{z}_{B,d}$ y $\mathbf{x}_C$, normalizándolo cuando es posible. Después se calcula $\mathbf{x}_{B,d} = \mathbf{y}_{B,d} \times \mathbf{z}_{B,d}$ y se construye la matriz $\mathbf{R}_{WB,d}$ con esos tres ejes como columnas. A continuación, esta matriz se transforma a cuaternión para comparar la actitud deseada con la actitud observada.

La estructura en cascada y la conversión de una fuerza a actitud son coherentes con formulaciones habituales de control de cuadricópteros [\[1, 2\]](#). El error de actitud se calcula como $\mathbf{q}_e = \mathbf{q}_{WB}^{-1} \otimes \mathbf{q}_{WB,d}$, donde $\mathbf{q}_{WB}$ es el cuaternión que describe la actitud observada del cuerpo $\mathcal{B}$ con respecto al mundo $\mathcal{W}$, $\mathbf{q}_{WB,d}$ es la actitud deseada calculada

en el paso anterior y el operador $\otimes$ representa el producto de cuaterniones. En caso de que la parte escalar del cuaternión de error sea negativa, este se cambia de signo para tomar la representación equivalente de menor giro. Para errores pequeños, la desviación angular se aproxima mediante el vector $\mathbf{e}_q = 2\mathbf{q}_{e,v}$, donde $\mathbf{q}_{e,v} \in \mathbb{R}^3$ es la parte vectorial del cuaternión $\mathbf{q}_e$. Asumiendo una velocidad angular de referencia nula, el momento solicitado en el sistema FRD $\boldsymbol{\tau}_B \in \mathbb{R}^3$ se define como:

$$\boldsymbol{\tau}_B = \mathbf{K}_{p,a} \odot \mathbf{e}_q + \mathbf{K}_{d,a} \odot (-\boldsymbol{\omega}_B) + \boldsymbol{\omega}_B \times \mathbf{I}_B \boldsymbol{\omega}_B. \quad (3.23)$$

Aquí, $\mathbf{K}_{p,a}$ y $\mathbf{K}_{d,a}$ son los vectores de ganancias proporcional y derivativa del lazo de actitud, $\boldsymbol{\omega}_B \in \mathbb{R}^3$ es la velocidad angular del vehículo en FRD y $\mathbf{I}_B$ es su matriz de inercia respecto al centro de masas. El último término compensa el acoplamiento giroscópico que aparece en la dinámica rotacional descrita en el [epígrafe de cinemática y dinámica del cuerpo rígido](#). El resultado se satura por componente con los límites declarados en el controlador.

Todos los controladores parten de unas ganancias por defecto que posteriormente se ajustarán y quedarán fijadas en un PD especializado por familia en archivos `pid_<familia>_v1.yaml`. Y como se puede observar, el controlador no incluye término integral. Esto se debe al riesgo de acumular mando durante las fases con saturación o cambios continuos de referencia, lo que exige mecanismos adicionales como anti-windup.

Elección del método de sintonización

El objetivo del ajuste no es encontrar el óptimo analítico para una trayectoria aislada, sino obtener un controlador adecuado para una familia completa de trayectorias, con variantes de parámetros, perturbaciones y dinámica de actuadores. Por este motivo, la sintonización se evalúa directamente en simulación sobre un conjunto de trayectorias de entrenamiento y se emplean las métricas de seguimiento, actitud, esfuerzo de control, saturación, degradación y finalización.

La alternativa de ajuste manual produciría un controlador válido si quedasen registradas todas las iteraciones, pero aportaría menos trazabilidad que una búsqueda reproducible. Reglas clásicas como Ziegler–Nichols se conciben para sistemas y criterios de respuesta que no representan directamente un cuadricóptero con lazos acoplados, saturaciones y trayectorias espaciales variables. Un diseño basado en modelo linealizado también sería posible, pero obligaría a fijar puntos de operación y criterios locales distintos de la evaluación que se quiere ejecutar.

Algoritmo de búsqueda progresiva

La búsqueda comienza con un diagnóstico por familia sobre el conjunto de entrenamiento definido en el [diseño del conjunto de datos clásico](#). En la [Figura 3.8](#) se ilustra un

esquema de este algoritmo. Para cada familia se toman dos geometrías representativas, una lenta y otra exigente, y se combinan con tres perfiles, nominal, viento Este y perturbación combinada. Si el controlador inicial supera los filtros en todos los casos y su RMSE medio queda por debajo del umbral seleccionado para la familia, se conserva sin reajuste. Los umbrales, establecidos por heurística, son de 0.25 m, 0.35 m, 0.45 m y 0.40 m para *hold*, *circle*, *lissajous* y *waypoint* respectivamente.

Cuando el diagnóstico exige ajuste, se modifican de forma conjunta los cuatro bloques de ganancias, $K_{p,p}$, $K_{d,p}$, $K_{p,a}$ y $K_{d,a}$. Cada candidato se define mediante cuatro multiplicadores que escalan esos bloques completos. La primera ronda evalúa 32 candidatos con multiplicadores log-uniformes entre 0.5 y 2, además del candidato inicial. A continuación se seleccionan los mejores candidatos de esa ronda y se generan 16 perturbaciones locales alrededor de ellos, de manera que cada familia cuenta con 49 evaluaciones para sintonizar su controlador. Esta segunda fase se plantea como un refinamiento local tipo simplex, inspirado en la lógica de Nelder–Mead [27] para explorar el entorno de los mejores candidatos pero sin llegar a la convergencia.

Antes de comparar las puntuaciones de los controladores se aplican filtros. Se descartan candidatos con métricas ausentes o no finitas, finalización no válida, más del 2 % de saturación, más del 2 % de degradación del mando, o un error máximo superior al límite.

Los candidatos restantes reciben la puntuación

$$J = e_{\text{rms}} + 0.5e_{\text{máx}} + 0.2e_{\text{att}} + 0.1u + 2s + 2d, \quad (3.24)$$

donde e_{rms} y $e_{\text{máx}}$ son métricas de posición, e_{att} es la RMS de los ángulos de actitud, s y d son las fracciones de muestras con saturación y degradación, y

$$u = \frac{\bar{T}}{mg} + \frac{\|\overline{\boldsymbol{\tau}_B}\|}{0.1}.$$

El primer término de u normaliza el empuje medio con el peso del vehículo y el segundo usa 0.1 N m como escala para los momentos. Esta métrica busca favorecer controladores que requieren una menor actuación.

La selección final se realiza sobre la menor puntuación del conjunto. Dentro de un 5 % de la mejor se escoge el controlador de menor esfuerzo medio, y si aún hay empate se prefiere el más cercano al controlador inicial en término de multiplicadores. De esta manera, se favorecen los cambios moderados a igualdad de resultado.

Los pesos de la [ecuación \(3.24\)](#), los umbrales, el intervalo de multiplicadores, el número de candidatos y el margen del 5 % son decisiones heurísticas de la ejecución. No son constantes derivadas de una teoría de control ni demuestran una optimización

global. Deben interpretarse como un óptimo local bajo un presupuesto de cómputo razonable y con criterios declarados antes de evaluar test u OOD.

Controladores con parámetros fijados y transferencia cruzada

El ajuste solo consulta escenarios del conjunto de entrenamiento. Una vez elegido el candidato de una familia, sus ganancias, origen, conjunto de trayectorias de diagnóstico y métricas se guardan en un YAML. Las ejecuciones posteriores, escenarios de validación, prueba y OOD usan esos archivos como conjunto de controladores con parámetros fijados, sin modificarlos.

Se denomina PD especializado al controlador cuyos parámetros se han fijado para la misma familia que la trayectoria evaluada. La transferencia cruzada consiste en ejecutar un escenario con los PD de las demás familias sin reajustarlos, manteniendo el resto de la configuración fija.

La matriz de transferencia permite cuantificar la importancia de disponer de un PD optimizado por familia. También define una referencia para el controlador neuronal, que debe compararse con los PD transferidos al evaluar la capacidad de usar la red para condiciones distintas.

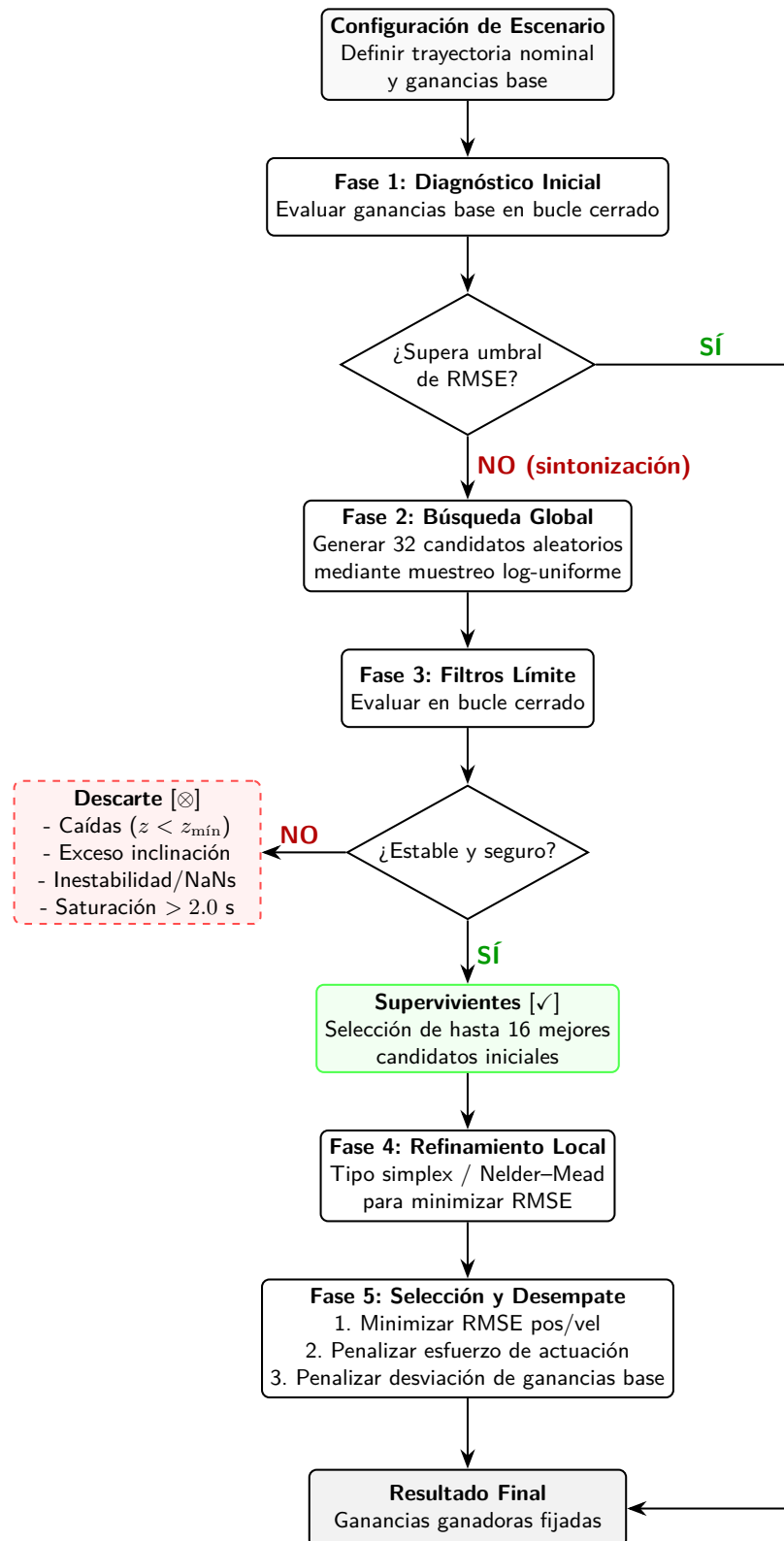


Figura 3.8. Flujo lógico de optimización y ajuste progresivo de ganancias clásicas. Los candidatos aleatorios globales se filtran por condiciones de estabilidad y seguridad antes de que se inicie el refinamiento local tipo simplex/Nelder–Mead. Fuente: elaboración propia.

3.3. CONTROL NEURONAL POR IMITACIÓN

Formulación híbrida de predicción de fuerza deseada

La implementación neuronal adoptada aprende la decisión de traslación que en el controlador clásico define el lazo externo. En cada ciclo de control recibe variables construidas a partir de la observación y de la referencia, y predice una fuerza deseada $\hat{\mathbf{F}}_{d,W} \in \mathbb{R}^3$ en el sistema ENU. Posteriormente, esta salida sigue los mismos límites y componentes clásicos que en la implementación previa. En la [Figura 3.9](#) se representa el flujo de funcionamiento.

La interfaz compartida permite comparar de forma directa la decisión aprendida y la decisión clásica. Por otro lado, una arquitectura donde la red neuronal actuase directamente sobre los rotores contaría con un alcance más amplio y mezclaría el aprendizaje de la capacidad de seguimiento con la estabilización.

La arquitectura híbrida obliga a separar las contribuciones clásica y neuronal en los resultados. Por este motivo, la evaluación no cuantifica únicamente el error de seguimiento sino las saturaciones y límites de actitud. De esta manera, se tiene una aproximación sobre las acciones observadas en el entrenamiento así como las limitaciones de la dependencia de la propia distribución de entrenamiento [19].

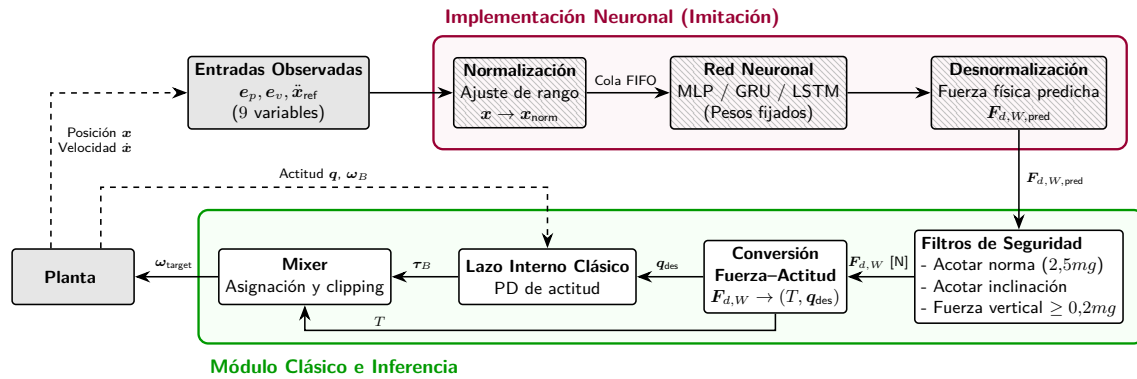


Figura 3.9. Estructura del controlador neuronal híbrido. La red neuronal sustituye únicamente la decisión traslacional del lazo externo para generar $\mathbf{F}_{d,W}$, delegando el filtrado de seguridad, la conversión fuerza-actitud y la estabilización en el lazo clásico interno. Fuente: elaboración propia.

Construcción y selección de datos de imitación

El conjunto de datos neuronal se construye en una fase posterior al ajuste de los PD clásicos. Primero, el flujo clásico fija los escenarios, trayectorias, perturbaciones, semillas y particiones entrenamiento-validación-prueba, así como los controladores de referencia por familia. Después, se toma cada escenario y se construye un conjunto

de variantes del lazo externo, modificando exclusivamente las ganancias $\mathbf{K}_{p,p}$ y $\mathbf{K}_{d,p}$ y conservando el resto de parámetros.

Las variantes implementadas escalan las ganancias mediante cinco perfiles, conservador, base, agresivo y dos perfiles más amortiguados. En términos relativos, estos perfiles aplican respectivamente los factores (0.7, 0.9), (1.0, 1.0), (1.3, 1.2), (1.6, 2.0) y (1.4, 1.8) sobre $(\mathbf{K}_{p,p}, \mathbf{K}_{d,p})$. Cada variante se ejecuta en el mismo escenario y se evalúa con las métricas de la simulación, considerando únicamente aquellos que superan los filtros.

Entre los candidatos de cada escenario se identifica primero el menor RMSE de posición. Se admiten después los candidatos situados hasta un 5 % por encima de ese valor y, dentro de ese subconjunto, se escoge el de menor esfuerzo de control y, si persiste el empate, se escoge el perfil más conservador. Esta segunda ronda de ajuste permite generar objetivos de imitación sin modificar el lazo interno de la referencia clásica cuyos parámetros se habían fijado previamente.

Este procedimiento debe interpretarse como una clonación de comportamiento. La red observa las acciones de los PD externos y aprende a aproximar sus fuerzas, no optimiza directamente el error de trayectoria ni posee una función de recompensa por seguimiento. En consecuencia, también puede heredar sesgos o errores de los PD en regiones concretas de la trayectoria. A su vez, si el conjunto de entrenamiento contiene errores grandes o estados transitorios suficientemente representados la red puede aprender respuestas en esa cobertura más agresiva alejada de la referencia. La distinción de la naturaleza de lo que aprende la red a ejecutar justifica la evaluación doble de la fidelidad supervisada y el comportamiento en el bucle cerrado.

Tensores, variables de entrada, objetivos y secuencias

Los datos de entrada representan cada instante mediante nueve variables:

$$\mathbf{x}_k = [\mathbf{e}_{p,k}^\top \mathbf{e}_{v,k}^\top \mathbf{a}_{r,k}^\top]^\top. \quad (3.25)$$

Los errores de posición $\mathbf{e}_{p,k}$ y velocidad $\mathbf{e}_{v,k}$ se calculan con la observación entregada al controlador, y la tercera parte del vector es la aceleración de referencia en ENU. El objetivo supervisado es la fuerza deseada de tres componentes que calcularía el PD externo con esas mismas variables, equivalente a la [ecuación \(3.22\)](#). Por tanto, existe una correspondencia directa, con el matiz de cómo se trata la aceleración de referencia, entre el PD y la red neuronal.

Podrían añadirse variables de actitud, actuación, guiñada o estado observado completo. Sin embargo, el objetivo de esta comparación es imitar el lazo externo y no proporcionar a la red información que el lazo no necesita para calcular la fuerza. Esta elección reduce la dimensionalidad y mantiene la interpretación como sustitución del PD externo.

La MLP procesa tensores de forma $[B, 9]$, donde B es el tamaño del lote de entrenamiento. GRU y LSTM reciben tensores $[B, L, 9]$, con $L = 20$, y producen la fuerza correspondiente al último instante de la ventana. Como el periodo de control es 0.02 s, la ventana recurrente cubre 0.40 s de muestra. En entrenamiento, las ventanas se construyen dentro de cada simulación mediante una ventana deslizante y se mantienen acotadas al conjunto de entrenamiento y sin atravesar a otras simulaciones. Cada vez que se añade una nueva muestra, se elimina la última en la cola, manteniendo el tamaño en 20. En inferencia, al comienzo de una simulación, dado que todavía no existen muestras previas, se rellena la cola con la primera entrada hasta rellenar la longitud.

La inferencia recurrente usada en este trabajo no es *stateful* en el sentido estricto. GRU y LSTM no conservan entre ciclos el estado oculto calculado en la llamada anterior para usarlo directamente como estado inicial de la siguiente. En cada ciclo se reconstruye una ventana con las últimas veinte entradas normalizadas y se procesa completa, obteniendo la fuerza aplicada en el último instante de esa ventana. Así, dos ejecuciones con la misma cola de entradas producen la misma salida, sin depender de un estado interno acumulado fuera de la ventana. En la [Figura 3.10](#) se ilustra este funcionamiento.

La ventana temporal permite estudiar si la historia reciente aporta información útil frente a retardos y dinámica de actuadores. El valor $L = 20$ fija una ventana corta y común para las 2 arquitecturas. Como comprobación, se ejecutó un estudio de sensibilidad con $L = 10$ y $L = 40$ para ambas redes y las diferencias observadas no fueron suficientes para justificar un cambio en la configuración.

Arquitecturas y parámetros de MLP, GRU y LSTM

La MLP emplea dos capas ocultas de 64 unidades con activación ReLU y una capa lineal final de tres salidas. Con nueve entradas, el número de parámetros se obtiene sumando pesos y sesgos de cada capa, $(9 \cdot 64 + 64) + (64 \cdot 64 + 64) + (64 \cdot 3 + 3) = 4995$. GRU y LSTM usan una única capa recurrente de 64 unidades y una proyección lineal final a tres componentes. Con las fórmulas de PyTorch para una capa recurrente, sus tamaños son 14 595 y 19 395 parámetros entrenables, respectivamente. Y como se indicó, la salida proviene del último instante de la ventana.

La MLP actúa como la referencia sin memoria explícita, de manera que dos entradas idénticas producen la misma fuerza; mientras que GRU y LSTM permiten comprobar si la historia reciente mejora la predicción. La anchura de 64 unidades, las dos capas ocultas de la MLP y la única capa recurrente se mantienen fijas durante toda la evaluación. La decisión de los parámetros se contrastó con una variante de 128 unidades; no obstante, la mejora supervisada no produjo una ventaja sistemática, por lo que se decidió no modificar los parámetros.

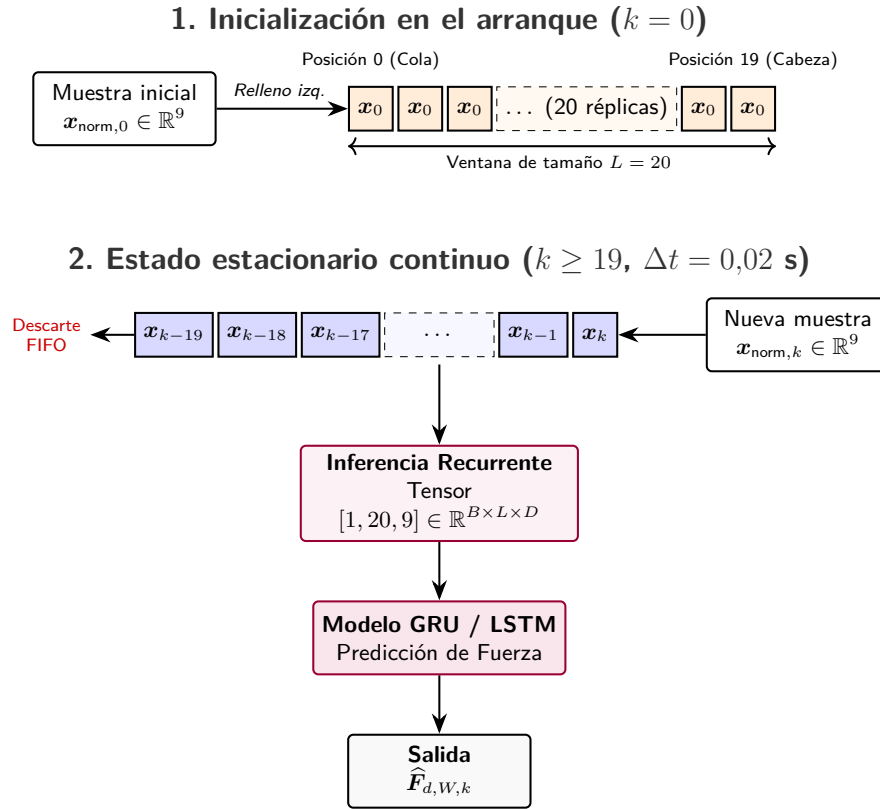


Figura 3.10. Gestión de la ventana temporal deslizante en redes recurrentes e inicialización mediante repetición de muestra en el arranque ($k = 0$). La cola FIFO genera un lote de dimensiones $[1, 20, 9]$ en cada inferencia. Fuente: elaboración propia.

Normalización, entrenamiento y selección del modelo

Antes del entrenamiento se calcula la media y la desviación típica de entradas y objetivos usando únicamente el conjunto de entrenamiento. Esta normalización evita que variables con magnitudes absolutas mayores dominen la pérdida por mayor escala y se expresa cada componente como desviación respecto a los estadísticos calculados en entrenamiento. Estos mismos se aplican después a validación, prueba e inferencia, y la salida de la red se desnormaliza antes de entregarla al controlador.

El aprendizaje minimiza el error cuadrático medio (MSE) entre la fuerza normalizada predicha por la red y la fuerza normalizada calculada por el PD especializado. La optimización se realiza con Adam [21] y una tasa de aprendizaje 10^{-3} . Se configura un límite de 100 épocas, lotes de 64 muestras y semilla 42 para reproducibilidad del entrenamiento. La robustez frente a la semilla se comprobó con 2 valores adicionales y la variabilidad resultante no supuso un cambio significativo en la interpretación de resultados. Este orden aleatorio se aplica solo durante el entrenamiento, para reducir la dependencia del gradiente respecto a la secuencia de lectura de los episodios.

Tras cada época se calcula la pérdida de validación y se guarda el checkpoint con menor valor. El entrenamiento se detiene si el resultado de la validación no obtiene una mejora tras diez épocas consecutivas y se conserva el mejor modelo, reduciendo así el número de ejecuciones y limitando el sobreajuste.

Una pérdida supervisada baja indica que la red replica con precisión las fuerzas emitidas por los controlados especializados en las muestras observadas. No obstante, no implica necesariamente que minimice el error de posición en bucle cerrado ya que pequeñas desviaciones de fuerza se pueden acumular y hacer a la red visitar estados poco representados en el entrenamiento. Por ello, la evaluación supervisada únicamente comprueba la fidelidad al controlador clásico.

Inferencia en bucle cerrado

Durante la inferencia se construyen en cada ciclo las mismas variables de entrada, se normalizan con los estadísticos aprendidos en entrenamiento y se entregan a la red. La fuerza predicha se desnormaliza y se interpreta de nuevo como $\widehat{\mathbf{F}}_{d,w}$. En MLP esta operación depende solo del instante actual mientras que en GRU y LSTM se mantiene una cola de las últimas 20 entradas normalizadas, procediendo según se indica en [tensores de entradas y objetivos](#).

Antes de convertir la fuerza a una actitud deseada, al igual que en la implementación clásica, se limita la norma de la fuerza a $2.5mg$, se restringe la inclinación mediante el ángulo máximo y se evitan solicitudes de empuje negativo imponiendo una componente ascendente mínima. A continuación, el lazo interno clásico calcula momentos y se aplican las saturaciones en los rotores.

La telemetría registra cuántas veces se activa el recorte de empuje y cuántas el recorte de inclinación y se calculan porcentajes respecto al número de pasos de control ejecutados en la simulación. Estas métricas se interpretan junto con el resto de parámetros de salida, ya que una trayectoria puede completarse por una intervención frecuente de las protecciones, indicando un funcionamiento subóptimo de la red.

Alternativas implementadas fuera de la comparación

Durante el desarrollo se implementaron alternativas que no forman parte de la comparación principal del trabajo. Una de ellas aprende una programación de ganancias instantánea en tiempo real, la red predice multiplicadores para las ganancias externas $\mathbf{K}_{p,p}$ y $\mathbf{K}_{d,p}$, limitados a un intervalo prefijado, y el PD en cascada calcula después la fuerza, la actitud y los mandos.

También se implementó una variante de predicción de fuerza con observación ampliada. En lugar de usar solo las nueve variables necesarias para reproducir el PD externo, se incorporan 31 variables que incluyen estado observado, referencia, errores y una

representación continua de la guiñada. Puede servir para estudiar si el contexto adicional mejora la predicción, pero la red deja de comportarse como una sustitución estricta del lazo externo PD.

3.4. CONJUNTOS DE DATOS Y MÉTRICAS DE EVALUACIÓN

Criterios de comparación

El objetivo no es demostrar una superioridad de una arquitectura en todos los casos, sino determinar en qué condiciones un controlador neuronal puede sustituir a varios PD especializados del lazo externo, qué se pierde al transferir un PD fuera de su familia y qué papel desempeña la memoria temporal en redes GRU y LSTM.

La primera referencia es el PD especializado de cada familia. Su función es marcar el rendimiento esperable cuando el controlador clásico se ajusta para el tipo de trayectoria evaluado. La segunda referencia es la transferencia cruzada, aplicar esos mismos PD, con los parámetros ya fijados, a familias distintas sin reajuste. De esta manera se pueden separar la calidad del controlador en sus condiciones especializadas y la pérdida en el resto de situaciones.

Sobre esa base se evalúan las redes neuronales entrenadas por imitación. Primero se mide la fidelidad supervisada con la fuerza deseada de los PD, si bien este valor no sustituye a la ejecución en lazo cerrado. La evaluación de MLP, GRU y LSTM no presupone que la recurrencia deba mejorar necesariamente el seguimiento. La memoria temporal puede ser útil ante retardos o dependencia de estados anteriores, pero también puede introducir una sensibilidad distinta cuando la trayectoria se aleja de la distribución de entrenamiento.

Los resultados se juzgan con error de seguimiento, actuación, saturación y uso de limitaciones de manera independiente. Asimismo, se separa entre condiciones conocidas, transferencia clásica y escenarios OOD para identificar en qué parte del procedimiento aparece cada diferencia y su origen.

Diseño del conjunto de datos clásico

El conjunto contiene 150 episodios: 18 de `hold`, 48 de círculo, 48 de Lissajous y 36 de `waypoint`. Sus divisiones por familia son 12/3/3, 34/7/7, 34/7/7 y 25/5/6 para entrenamiento, validación y prueba. En total resultan 105 episodios de entrenamiento, 22 de validación y 23 de prueba. La asignación de escenarios a cada conjunto se genera de forma determinista usando la semilla 1042.

Las familias se eligen para cubrir equilibrio en situación estacionaria, seguimiento periódico en un plano, movimiento periódico en múltiples ejes y ejecuciones finitas

con aceleración, frenado y establecimiento. Cada una combina geometrías de distinta exigencia con perfiles de drag, viento, ruido y actuadores. **hold** usa 18 combinaciones y menos perfiles porque no introduce la diversidad geométrica del resto de familias y así evita una sobrerepresentación de muestras en el entrenamiento.

Métricas y criterios de éxito

Para las muestras $e_k = \|\mathbf{p}_{r,k} - \mathbf{p}_k\|_2$, se calculan para cada escenario

$$e_{\text{RMSE}} = \sqrt{\frac{1}{N} \sum_{k=1}^N e_k^2}, \quad e_{\text{MAE}} = \frac{1}{N} \sum_{k=1}^N e_k, \quad e_{\text{máx}} = \max e_k. \quad (3.26)$$

RMSE penaliza errores grandes, MAE describe el error medio y el máximo detecta extremos puntuales.

La actuación se describe separadamente mediante empuje colectivo en N, momentos en Nm y velocidad de rotor en rad/s. La saturación y degradación son porcentajes de muestras y también se añaden los porcentajes de recorte de norma e inclinación de la fuerza predicha por las redes neuronales.

Una trayectoria se considera exitosa si la terminación es de carácter temporal, o finalización de trayectoria o de composición. Las referencias infinitas deben alcanzar el límite temporal mientras **waypoint** debe completar su ruta.

3.5. FLUJO EXPERIMENTAL Y REPRODUCIBILIDAD

Diseño global del procedimiento

El procedimiento experimental sigue diez fases. Primero se ejecutan comprobaciones iniciales del simulador y de escenarios representativos. Después se genera el conjunto clásico inicial, se diagnostican los PD por familia y, si es necesario, se busca un PD especializado para cada familia. Con esos controladores con parámetros fijados se regeneran los escenarios clásicos y se ejecutan las simulaciones tanto nominales como de transferencia cruzada.

A partir de esa base se generan los datos de imitación. Las fases siguientes entrenan MLP, GRU y LSTM, evalúan su fidelidad supervisada, ejecutan la comparación cerrada sobre el conjunto de prueba, evalúan el rendimiento ante la serie de escenarios OOD y, finalmente, se preparan los resultados en tablas comparativas. Los comandos extensos de estas fases se remiten al [Anejo 1](#). La [Figura 3.11](#) sintetiza esta secuencia y separa conjuntos de pasos diferenciados.

El conjunto de entrenamiento interviene en el ajuste de controladores, la normalización y la optimización de pesos, validación se usa para seleccionar el checkpoint de las redes, y prueba y OOD se emplean para la comprobación final del rendimiento.

Configuración y comparación del entrenamiento

Las tres arquitecturas comparten dataset, nueve entradas, tres objetivos, 64 unidades ocultas, normalización de entrenamiento, MSE, Adam, tasa 10^{-3} , lote 64, máximo de 100 épocas, paciencia 10 y semilla 42. Comparten también el criterio de guardar el menor MSE de validación.

La comparación se realiza sobre el conjunto común de parámetros para evaluar únicamente el efecto de sustitución de la arquitectura. Adicionalmente, se realizó un estudio de sensibilidad a los parámetros de configuración para comprobar la robustez de la misma.

Evaluación en familias conocidas y transferencia clásica

La evaluación ID utiliza los 23 episodios de prueba, 3 hold, 7 círculos, 7 Lissajous y 6 waypoint. En cada escenario se ejecutan el PD nativo y las tres redes bajo idéntica configuración física. Así, se realiza una primera comparativa contra el PD especializado de cada familia, que es la referencia de mayor exigencia.

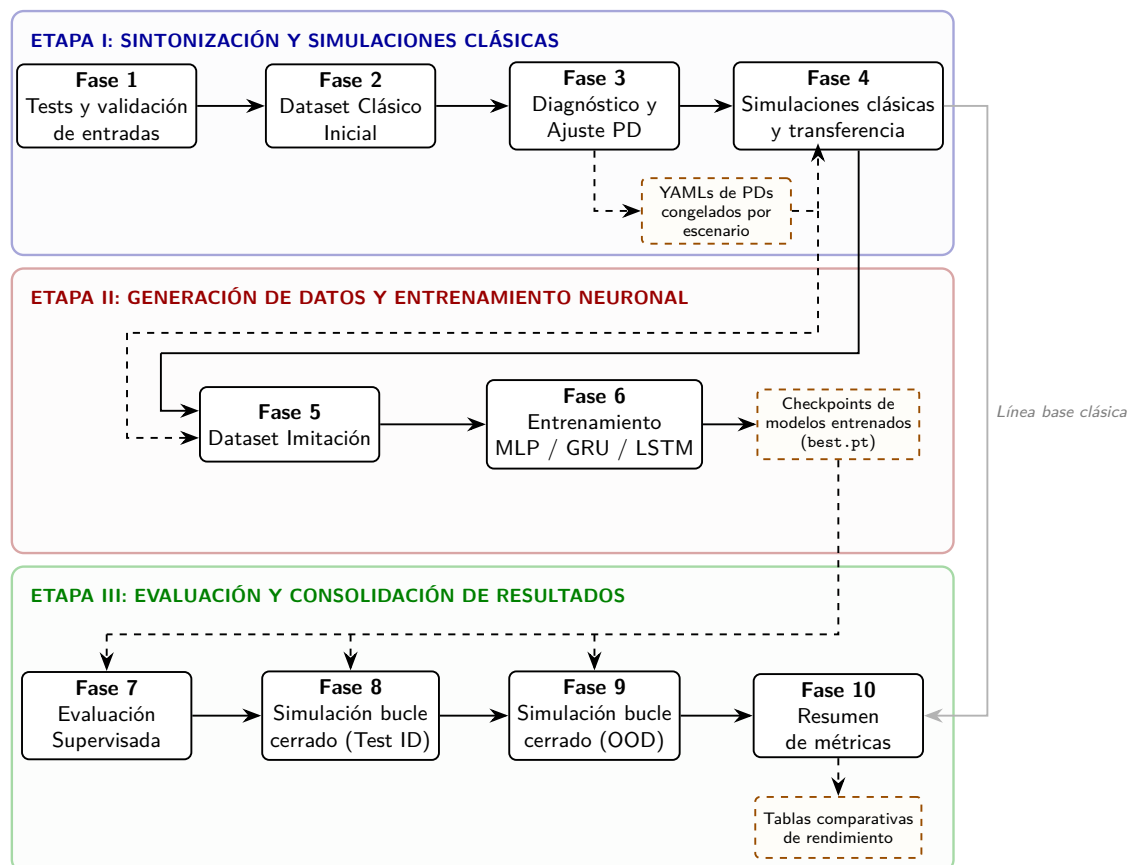


Figura 3.11. Flujo general del procedimiento experimental estructurado en 10 fases reproducibles. Se ilustran la generación progresiva de los datasets de entrenamiento, el flujo de optimización clásico, el entrenamiento neuronal y las etapas de validación y evaluación OOD. Fuente: elaboración propia.

La transferencia clásica aplica los cuatro PD con parámetros fijados a cada uno de los 23 escenarios. Esto produce $23 \times 4 = 92$ ejecuciones y una matriz controlador-familia. Las diferencias se atribuyen únicamente al cambio de controlador dado que se mantienen el resto de parámetros fijos, incluida la semilla.

Evaluación fuera de la distribución de entrenamiento

La evaluación OOD agrupa escenarios cuyos episodios no intervienen en ninguna otra fase de la implementación. Su función es comprobar qué ocurre cuando los controladores se someten a condiciones nunca vistas en la sintonización o entrenamiento de las redes. La serie OOD se organiza por nivel de novedad. En todos los casos se ejecutan las redes y los PD transferidos bajo las mismas condiciones y son los PD disponibles los que se emplean como referencia para la comparativa.

La [Figura 3.12](#) resume las trayectorias empleadas en cada nivel de evaluación. Se comienza con la variación sobre familias conocidas, después se evalúa la transferencia cruzada y finalmente se presentan escenarios OOD para estudiar la recombinación de capacidades ante geometrías nuevas. Con esta separación, los resultados en OOD se pueden interpretar según el grado de novedad introducido.

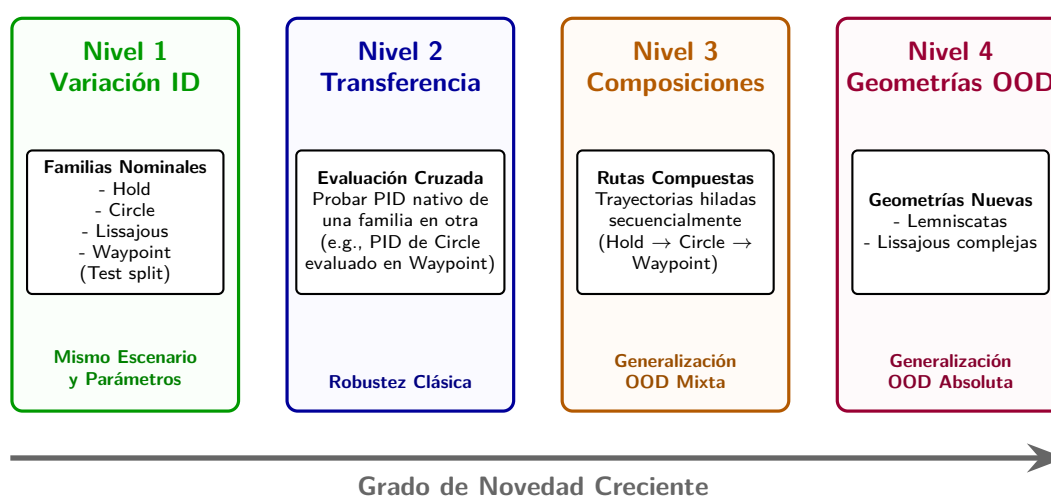


Figura 3.12. Clasificación de los niveles de evaluación y generalización experimental. Se estructuran cuatro niveles con novedad y exigencia incremental. Interpolación ID nominal, transferencia clásica, recombinación de capacidades conocidas y geometrías OOD. Fuente: elaboración propia.

Variaciones y composiciones de familias conocidas

El primer grupo OOD conserva rutas presentes en el entrenamiento, pero modifica su exigencia o las combina. Incluye Lissajous tridimensionales, más rápidas o con ruido elevado, composiciones secuenciales de hold, circle, waypoint y tramos de mayor agresividad, así como trayectorias helicoidales aproximadas mediante waypoints.

Esta separación permite evaluar dos preguntas distintas. Las variaciones de una familia conocida miden si el controlador tolera cambios de parámetros de una estructura ya presente. Las composiciones evalúan si un único procedimiento puede atravesar tramos heterogéneos sin cambiar un controlador de forma explícita. Un PD especializado puede funcionar bien en la parte para la que fue ajustado y degradarse al cambiar de geometría, mientras que las redes MLP, GRU y LSTM se evalúan como controladores únicos aplicados a toda la trayectoria.

Trayectorias completamente nuevas

El segundo grupo OOD introduce una novedad geométrica más intensa. Las lemniscatas no pertenecen a las cuatro familias de entrenamiento y obligan a seguir una curva cerrada, con cambios de curvatura y componentes verticales. La implementación usa una lemniscata de Geroni en el plano horizontal,

$$x(t) = c_x + a \sin(\omega t), \quad y(t) = c_y + b \sin(2\omega t), \quad (3.27)$$

con una oscilación vertical opcional $z(t) = c_z + A_z \sin(\omega_z t)$. En la serie OOD se emplean variantes tridimensionales, de diversas velocidades y con perturbaciones combinadas. Esta categoría es una prueba de mayor exigencia ya que cambia la estructura completa de la referencia y exige modificaciones en la curvatura. Al mantenerla separada de las otras trayectorias OOD se puede estudiar de manera independiente su comportamiento.

4. Resultados experimentales

4.1. COBERTURA Y VALIDEZ DE LA EJECUCIÓN

La comparativa usada para contrastar las hipótesis se apoya en una ejecución cerrada, reproducible y trazable. Antes de realizar interpretaciones, se expone el tamaño de la serie de evaluación, por ello la [Tabla 4.1](#) resume escenarios, controladores, ejecuciones y porcentaje de casos que completan la simulación sin sobrepasar las limitaciones establecidas. Por último, los comandos de generación y visualización se recogen en el [Anejo 1](#) y la estructura de código que produce estos artefactos se sintetiza en el [Anejo 2](#).

Tabla 4.1. Cobertura de la ejecución comparativa.

Split	Escenarios	Controladores	Ejecuciones	Completadas [%]	Límites [%]
test	23	8	184	99.5	99.5
ood	10	8	80	66.3	76.3

Los ocho controladores de la tabla corresponden a cuatro PD de las familias (**hold**, **circle**, **lissajous** y **waypoint**), tres redes neuronales (MLP, GRU y LSTM) y una referencia PD representativa, para la cual se toma el PD nativo en condiciones nominales y el que ha sido asignado para cada escenario OOD.

En la tabla se observa una clara diferencia entre las condiciones ID y OOD. En esta última aparecen una mayor cantidad de fallos en la trayectoria y superación de los límites de actuación.

4.2. REFERENCIA CLÁSICA Y TRANSFERENCIA ENTRE FAMILIAS

Antes de valorar las redes se ha de medir la transferencia de los PD especializados a familias distintas. Esta fase es de elevada importancia ya que el objetivo de las redes neuronales reside en competir contra esta transferencia.

La [Figura 4.1](#) muestra una especialización parcial. El PD de **hold** es el que peor transfiere al resto de familias, y la propia familia **hold** obtiene mejor RMSE con otros PD que con su controlador especializado. Este resultado sugiere que la sintonización de **hold**, al menos con el criterio aplicado en esta ejecución, no logra el mejor resultado de esta familia.

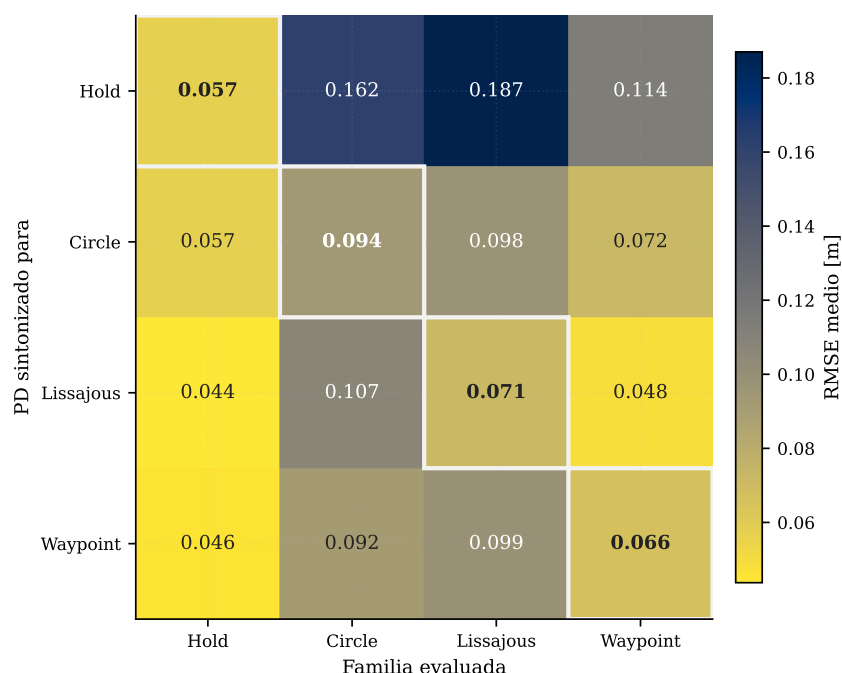


Figura 4.1. Transferencia cruzada de los controladores PD en el conjunto de **test**. Las filas indican la familia usada para la sintonización y las columnas la familia evaluada. La diagonal marca el uso del controlador especializado en su familia. Fuente: evaluación propia

En **circle**, el PD especializado ofrece un error bajo y también transfiere de forma razonable a **waypoint**, coherente con que ambas referencias exigen seguimiento continuo y regular, aunque con geometría distinta. El PD de **lissajous** presenta el mejor RMSE en su familia y también funciona muy bien en **hold** y **waypoint**, pero no reproduce el mismo margen en **circle**. Por último, el PD de **waypoint** no transfiere con la misma calidad a **circle** ni a **lissajous**, donde el seguimiento curvo y sostenido exige una respuesta distinta a la de tramos esencialmente rectos.

4.3. APRENDIZAJE NEURONAL EN CONDICIONES CONOCIDAS

La evaluación supervisada no mide seguimiento de trayectorias, sino fidelidad a la fuerza deseada calculada por los PD especializados. Así, la [Tabla 4.2](#) se introduce en primera instancia para poder responder a dos preguntas. Por un lado, cómo de bien imita cada red la señal de entrenamiento y qué ocurre posteriormente al realimentarse a la dinámica.

Tabla 4.2. Fidelidad supervisada de fuerza y recorte de inclinación asociado. Los porcentajes de recorte corresponden a la activación del límite de inclinación antes de aplicar la fuerza en el lazo interno.

Arquitectura	MSE norm.	RMSE fuerza [N]	Clip supervisado [%]	Clip cerrado [%]
MLP	0.0185	0.083	0.057	0.062
GRU	0.0102	0.053	0.000	0.000
LSTM	0.0111	0.055	0.000	0.000

GRU y LSTM imitan mejor la fuerza del PD especializado. Sin embargo, esa ventaja no se traslada de forma directa al seguimiento de posición. La [Figura 4.2](#) muestra que MLP obtiene menor RMSE en `circle` y `lissajous`, mientras que LSTM queda por detrás pese a su mejor fidelidad de fuerza. En `hold` todas las redes se mantienen en errores bajos, y en `waypoint` ninguna supera con claridad al PD representativo.

El contraste entre la [Tabla 4.2](#) y la [Figura 4.2](#) muestra que una red puede aproximar mejor la fuerza del PD especializado en las muestras y, aún así, no ser la mejor al cerrar el bucle.

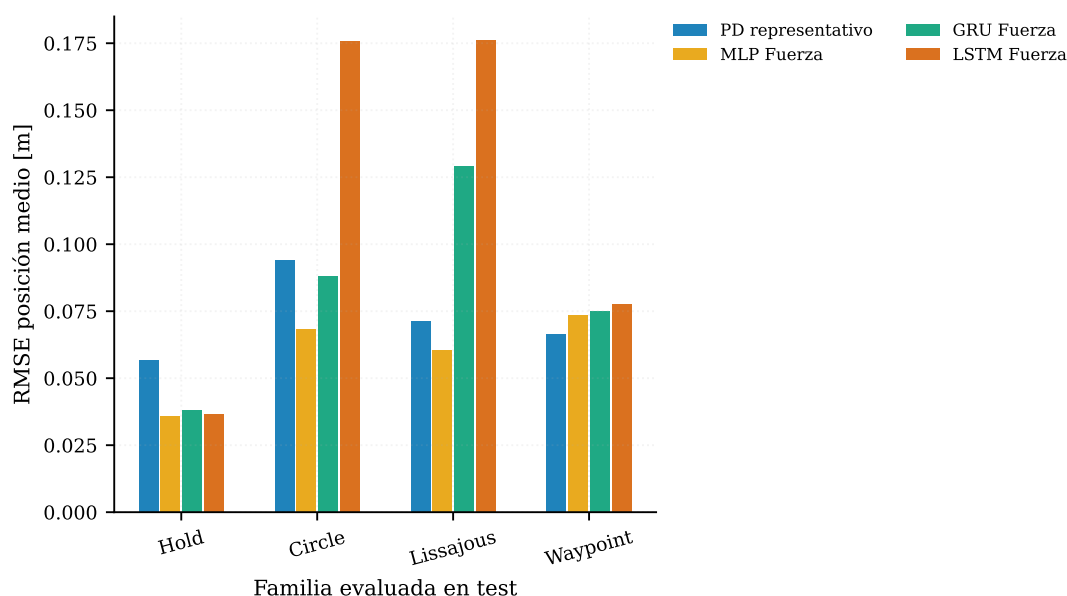


Figura 4.2. RMSE medio en condiciones conocidas para el PD representativo y las tres arquitecturas neuronales. Se muestra únicamente el seguimiento, por ser la métrica principal en esta comparación.

4.4. GENERALIZACIÓN FUERA DE DISTRIBUCIÓN

La evaluación OOD es la parte más exigente de la comparativa ya que somete a los controladores a geometrías, velocidades y combinaciones de perturbaciones no presentes de forma directa en el conjunto de entrenamiento. En esta serie, el PD representativo se toma del PD especializado más cercano al escenario: `lissajous` para `lemniscate`, `composite` y `lissajous`, y `waypoint` para las hélices.

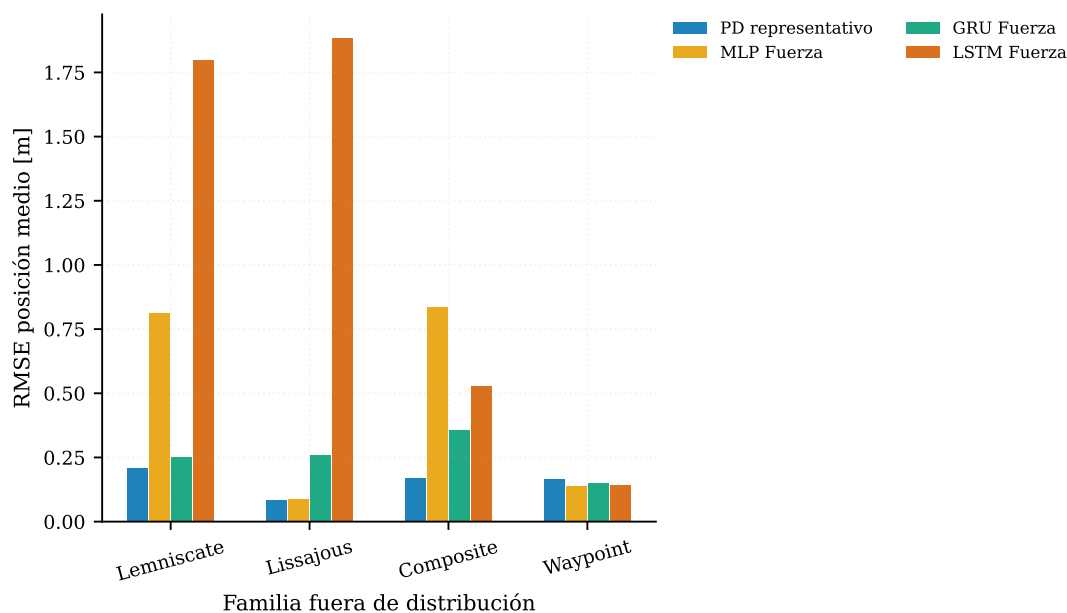


Figura 4.3. RMSE medio fuera de distribución por familia de escenario.

La [Figura 4.3](#) agrega primero por familia OOD para identificar tendencias generales. Muestra que el PD representativo conserva un comportamiento competitivo en varios grupos OOD. También confirma que LSTM vuelve a ser la arquitectura con mayor error medio en las familias más exigentes. GRU obtiene mejor resultado que MLP en las familias completamente nuevas de `lemniscate` y `composite`, mientras que MLP se mantiene especialmente competitivo en las variantes OOD más próximas a familias conocidas, como `lissajous` y `waypoint`.

Esa agregación de alto nivel puede ocultar una gran variabilidad. Por ello la [Figura 4.4](#) desciende al nivel de escenario y muestra qué combinaciones concretas de controlador y trayectoria concentran el error.

El desglose por escenario de la [Figura 4.4](#) matiza las medias de [Figura 4.3](#). La mayoría de escenarios mantiene errores reducidos, pero los casos que combinan lemniscatas, transiciones compuestas o exigencia dinámica elevada concentran los fallos. En particular, MLP aumenta mucho el error en escenarios compuestos con lemniscata, mientras que LSTM se degrada de forma clara en trayectorias rápidas o con cambios geométricos pronunciados. El patrón sugiere que la memoria temporal no aporta una ventaja uniforme y puede introducir sensibilidad adicional cuando el estado visitado se aleja de la distribución de entrenamiento.

El siguiente paso es distinguir entre error y fallo en la simulación. La [Figura 4.5](#) agrupa las causas de finalización para analizar si la degradación procede de actitud, saturación, tiempo máximo u otras condiciones.

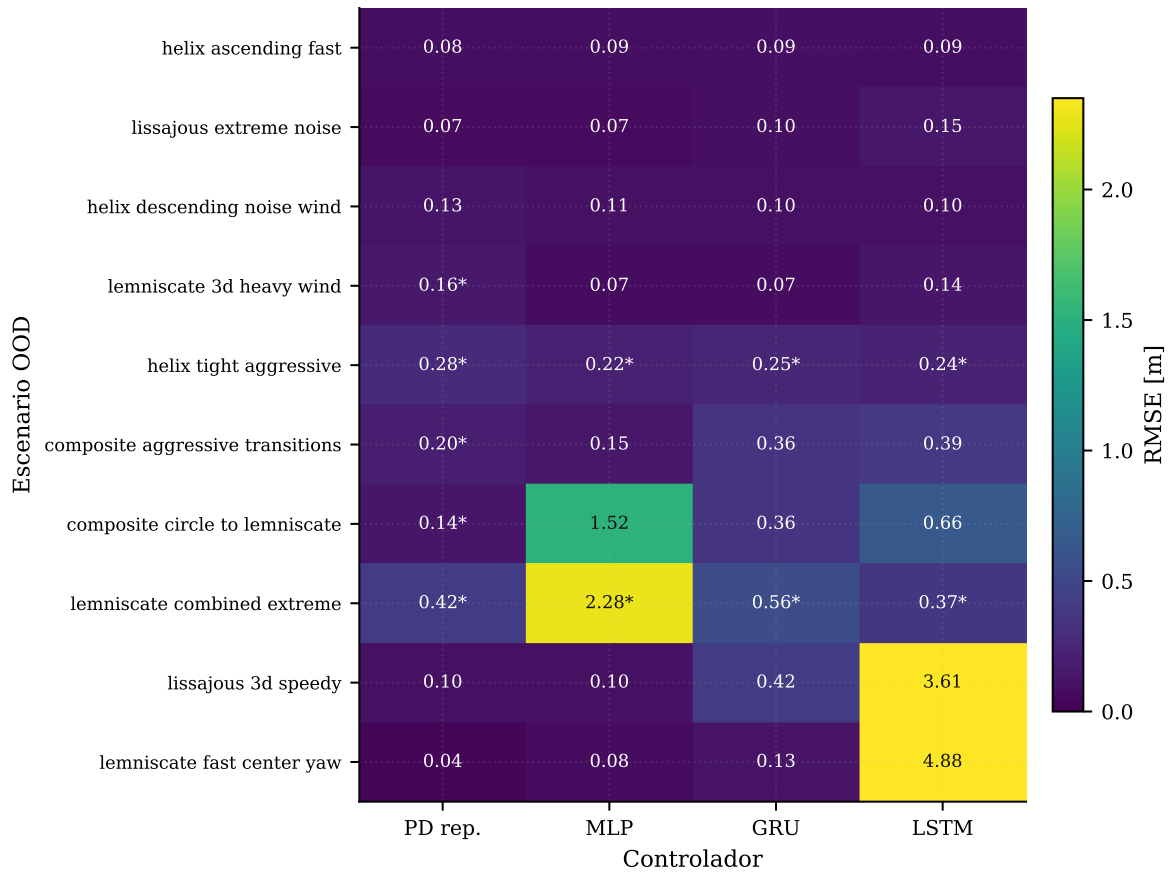


Figura 4.4. Desglose OOD por escenario y controlador para las cuatro referencias principales. El número de cada celda es el RMSE en metros; el asterisco indica que la ejecución no se completó aunque el episodio se conserva en la agregación.

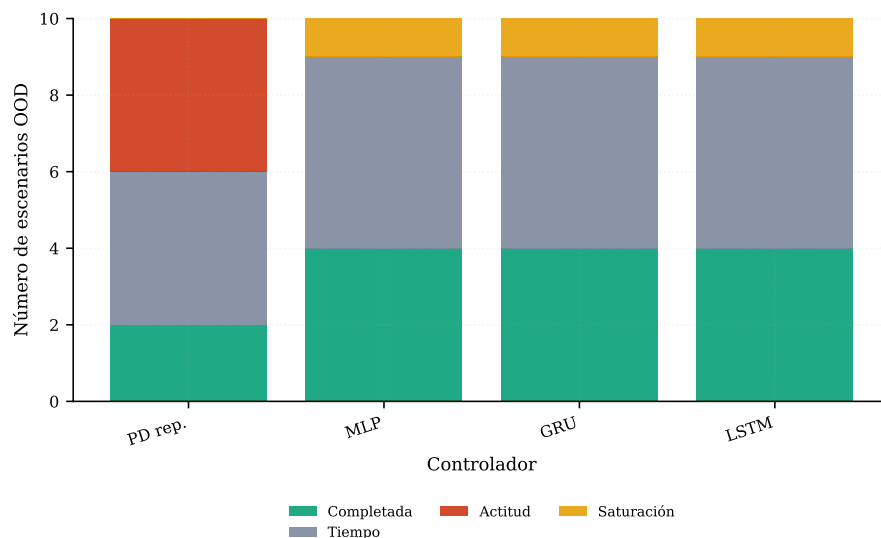


Figura 4.5. Tipos de terminación en la serie OOD para los controladores principales.

El PD representativo acumula fallos de actitud en varios escenarios OOD, mientras que las redes concentran sus fallos en saturación. Además, la cantidad de errores es mayor en la implementación clásica, lo cual es relevante para emitir una conclusión.

4.5. CASO REPRESENTATIVO DE ARQUITECTURA

La [Figura 4.6](#) ilustra un caso concreto en el que dos arquitecturas con la misma interfaz producen comportamientos muy distintos. MLP presenta un error inicial mayor, pero estabiliza el seguimiento y completa la geometría de forma razonable. LSTM, por el contrario, se separa de la referencia durante toda la simulación. Este resultado refuerza la necesidad de evaluar el comportamiento trayectoria por trayectoria y por ello se presenta en el [Anejo 3](#) un conjunto de representaciones de geometrías así como información de telemetría.

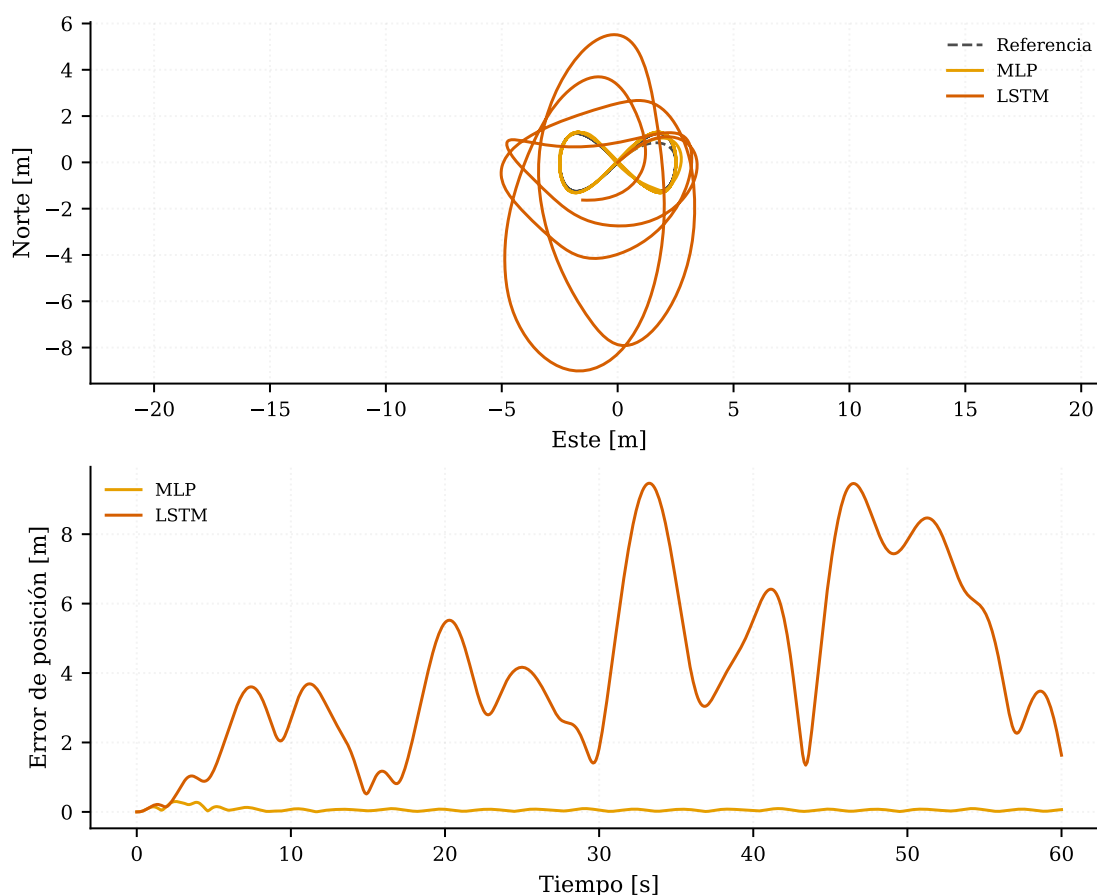


Figura 4.6. Escenario OOD `lemniscate_fast_center_yaw`: comparación visual entre MLP y LSTM. El panel superior muestra el plano horizontal y el inferior el error de posición a lo largo del episodio.

4.6. ACTUACIÓN, PROTECCIONES Y LÍMITES COMPARTIDOS

Los resultados observados no dependen únicamente de la red. El lazo interno clásico, el mixer, la saturación de actuadores y los recortes de fuerza limitan las órdenes que finalmente llegan al vehículo. En la [Figura 4.7](#), la degradación del mixer mide el porcentaje de tiempo en que se ha de intervenir sobre el colectivo para respetar límites de actuación y los porcentajes de clipping indican cuánto se recortan las fuerzas antes de pasar al lazo interno.

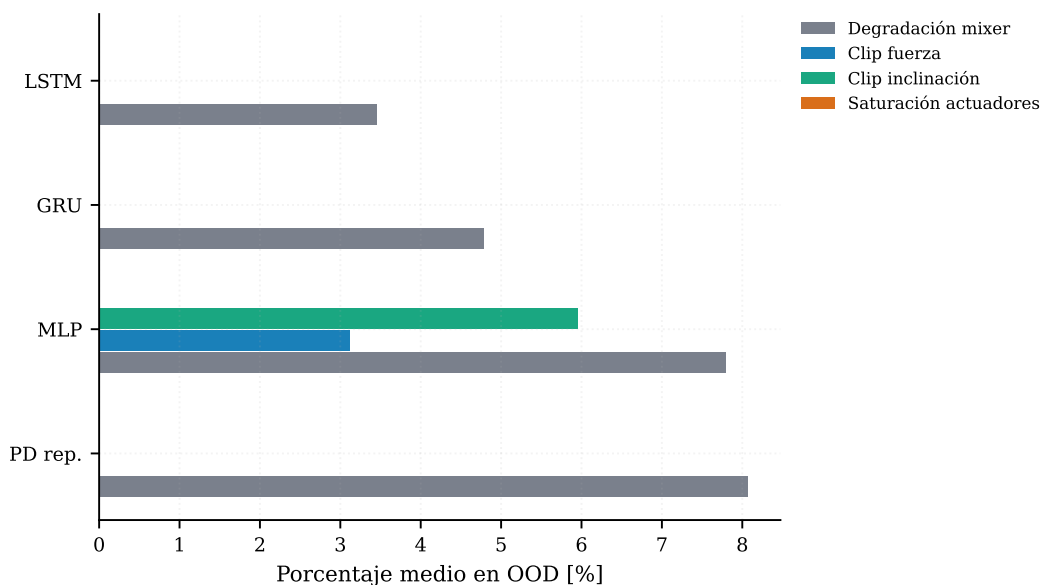


Figura 4.7. Activación media de protecciones y límites de actuación en OOD. La figura separa error de seguimiento, demanda de control y restricciones compartidas por la arquitectura de control.

MLP es la única de las tres redes que activa clipping de fuerza e inclinación de forma apreciable en OOD. Este comportamiento es coherente con una arquitectura sin estado que responde a errores instantáneos con demandas más agresivas. GRU y LSTM, al incorporar historia temporal, no activan esos recortes en la misma medida, ya que observan la tendencia reciente.

4.7. DISCUSIÓN DE RESULTADOS Y LIMITACIONES

Síntesis de los resultados

Los resultados muestran que la comparación no puede reducirse a una única métrica. La transferencia de los PD especializados depende de la familia de trayectoria, algunas combinaciones mantienen errores bajos, mientras que otras se degradan de forma clara con el cambio de geometría o requisitos de dinámica. En las redes, la fidelidad

supervisada tampoco predice por sí sola el comportamiento en bucle cerrado. Aunque GRU y LSTM aproximan mejor la fuerza del PD especializado en entrenamiento, MLP obtiene errores de seguimiento más favorables en varias familias dentro de distribución, confirmando que imitar la acción del PD no equivale necesariamente a mejorar la trayectoria resultante.

Fuera de distribución, se observa un mejor rendimiento en los casos moderados mientras que los escenarios exigentes muestran limitaciones en los controladores. Las arquitecturas neuronales son competitivas en varias recombinaciones próximas a las familias entrenadas, pero no superan de forma global al PD representativo. Además, los fallos no proceden solo del error de posición; las saturaciones, el clipping y las protecciones del lazo interno condicionan la lectura de los resultados. Así, el rendimiento depende de la interacción entre familia de trayectoria, arquitectura neuronal, PD imitado y los límites del sistema.

Interpretación respecto a la hipótesis y los objetivos

La evidencia expuesta apoya la hipótesis de que sustituir varios PD especializados por una implementación neuronal común es viable en familias conocidas, pero no conduce a un comportamiento homogéneo entre arquitecturas. En **test**, MLP, GRU y LSTM completan las trayectorias y se mantienen en un rango de RMSE comparable al PD especializado. En OOD, las redes resultan competitivas en recombinaciones de dificultad moderada, mientras que las geometrías más nuevas, rápidas o exigentes muestran que la generalización depende tanto de la novedad geométrica como de los requisitos dinámicos.

No puede afirmarse una superioridad neuronal global. El éxito de simulación en OOD (66.3 %) queda claramente por debajo del obtenido en **test** (99.5 %), y los fallos no se explican solo por un aumento del error de seguimiento sino que también aparecen límites de actitud, saturación persistente y agotamiento de tiempo en trayectorias finitas.

Dependencia del PD especializado y del conjunto de datos

Las redes imitan la fuerza deseada calculada por PD especializados, no se entrenan por refuerzo ni optimizan directamente el error de posición. Por ello, su desempeño está limitado por la calidad de los PD especializados, los filtros aplicados a las simulaciones de entrada y la cobertura geométrica y dinámica del conjunto de datos. Si el PD especializado produce acciones subóptimas para una región concreta, la red puede reproducir tal limitación.

La comparación entre fidelidad supervisada y bucle cerrado muestra precisamente esa limitación. GRU y LSTM presentan menor error de fuerza en el entrenamiento, pero eso no se traslada de forma unánime al RMSE. Una interpretación plausible es que MLP, al no depender de una ventana temporal explícita, puede responder de forma más

local a estados que se parecen a muestras vistas durante el entrenamiento, mientras que las redes recurrentes pueden ser más sensibles cuando la historia reciente se aleja de los patrones aprendidos.

Valor y límites del enfoque con lazo interno clásico

Mantener el lazo interno clásico permite aislar la aportación de la red, de manera que la red solo es responsable de la fuerza externa deseada mientras que el seguimiento de actitud, el mixer y las limitaciones de actuación siguen un diseño clásico interpretable.

La [Figura 4.7](#) muestra que la seguridad final depende de esa combinación. Las terminaciones por saturación no indican una inestabilidad sino que se solicitan fuerzas o cambios cinemáticos que exceden los límites fijados que, si bien son necesarios, condicionan la lectura de los resultados.

Papel de la arquitectura neuronal

Los resultados indican que la arquitectura, y en particular la presencia o no de memoria temporal, afecta de forma sustancial al comportamiento. MLP obtiene errores bajos en varias familias ID y se mantiene competitiva en OOD mixto, pero es también la arquitectura que activa clipping de fuerza e inclinación con mayor claridad. Esto sugiere una respuesta más agresiva ante errores instantáneos.

GRU ofrece un compromiso sólido entre fidelidad supervisada y comportamiento en varios escenarios OOD, especialmente cuando se evalúa en familias no vistas de forma directa. LSTM, en cambio, no supera de forma sistemática a GRU y presenta fallos muy marcados en algunos escenarios rápidos o geométricamente exigentes, como el caso representativo de la [Figura 4.6](#). La memoria recurrente no se interpreta como una mejora directa, sino que aporta el contexto temporal a la vez que añade sensibilidad ante datos poco representados y acumulación de desviaciones.

Validez, limitaciones y reproducibilidad

El simulador proporciona una plataforma válida para estudiar la comparación planteada, si bien no sustituye una validación con hardware real. La serie OOD incluye diez escenarios y contiene casos de alta exigencia.

La ejecución es determinista por escenario y semilla, por lo que no se ha estimado incertidumbre estadística sobre múltiples repeticiones. Además, durante el trabajo se exploraron otras formulaciones neuronales que no se incorporan a la comparativa final. Tampoco se reentrenan las redes tras definir la serie OOD y los checkpoints permanecen fijos, de manera que se evalúa una transferencia directa y no un proceso de adaptación recurrente.

Otra limitación procede de la elección del PD representativo en OOD. Para lemniscatas, lissajous y trayectorias compuestas se usa el PD de `lissajous` y para hélices se usa el PD de `waypoint`. Esta decisión es razonable por la similitud geométrica entre las trayectorias, pero no deja de ser una aproximación. Las curvas comparten algunos rasgos y las trayectorias compuestas dependen del tramo que suponga una mayor dificultad. Finalmente, si bien se probaron diversas configuraciones de parámetros e hiperparámetros para defender las decisiones finales, no se ejecutó una búsqueda exhaustiva de hiperparámetros ni una evaluación del orden de miles de semillas.

La regeneración de escenarios, la ejecución de controladores y la generación de figuras quedan recogidas en el repositorio de GitHub. Los comandos principales se documentan en el [Anejo 1](#), de modo que las tablas y figuras del capítulo pueden reconstruirse.

5. Conclusiones y trabajo futuro

5.1. CONCLUSIONES

Este trabajo se ha desarrollado en un entorno reproducible y trazable para comparar control clásico y neuronal por imitación en el seguimiento de trayectorias de un cuadricóptero. La aportación consiste en una cadena completa desde un simulador 6DOF completamente modular hasta la generación de métricas comparativas pasando por la creación de escenarios de prueba, sintonización de PD especializados, entrenamiento de redes MLP, GRU y LSTM y su ejecución en bucle cerrado.

La hipótesis inicial se cumple de forma parcial y con matices. Las arquitecturas de carácter neuronal entrenadas para la predicción de fuerza deseada del lazo externo son viables en familias conocidas durante su entrenamiento y pueden sustituir al conjunto de controladores PD en varios casos y completar de forma exitosa su objetivo. Sin embargo, no se observa una superioridad global de las redes neuronales. En condiciones fuera de distribución emergen fallos, diferencias notables entre arquitecturas y una clara dependencia de la geometría y el nivel de exigencia dinámica de la trayectoria de referencia.

La primera conclusión aborda la especialización del control clásico. Algunos PD transferidos funcionan mejor que el PD especializado para la familia concreta, mientras que otros pierden prácticamente la totalidad del rendimiento al salir de su trayectoria de sintonización. Esto confirma que la transferencia clásica es desigual y que por tanto este conjunto de PD especializados supone una referencia necesaria para valorar la imitación neuronal. También muestra que el proceso de sintonización de los PD se podría refinar en futuras versiones, de manera más señalizada en familias en que el controlador especializado no es el mejor en sus propias trayectorias o que se comportan mejor en transferencia que en modo nativo.

La segunda conclusión trata sobre la comparación entre fidelidad supervisada y seguimiento en bucle cerrado. GRU y LSTM aproximan mejor la fuerza del PD especializado en la evaluación supervisada, pero no dominan de forma sistemática el RMSE de posición. MLP, pese a no usar memoria temporal explícita, obtiene resultados muy competitivos en varias familias ID y en OOD mixto. Por lo tanto, si bien la fidelidad supervisada es necesaria, no es suficiente para predecir la calidad del seguimiento dinámica.

La tercera conclusión se desarrolla en torno a la memoria recurrente, dado que no aporta una mejora automática. GRU ofrece un resultado sólido en varias simulaciones, mientras que la LSTM presenta fallos graves en trayectorias geométricamente exigentes y más rápidas que las que conoció en el entrenamiento. Esto sugiere que la historia puede

aportar valor en ciertos estados, pero también puede introducir una sensibilidad a estados menos representados, retardos mayores y desviaciones acumuladas. En consecuencia, la elección de la arquitectura se debería analizar junto al tipo de trayectoria y sus características concretas.

La cuarta conclusión gira en torno a la seguridad en las simulaciones, ya que pertenece al control en su conjunto, no solamente a la red neuronal. El lazo interno clásico, el mixer, la saturación de actuadores y los recortes de fuerza condicionan el comportamiento final. Esta decisión de diseño fue adecuada porque ha permitido estudiar el comportamiento de la red neuronal manteniendo controlabilidad sobre el resto del sistema, si bien ha obligado a efectuar un mayor análisis antes de atribuir observaciones a la parte neuronal.

En conjunto, el trabajo cumple su objetivo de comparar la sustitución de un conjunto de PD por una implementación neuronal de fuerza deseada, todo ello en un entorno de experimentación trazable, reproducible y de alcance coherente. Los resultados muestran que la imitación neuronal es prometedora en condiciones conocidas durante el entrenamiento y en ciertas variaciones OOD, si bien la generalización no se puede dar por supuesta.

5.2. ALCANCE Y LIMITACIONES

El alcance físico del trabajo queda definido por el simulador 6DOF descrito en la [Subsección 3.1](#). El vehículo se representa como un cuerpo rígido, la posición y la velocidad se expresan en el sistema mundo East–North–Up (ENU), mientras que la velocidad angular, los momentos y la geometría de rotores emplean el sistema cuerpo Forward–Right–Down (FRD). La actitud se propaga mediante cuaterniones y la dinámica mediante ecuaciones de Newton–Euler integradas con RK4 de paso fijo. El modelo incluye gravedad, drag lineal simplificado, viento constante, ruido gaussiano para observación de posición y velocidad, ley cuadrática de rotores, retardo, dinámica de primer orden y saturación de actuadores.

Estas elecciones son hipótesis en la fase de modelado. Se suponen masa, geometría, inercia y parámetros de rotor constantes. No se modelan flexibilidad estructural, efecto suelo, interferencia entre rotores, flapping de rotores, aerodinámica no lineal, turbulencia, descarga de batería ni variación de empuje con tensión. El viento se trata como una perturbación de carácter constante y el drag lineal como un término disipativo simplificado. La observación incorpora ruido sintético en posición y velocidad, no se incluye la variabilidad esperada de una IMU, receptor GNSS o retardos específicos de sensores reales.

El alcance de los experimentos también queda acotado. El conjunto de datos dentro de distribución contiene 150 episodios distribuidos entre `hold`, `circle`, `lissajous` y

waypoint, con 105 episodios de entrenamiento, 22 de validación y 23 de prueba. En este entorno se evalúan las familias ID, transferencia de PD entre familias y una serie OOD separada de diez escenarios con composiciones, variaciones, hélices y lemniscatas. El resumen en la [Tabla 4.1](#) reúne 184 ejecuciones en **test** y 80 ejecuciones OOD, suficientes para contrastar la hipótesis, si bien no para cubrir de manera exhaustiva todas las geometrías, perfiles de velocidad, acciones verticales, perturbaciones o maniobras que aparecerían en un vuelo real.

La comparación de controladores se limita a los PD especializados con parámetros fijados y a tres arquitecturas de redes neuronales para predecir fuerza externa; MLP, GRU y LSTM. Las redes se entrenan por clonación supervisada de la fuerza deseada calculada por PD especializados y se conserva el lazo interno clásico de actitud, el mixer, las saturaciones y los límites de fuerza e inclinación. Por ello, no se evalúa un procedimiento neuronal completo ni un controlador entrenado para minimizar por refuerzo el error de posición en bucle cerrado. La implementación neuronal de predicción de fuerza puede heredar sesgos de los PD y de la cobertura limitada del conjunto de entrenamiento. Además, el PD que se emplea como referencia en los escenarios OOD es una aproximación por similitud geométrica, no un controlador optimizado para estas geometrías.

El estudio paramétrico se ha mantenido limitado. Los PD se sintonizan por medio de un procedimiento reproducible y después se fijan sus parámetros, pero no se ejecuta una búsqueda global en el espacio de ganancias posibles ni una optimización multiobjetivo exhaustiva. De forma análoga, MLP, GRU y LSTM comparten la configuración de entradas, método de normalización, cantidad de unidades ocultas, tamaño de lote, tasa de aprendizaje, criterio de parada temprana y semilla para aislar el efecto de la arquitectura. Se han ejecutado pruebas de sensibilidad sobre el número de unidades ocultas, tamaño de ventana y algunas semillas, pero no un barrido completo de hiperparámetros, regularización, tamaño de dataset o criterios para selección de checkpoint. Tampoco se ha realizado un barrido sobre parámetros físicos del vehículo, niveles de retardo y saturación, ruido o viento.

La validez a nivel estadístico de los resultados se toma con cautela. Las ejecuciones son deterministas para un escenario y semilla dados, favoreciendo la reproducibilidad, pero no se cuenta con intervalos de confianza ni estimación sobre la variabilidad entre semillas.

Finalmente, los límites de actitud, saturación persistente, límites de fuerza e inclinación delimitan la región de validez de las ejecuciones. Estas limitaciones son necesarias para el análisis de las simulaciones pero condiciona la interpretación, parte del comportamiento proviene de la arquitectura de control completa y no solo de la red neuronal.

5.3. TRABAJO FUTURO

Una primera línea de trabajo consiste en adaptar el simulador a un vehículo concreto. Esto requiere identificar masa, inercia, geometría de rotores, límites de los actuadores, curva de empuje, retardos, saturaciones y parámetros de los sensores a partir de ensayos o documentación técnica precisa. Una vez cerrada la identificación, los resultados observados en las simulaciones supondrían una relación más directa con la capacidad de un cuadricóptero real.

La segunda línea es aumentar la fidelidad física del modelo. El simulador actual incluye un conjunto acotado de fenómenos, suficiente para realizar esta comparación, pero se pueden incorporar modelos más avanzados de aerodinámica, efecto suelo, turbulencia, interferencia entre rotores, efecto de descarga de baterías, variación del empuje con la tensión y restricciones de carácter térmico, eléctrico y espacial flexible. Estas ampliaciones permitirían estudiar si los controladores mantienen un comportamiento estable cuando la respuesta de la actuación varía durante una simulación de manera no predecible o cuando aparecen efectos que ahora se estiman de forma simplificada.

También resulta necesario ampliar y refinar el conjunto de trayectorias. En el trabajo se han usado pocas familias representativas, pero una implementación orientada al vuelo debería aumentar el número de familias, la diversidad geométrica dentro de ellas, las velocidades, los perfiles verticales y las transiciones entre diversas trayectorias. Este refinamiento debe ir acompañado de una revisión de los límites de finalización, empuje, inclinación y saturación, de manera que reflejen la exigencia real de un tipo de vehículo.

Otra línea de gran relevancia es el estudio sistemático de arquitecturas neuronales. En este trabajo MLP, GRU y LSTM se comparan bajo una serie de configuraciones comunes para aislar el efecto de la arquitectura. En una fase posterior el objetivo sería buscar la mejor configuración para el seguimiento de trayectorias, explorando anchura, profundidad, longitud de ventana, normalización, selección de entradas, regularización, semillas, tamaño del conjunto de entrenamiento y criterios de selección de checkpoints. Es factible que cada arquitectura requiera un tamaño y una ventana distintos, así como tener fortalezas distintas, aspecto que se deja entrever en los resultados discutidos.

Dentro de esa línea también debería estudiarse el método de inferencia de las redes recurrentes. Actualmente se reconstruye y procesa una ventana de 20 muestras en cada ciclo de control, sin reutilizar directamente los estados ocultos calculados en el ciclo anterior. Una alternativa sería conservar estos estados entre pasos para tanto reducir el recálculo como aproximarse a una inferencia más stateful. Sin embargo, esta modificación alteraría la relación de la red con su funcionamiento, ya que la salida pasaría a depender de un estado propagado entre ciclos internos y no solo de una ventana de muestras calculada en cada paso.

Asimismo, conviene hacer un estudio de correlación para conocer qué subconjuntos de datos aportan capacidad de generalización. No basta con añadir más trayectorias, sino que se debe analizar qué familias, parámetros y perturbaciones enriquecen realmente la distribución de entrenamiento. Esta línea permitiría distinguir si los fallos OOD provienen de una limitación geométrica, falta de estados con alta aceleración, de límites de actuación demasiado restrictivos o de una arquitectura incapaz de representar la relación entre observación, referencia y fuerza deseada en ese tipo de trayectorias.

El paso natural posterior es trasladar el enfoque a un vehículo real. Para ello no basta con cargar la red entrenada en una simulación, sino que se requiere una portabilidad a otro lenguaje de programación de más bajo nivel; además de garantizar que el cuadricóptero real ofrece las variables necesarias de estado, referencia y actuación. También habría que alinear las frecuencias de control, telemetría y reloj entre el simulador y vehículo, definir un procedimiento de pruebas progresivas y establecer límites seguros de operación.

Finalmente, una línea de cierre consiste en emplear el simulador como plataforma de validación antes del vuelo. Esto permitiría probar trayectorias nuevas, detectar regímenes donde el controlador demanda fuerzas excesivas, ajustar los límites de protección y acotar el riesgo para ensayos reales. De esta forma, el trabajo desarrollado serviría como una base para transicionar de una simulación para realizar comparativas a una evaluación experimental sobre hardware.

Bibliografía

- [1] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, 2012.
- [2] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.
- [3] T. Lee, M. Leok, and N. H. McClamroch, “Geometric tracking control of a quadrotor UAV on SE(3),” in *Proceedings of the IEEE Conference on Decision and Control*, 2010, pp. 5420–5425.
- [4] D. J. Leith and W. E. Leithead, “Survey of gain-scheduling analysis and design,” *International Journal of Control*, vol. 73, no. 11, pp. 1001–1025, 2000.
- [5] J. B. Kuipers, *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton University Press, 1999.
- [6] P. Pounds, R. Mahony, and P. Corke, “Modelling and control of a large quadrotor robot,” *Control Engineering Practice*, vol. 18, no. 7, pp. 691–699, 2010.
- [7] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. John Wiley & Sons, 2016.
- [8] K. J. Åström and B. Wittenmark, *Computer-Controlled Systems: Theory and Design*, 3rd ed. Prentice Hall, 1997.
- [9] The MathWorks, Inc., “UAV Toolbox Documentation,” <https://www.mathworks.com/help/uav/index.html>, consultado el 1 de julio de 2026.
- [10] S. Folk, J. Paulos, and V. Kumar, “RotorPy: A python-based multirotor simulator with aerodynamics for education and research,” arXiv:2306.04485, 2023. [Online]. Available: <https://arxiv.org/abs/2306.04485>
- [11] PyTorch Contributors, “PyTorch Documentation,” <https://docs.pytorch.org/docs/stable/index.html>, consultado el 1 de julio de 2026.
- [12] Astral Software Inc., “uv Documentation,” <https://docs.astral.sh/uv/>, consultado el 1 de julio de 2026.
- [13] YAML Language Development Team, “YAML Ain’t Markup Language version 1.2.2,” <https://yaml.org/spec/1.2.2/>, consultado el 2 de julio de 2026.

-
- [14] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, 2017.
- [15] TOML Contributors, “TOML: Tom’s obvious minimal language,” <https://toml.io/en/v1.0.0>, consultado el 2 de julio de 2026.
- [16] GitHub, Inc., “GitHub Docs: Repositories,” <https://docs.github.com/en/repositories>, consultado el 1 de julio de 2026.
- [17] J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” *Transactions of the ASME*, vol. 64, no. 8, pp. 759–765, 1942.
- [18] D. A. Pomerleau, “ALVINN: An autonomous land vehicle in a neural network,” in *Advances in Neural Information Processing Systems*, 1989, pp. 305–313. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87fb-Paper.pdf
- [19] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.
- [20] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [21] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [22] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in *Proceedings of the 27th International Conference on Machine Learning*, 2010, pp. 807–814. [Online]. Available: <https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf>
- [23] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.
- [24] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [25] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

- [26] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734. [Online]. Available: <https://aclanthology.org/D14-1179/>
- [27] J. A. Nelder and R. Mead, “A simplex method for function minimization,” *The Computer Journal*, vol. 7, no. 4, pp. 308–313, 1965.

Anejo 1. Comandos y trazabilidad experimental

Este anejo recoge los comandos mínimos necesarios para reconstruir la evidencia experimental usada en la memoria. No sustituye al `README.md` del repositorio, sino que deja una ruta de ejecución compacta.

Preparación del entorno

El proyecto se gestiona con `uv`. La primera orden sincroniza el entorno definido por `pyproject.toml` y `uv.lock`, la segunda ejecuta las pruebas disponibles antes de realizar ejecuciones largas.

```
uv sync
uv run pytest
```

Ejecución mínima de un escenario

La ejecución elemental toma un escenario YAML, inicia el vehículo, trayectoria, controlador, perturbaciones y condiciones de finalización, y escribe dos archivos:

- `telemetry.json` con la serie temporal y
- `metrics.json` con las métricas agregadas.

```
uv run simulador-quad run escenarios\hover_clean.yaml --no-visualization

set CASE=results\hover_clean
uv run simulador-quad plot ^
    %CASE%\telemetry.json ^
    --metrics %CASE%\metrics.json ^
    --out %CASE%\figures_report ^
    --profile report --formats png pdf
```

El mismo comando permite ejecutar escenarios sueltos durante el desarrollo. Para las figuras finales del anejo visual, sin embargo, se emplean las telemetrías del dataset clásico y no ejecuciones aisladas.

```
uv run simulador-quad run ^
    escenarios\lissajous_clean.yaml ^
    --no-visualization
```

Dataset clásico y expertos PD

El dataset clásico fija escenarios reproducibles y permite ajustar o fijar controladores PD por familia. Tras modificar ganancias o criterios de ajuste, los escenarios deben regenerarse con `--overwrite` para que las ganancias incorporadas y los manifiestos vuelvan a ser coherentes.

```
uv run python tools\generate_classic_dataset.py ^  
--version v1 --out data\classic_dataset\v1 --overwrite
```

```
uv run python tools\tune_classic_pid.py ^  
--dataset data\classic_dataset\v1 ^  
--out data\classic_dataset\v1\pids --workers 8  
%% Ejecuta en paralelo en 8 núcleos de CPU
```

```
uv run python tools\run_classic_dataset.py ^  
--dataset data\classic_dataset\v1 ^  
--no-visualization --workers 4
```

```
uv run python tools\summarize_classic_dataset.py ^  
--dataset data\classic_dataset\v1
```

Dataset neuronal de fuerza externa

El controlador neuronal aprende por imitación la fuerza deseada del lazo externo. Para ello se genera primero una serie de candidatos PD externos, se selecciona el experto seguro por escenario y se exportan las muestras de entrada y salida usadas por MLP, GRU y LSTM.

```
uv run python tools\generate_outer_force_pid_bank.py ^  
--dataset data\classic_dataset\v1 ^  
--out data\outer_force_pid_bank\v1 --workers 8
```

```
uv run python tools\generate_outer_force_dataset.py ^  
--source-dataset data\classic_dataset\v1 ^  
--pid-bank data\outer_force_pid_bank\v1 ^  
--out data\outer_force_dataset\v1
```

```
uv run python tools\train_neural_controller.py ^  
--dataset data\outer_force_dataset\v1 ^  
--architecture mlp ^  
--feature-version outer_force_min_v1 ^  
--out data\neural_control\outer_force_mlp_min_v1 ^  
--device auto
```

```
uv run python tools\evaluate_neural_controller.py ^  
--dataset data\outer_force_dataset\v1 ^
```



```
--run data\neural_control\outer_force_mlp_min_v1 ^  
--device auto
```

Las arquitecturas recurrentes se entrenan con el mismo comando, sustituyendo `mlp` por `gru` o `lstm`. En la evaluación cerrada se pasan siempre el checkpoint y la normalización correspondientes para evitar mezclar modelos y escalados.

Serie OOD y comparación cerrada

La serie OOD usada en resultados procede de `data/neural_ood/battery_v1`. Se fijó `max_duration_s=60s` en el generador para que las trayectorias trapezoidales rápidas dispongan de margen suficiente.

```
uv run python tools\generate_ood_battery.py ^  
--out data\neural_ood\battery_v1 ^  
--pid-source-dataset data\classic_dataset\v1 ^  
--overwrite
```

```
uv run python tools\run_classic_transfer_dataset.py ^  
--dataset data\neural_ood\battery_v1 ^  
--pid-source-dataset data\classic_dataset\v1 ^  
--pid-family all --include-native --workers 8 ^  
--no-visualization --rerun
```

```
uv run python tools\run_neural_outer_force_dataset.py ^  
--dataset data\neural_ood\battery_v1 --split ood ^  
--architecture mlp --device auto --workers 4 ^  
--no-visualization --rerun
```

```
uv run python tools\run_neural_outer_force_dataset.py ^  
--dataset data\neural_ood\battery_v1 --split ood ^  
--architecture gru --device auto --workers 4 ^  
--no-visualization --rerun
```

```
uv run python tools\run_neural_outer_force_dataset.py ^  
--dataset data\neural_ood\battery_v1 --split ood ^  
--architecture lstm --device auto --workers 4 ^  
--no-visualization --rerun
```

```
uv run python tools\summarize_comparison.py
```

Figuras de resultados y anejos

Las figuras comparativas del capítulo de resultados se regeneran desde el CSV. Las figuras de telemetría del anejo visual se generan desde el dataset clásico, donde se guardan las ejecuciones de los PD expertos con parámetros fijados para train, validación y test. Cuando se quiere ilustrar transferencia cruzada se usa la carpeta de resultados de transferencia.

```
uv run simulador-quad plot-comparison ^
  results\comparison_all_runs.csv ^
  --out TFG_Memoria\Figuras\resultados ^
  --formats pdf png
```

```
set ROOT=data\classic_dataset\v1\results\circle
set RUN=circle_g04_P5_combined_s1083
uv run simulador-quad plot ^
  %ROOT%\%RUN%\telemetry.json ^
  --metrics %ROOT%\%RUN%\metrics.json ^
  --out TFG_Memoria\Figuras\resultados\classic_circle_combined ^
  --profile report --formats pdf png
```

```
set ROOT=data\classic_dataset\v1\results_transfer
set RUN=circle_g04_P5_combined_s1083_with_pid_lissajous
uv run simulador-quad plot ^
  %ROOT%\%RUN%\telemetry.json ^
  --metrics %ROOT%\%RUN%\metrics.json ^
  --out TFG_Memoria\Figuras\resultados\transfer_circle_lissajous ^
  --profile report --formats pdf png
```

El mismo comando de plot se ha aplicado a cinco casos nativos: hold_g05_P5, circle_g04_P5, circle_g07_P2, lissajous_g07_P5 y waypoint_g03_P5. También se ha usado para una pareja de transferencia del caso circular. En todos ellos, el perfil report elimina títulos redundantes y conserva ejes, unidades y leyendas.

Archivos de evidencia

- results/comparison_all_runs.csv: Ejecuciones comparables usadas por las figuras principales de resultados.
- results/comparison_summary.csv: Agregación por controlador, split y familia.

- `results/evidence_manifest.csv`: Procedencia de datasets, checkpoints y decisiones de inclusión.
- Archivos de series temporales de los PD expertos nativos en `data/classic_dataset/v1/results/**/*.telemetry.json`.
- Archivos de métricas y metadatos de esas ejecuciones en `data/classic_dataset/v1/results/**/*.metrics.json`.
- Ejecuciones de transferencia cruzada entre familias de PD en `data/classic_dataset/v1/results_transfer/**/*.`
- Directorio de figuras de la memoria en `TFG_Memoria/Figuras/resultados/`.

Anejo 2. Arquitectura del código y fragmentos relevantes

Este anejo complementa la descripción metodológica con una lectura de ingeniería del código. Pretende aislar los puntos que sostienen la reproducibilidad del trabajo, el contrato de simulación, la separación entre dinámica, control, telemetría y visualización, y la forma en que se integra el controlador neuronal dentro del lazo clásico.

Organización funcional del paquete

El paquete `simulador_quad` está organizado de forma modular según sus responsabilidades bajo el directorio `src/simulador_quad/`:

- `dynamics`: Dinámica de cuerpo rígido y actuadores.
- `core`: Sistemas de referencia, contratos de datos y utilidades comunes.
- `control`: Implementación de los diferentes controladores.
- `trajectories`: Generación y gestión de trayectorias de referencia.
- `telemetry`: Registro estructurado de la telemetría de simulación.
- `metrics`: Cálculo de métricas de rendimiento y error.
- `visualization`: Renderizado y salidas gráficas.

Los scripts de `tools/` gestionan la generación de datasets, series de PD expertos, entrenamiento neuronal, ejecución masiva y consolidación de resultados.

Esta separación es relevante porque el trabajo compara controladores sin cambiar el entorno físico. La ejecución puede sustituir el lazo externo por una red, fijar expertos PD o ejecutar una serie OOD, pero el integrador, el mixer, los actuadores, las perturbaciones y las métricas siguen siendo comunes.

Bucle de simulación multirate

El fragmento siguiente resume el núcleo del contrato temporal. En cada ciclo se comprueban límites de seguridad, se calcula la referencia, se actualiza el control con bloqueo de orden cero, se aplica el sistema de actuadores, se registra telemetría a una frecuencia menor y se integra la física mediante RK4. Esta estructura es la que permite fijar 100 Hz para la física, 50 Hz para control y 10 Hz para telemetría sin mezclar responsabilidades.

```
1 while True:
2     is_saturated = any(current_applied.saturation_flags)
3     term, reason = self._check_safety_termination(state, is_saturated)
4     if term:
5         break
```

```
6
7     current_ref = trajectory.get_reference_for_state(time_s, state)
8
9     if time_s - last_control_time >= self.control_dt_s:
10         obs_pos, obs_vel = self.noise.apply_noise(
11             state.position_W_m,
12             state.velocity_W_m_s,
13         )
14         current_control = controller.compute_control(time_s, current_obs,
15             current_ref)
16         current_rotor_cmd = self.mixer.compute_rotor_commands(
17             current_control.collective_thrust_N,
18             current_control.body_moments_Nm,
19         )
20         current_applied, torque_B, thrust_B = \
21             self.actuators.compute_applied_forces(current_rotor_cmd,
22             target_omega_rad_s)
23
24         if time_s - last_telemetry_time >= self.telemetry_dt_s:
25             telemetry.append(sample_from_state_control_and_reference(...))
26
27         state = rk4_step(..., thrust_B, torque_B, wind_W_m_s,
28             drag_coefficients)
```

Listing 2: Esquema simplificado del bucle de simulación.

Integración del controlador neuronal

El controlador neuronal no sustituye todo el control. La red estima la fuerza deseada del lazo externo en el sistema mundo y el controlador clásico conserva la conversión fuerza-actitud, el lazo interno, las saturaciones y el mixer. La inferencia recurrente se hace con una ventana de longitud fija, por tanto, la red no se ejecuta en modo *stateful* persistente, sino reconstruyendo la entrada secuencial en cada paso a partir de la cola FIFO de muestras normalizadas.

```
1 x = build_outer_force_min_features_from_observation(obs_state, reference)
2 x_norm = self.normalizer.normalize_x(torch.tensor(x, dtype=torch.float32)
3     )
4
5 if self.architecture == "mlp":
6     model_input = x_norm.unsqueeze(0) # [1, input_dim]
```

```
6 else:
7     self.window.append(x_norm)
8     while len(self.window) < self.sequence_length:
9         self.window.appendleft(x_norm)
10    model_input = torch.stack(list(self.window)).unsqueeze(0) # [1, 20,
    input_dim]
11
12    with torch.no_grad():
13        y_norm = self.model(model_input).squeeze(0)
```

Listing 3: Preparación de entrada para MLP y redes recurrentes.

Esta decisión facilita una interfaz común entre MLP, GRU y LSTM y evita que el estado oculto dependa de ejecuciones previas. Como contrapartida, introduce recálculo de las muestras históricas en cada inferencia recurrente.

Trazabilidad de salidas

Cada ejecución escribe `telemetry.json` y `metrics.json`. La telemetría conserva estado, observación, referencia, orden de control, comando de rotores, actuación aplicada, viento y, cuando procede, fuerzas neuronales antes y después del recorte. Las métricas agregan RMSE, error máximo, duración, causa de finalización, saturación, degradación del mixer y metadatos de procedencia.

Anejo 3. Evidencias visuales de simulación

Las figuras de este anejo tienen como objetivo mostrar la evidencia visual tras una simulación y dar contexto a las métricas agregadas visibles en el capítulo de resultados. Las telemetrías usadas en este anejo proceden del dataset clásico `data/classic/dataset/v1/results`. Por tanto, corresponden a los PD especializados con parámetros fijados que alimentan las fases de entrenamiento, validación y prueba. Al final se añade un ejemplo de `results/transfer` para ilustrar la transferencia cruzada de esos mismos PD.

Atlas de trayectorias

Antes de entrar en señales temporales, la [Figura 5.1](#), la [Figura 5.2](#) y la [Figura 5.3](#) sitúan visualmente la variedad de referencias. Las dos primeras figuras resumen familias ID y trayectorias OOD, mientras que la tercera separa una hélice para que se aprecie su componente tridimensional.

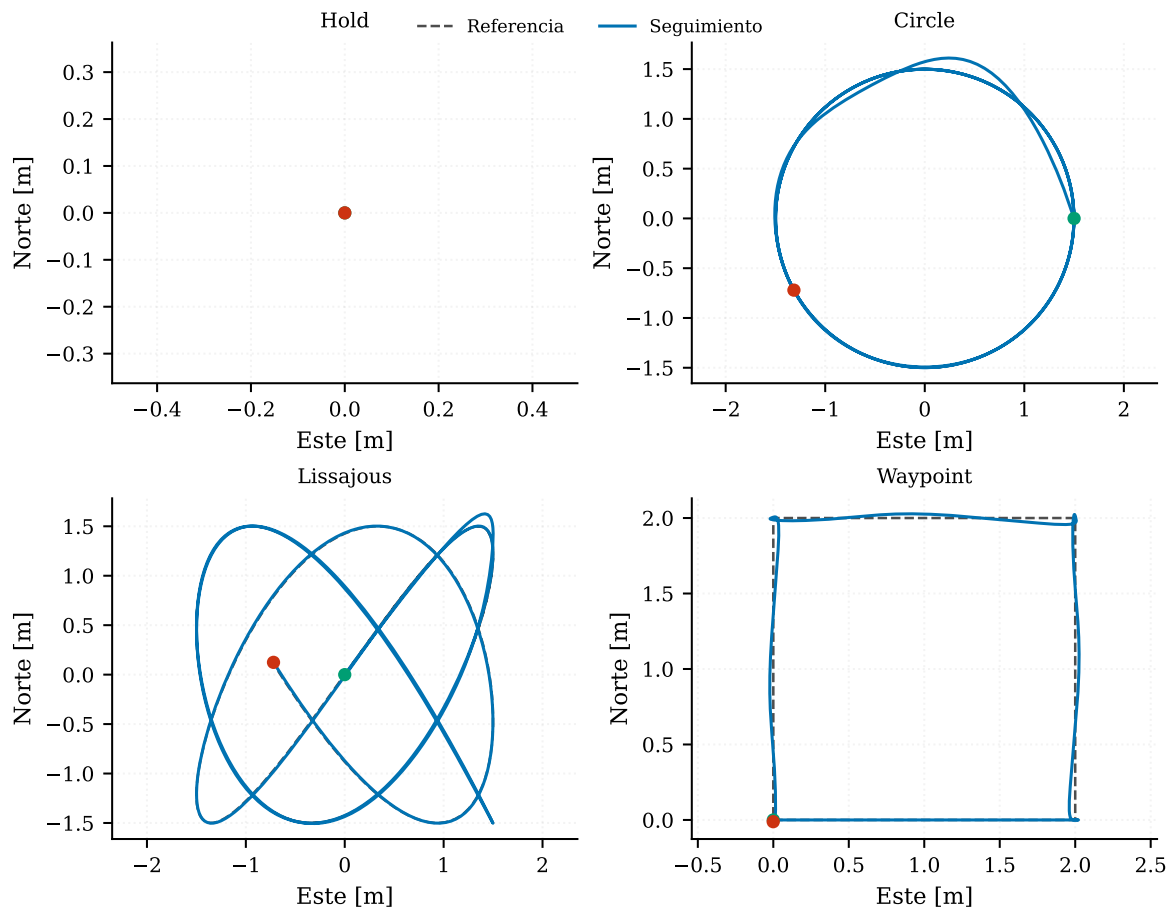


Figura 5.1. Muestra visual de familias conocidas: hold, circle, lissajous y waypoint. Fuente: elaboración propia.

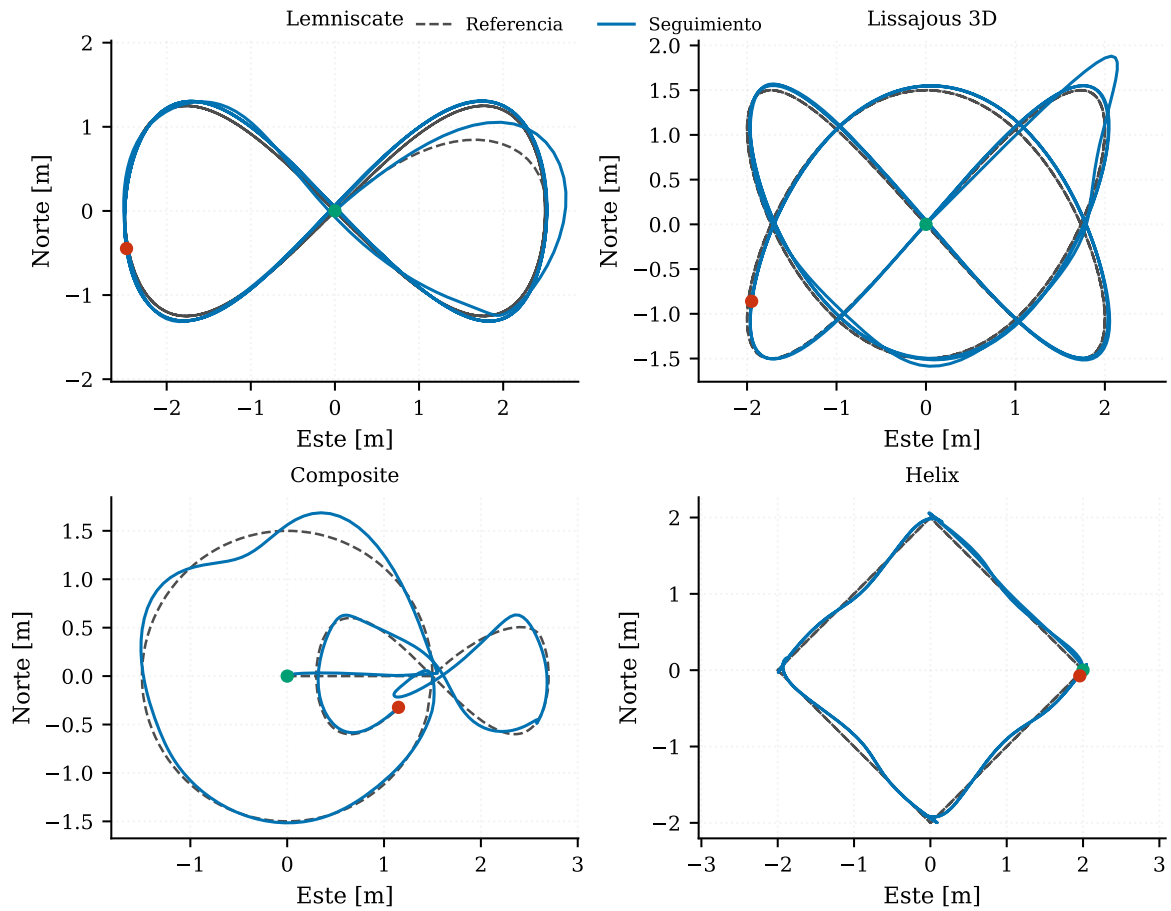


Figura 5.2. Muestra visual de trayectorias fuera de distribución: lemniscate, lissajous 3D, composición y hélice. Fuente: elaboración propia.

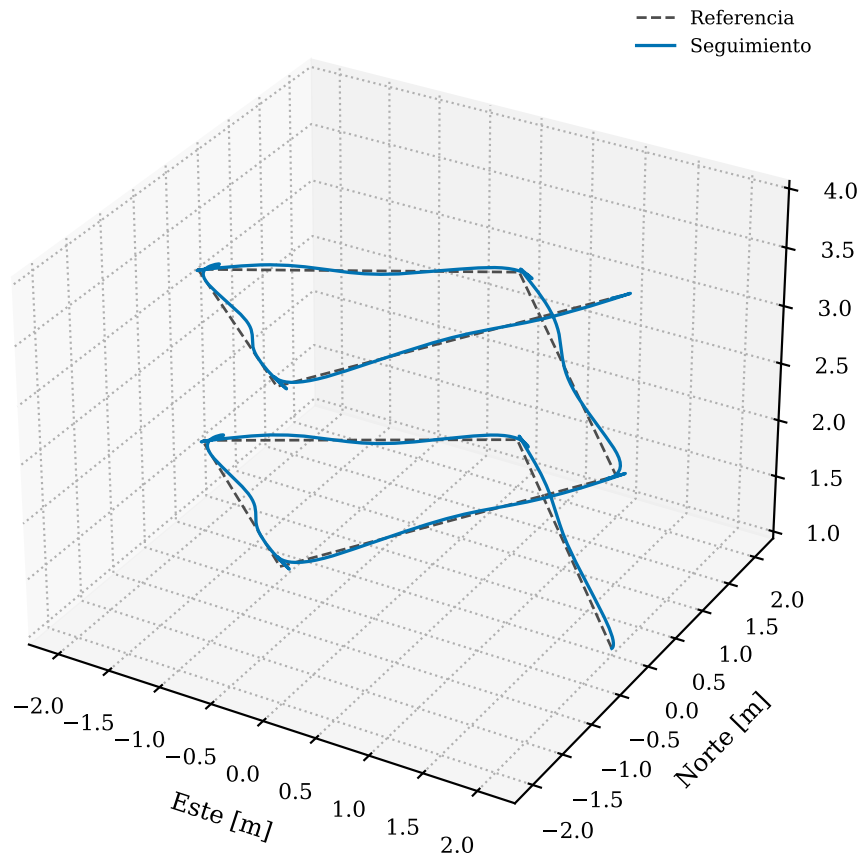


Figura 5.3. Vista tridimensional de una hélice OOD representativa. Fuente: elaboración propia.

Episodios seleccionados

La [Tabla 5.1](#) resume los episodios incluidos en las figuras siguientes. Se han escogido casos de `test` del dataset clásico para cubrir familias, perturbaciones y modos de finalización distintos. La selección busca mostrar qué se revisa de una ejecución más allá del RMSE.

Tabla 5.1. Casos del dataset clásico usados para las figuras de telemetría.

Caso	Procedencia	Motivo de inclusión
hold_g05_P5	test, PD hold	Estacionario con perturbación combinada; permite ver equilibrio, corrección frente a viento y ruido, y empuje próximo al peso.
circle_g04_P5	test, PD circle	Seguimiento curvo con perturbación combinada y trayectoria de duración fija.
circle_g07_P2	test, PD circle	Caso de círculo con viento este, útil para observar error transitorio y actitud sostenida.
lissajous_g07_P5	test, PD lissajous	Trayectoria periódica multieje con perturbación combinada.
waypoint_g03_P5	test, PD waypoint	Trayectoria finita con tramos, cambios de dirección y terminación por trayectoria completada.

Lectura geométrica

La [Figura 5.4](#) y la [Figura 5.5](#) comparan referencia y estado en el plano horizontal. La primera muestra el contraste entre una referencia estacionaria y una curva periódica, la segunda enfrenta seguimiento circular sostenido y una trayectoria por puntos, donde aparecen tramos rectos y cambios de dirección.

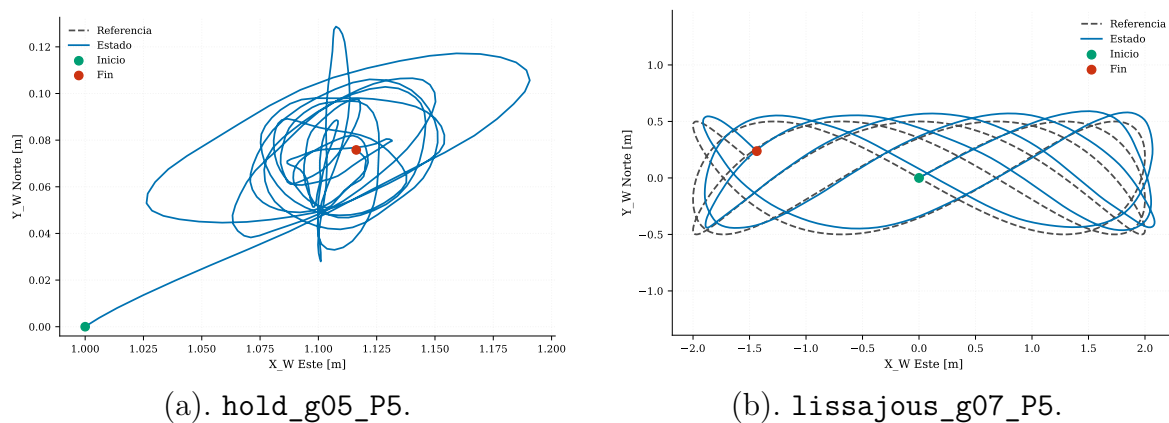
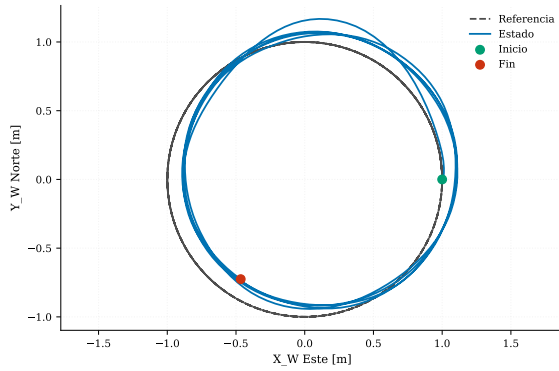
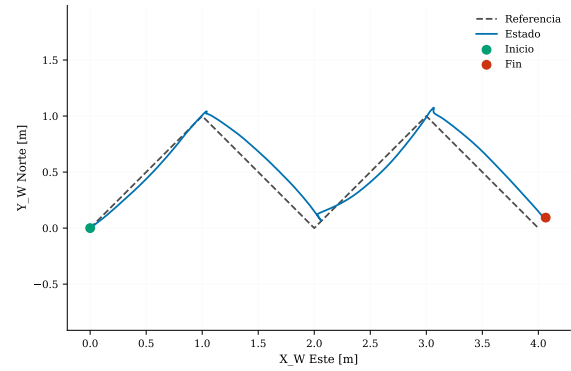


Figura 5.4. Seguimiento geométrico de dos PD especializados del dataset clásico: caso estacionario perturbado y trayectoria Lissajous multieje. Fuente: elaboración propia.



(a). circle_g04_P5.

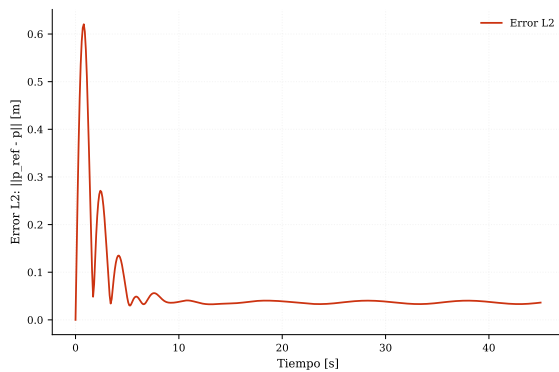


(b). waypoint_g03_P5.

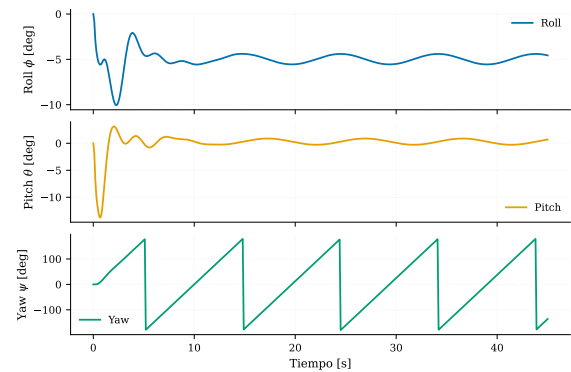
Figura 5.5. Seguimiento geométrico en una referencia circular perturbada y en una trayectoria por waypoints del split de prueba. Fuente: elaboración propia.

Error y actitud

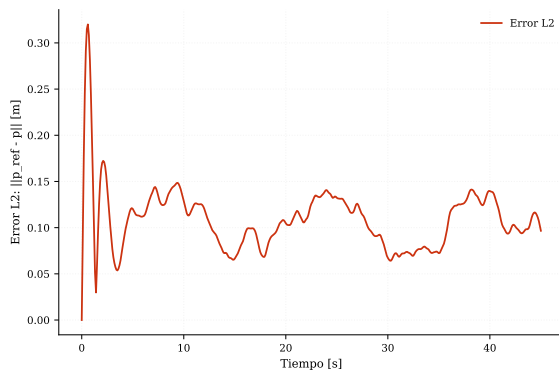
Para interpretar una simulación no basta con ver la trayectoria. La [Figura 5.6](#) combina error de seguimiento y actitud en dos episodios del dataset clásico: un círculo con viento este y una Lissajous con perturbación combinada. Así se relacionan errores transitorios con la inclinación que el lazo interno debe estabilizar.



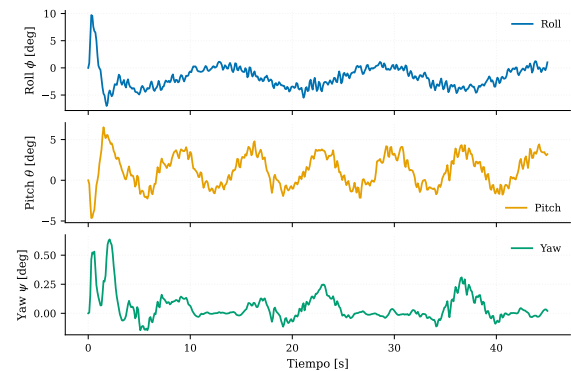
(a). Error en circle_g07_P2.



(b). Actitud en circle_g07_P2.



(c). Error en lissajous_g07_P5.



(d). Actitud en lissajous_g07_P5.

Figura 5.6. Relación visual entre error de seguimiento y actitud en dos casos del dataset clásico con perturbaciones. Fuente: elaboración propia.

Series temporales por eje

La lectura por ejes de la [Figura 5.7](#) resulta útil cuando la geometría en planta no muestra toda la exigencia de la ejecución. En Lissajous se aprecia la variación simultánea de altura, mientras que en waypoint se distinguen tramos de avance y mesetas asociadas a la llegada a cada punto.

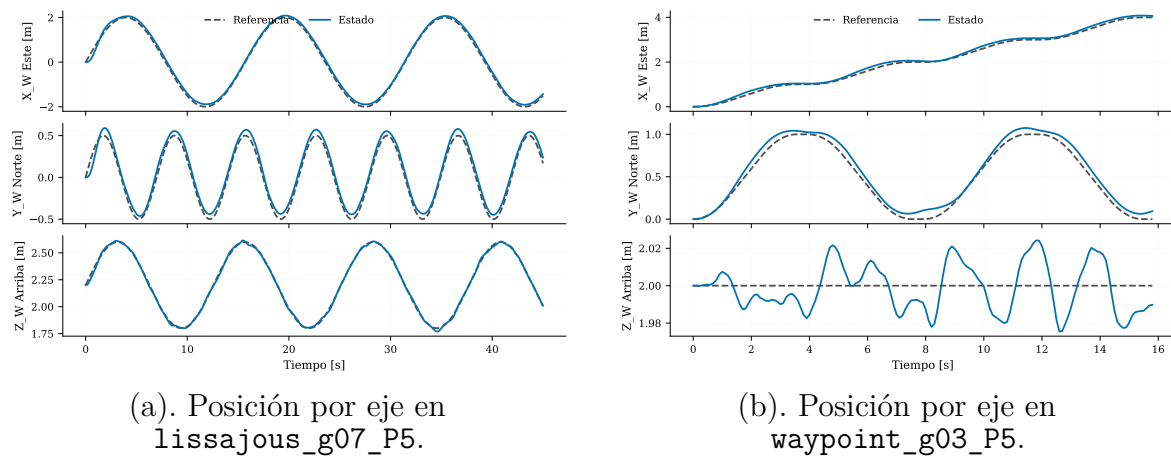
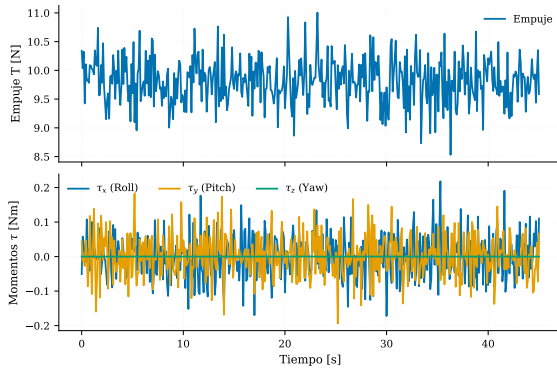


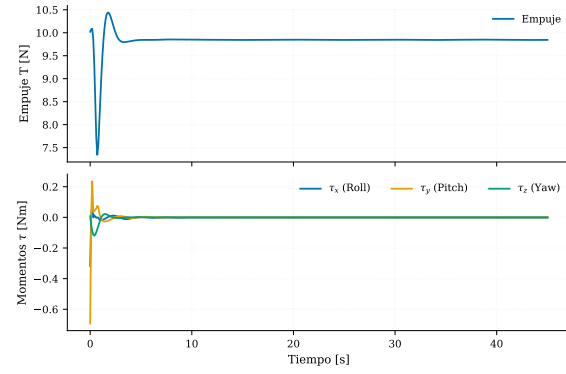
Figura 5.7. Evolución temporal de la posición frente a la referencia en dos familias con lectura geométrica diferente. Fuente: elaboración propia.

Actuación

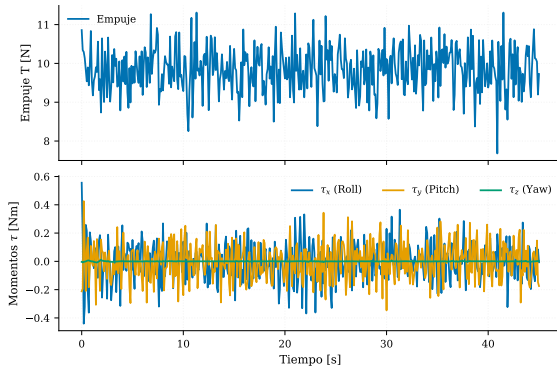
La [Figura 5.8](#) muestra telemetría de actuación en tres casos del dataset clásico. El caso `hold` permite reconocer el nivel de empuje alrededor del peso del vehículo, el círculo con viento exige correcciones sostenidas, y la trayectoria Lissajous combinada muestra cómo se reparten empuje, momentos y velocidades de rotor en una referencia multieje.



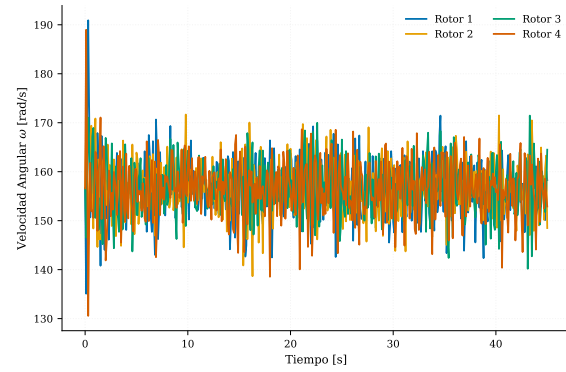
(a). Esfuerzo de control en hold_g05_P5.



(b). Esfuerzo de control en circle_g07_P2.



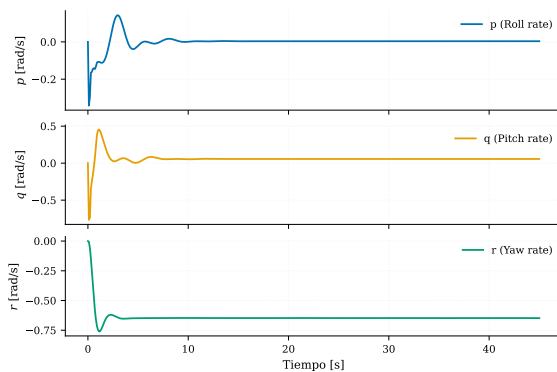
(c). Esfuerzo de control en lissajous_g07_P5.



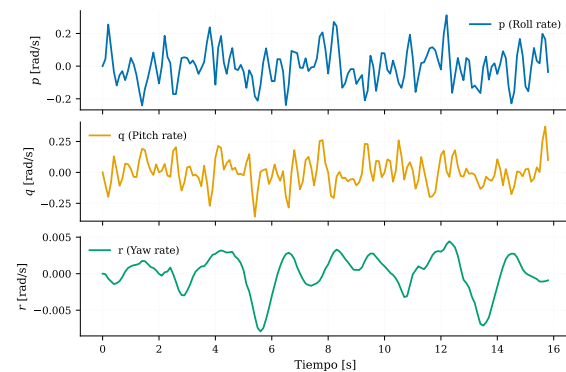
(d). Velocidad de rotores en lissajous_g07_P5.

Figura 5.8. Señales de actuación inspeccionadas tras la simulación cerrada con PD experto nativo. Fuente: elaboración propia.

Finalmente, la [Figura 5.9](#) completa la lectura de actuación con velocidades angulares en FRD. Estos datos permiten detectar si una ejecución aparentemente aceptable en posición se apoya en oscilaciones de actitud o en demandas rápidas del lazo interno.



(a). Velocidades angulares en circle_g07_P2.



(b). Velocidades angulares en waypoint_g03_P5.

Figura 5.9. Velocidades angulares del vehículo en cuerpo para dos escenarios de seguimiento del dataset clásico. Fuente: elaboración propia.

Ejemplo de transferencia cruzada

El dataset `results_transfer` permite ejecutar un mismo escenario con PD de otras familias sin reajustar sus ganancias. La Figura 5.10 muestra el escenario `circle_g04_P5` con su PD nativo de círculo y con el PD de Lissajous. El objetivo es ilustrar lo que ocurre, y cómo lissajous falla al comienzo.

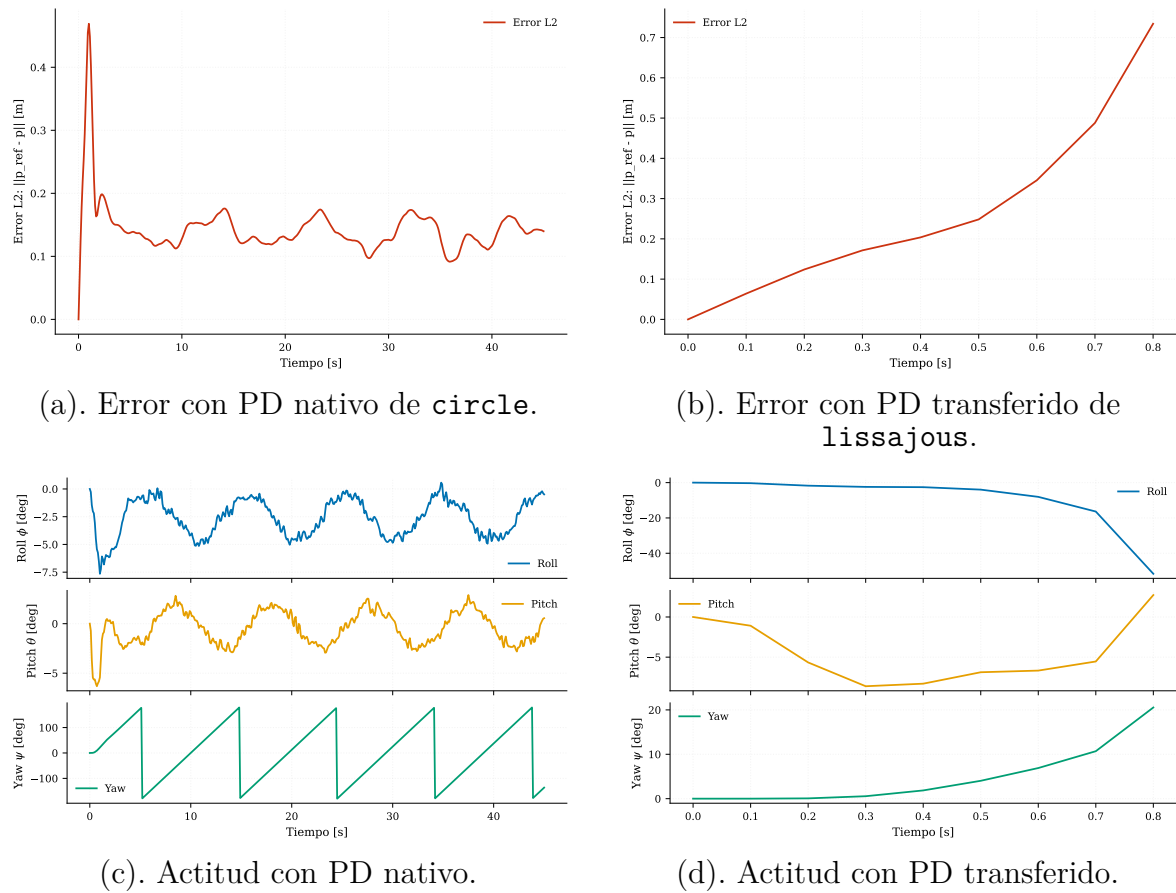


Figura 5.10. Inspección visual de una transferencia cruzada en el escenario `circle_g04_P5`: comparación entre el PD nativo y un PD de otra familia. Fuente: elaboración propia.